Visvesvaraya Technological University

Jnana Sangama, Belagavi – 590018

A Project Work Phase-2 (BCB786)

on

"MULTI-MODEL BLOCKCHAIN BASED VOTING SYSTEM"

Project Report submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF ENGINEERING (B.E)

In

Computer Science and Business Systems

Submitted by

PRANAV ARUN	**1KG23CB038**
ROHITH R	**1KG23CB044**
SUPREETH N	**1KG23CB051**
SYED MOHAMMED NAQVI	**1KG23CB052**

Under the guidance of

Mr. Ramesh Babu
Associate Professor
Head of Department
Department of CS&BS

DEPARTMENT OF COMPUTER SCIENCE AND BUSINESS SYSTEMS
K.S. School of Engineering and Management
No. 15, Mallasandra, off Kanakapura Road, Bengaluru-560109

2026-2027

K. S. School of Engineering and Management

#15, Mallasandra, off Kanakapura Road, Bengaluru - 560109

Department of Computer Science & Business Systems

CERTIFICATE

Certified that the Project Work Phase-2 (BCS786) entitled **"MULTI-MODEL BLOCKCHAIN BASED VOTING SYSTEM""** is a Bonafide work carried out by: **Pranav Arun (1KG23CB038), Rohith R (1KG23CB044), Supreeth N (1KG23CB051), Syed Mohammed Naqvi (1KG23CB052)** in partial fulfilment for VII semester B.E., Project Work in the branch of **Computer Science and Business Systems** prescribed by **Visvesvaraya Technological University, Belagavi,** during the period of August 2026 to December 2026. It is certified that all the corrections and suggestions indicated for internal assessment have been incorporated. The Project Work Phase-2 Report has been approved as it satisfies the academic requirements in report of project work prescribed for the Bachelor of Engineering degree.

.....	.....	.....
Signature of the Guide	**Signature of the HOD**	**Signature of the Principal**
[Mr. Ramesh Babu N] Associate Professor, Head of Department, Department of CS & BS, KSSEM, Bengaluru	**[Mr. Ramesh Babu N]** Associate Professor, Head of Department, Department of CS & BS, KSSEM, Bengaluru	**[Prof. Suresh Ramaswwamyreddy]** I/c Principal, KSSEM, Bengaluru

Examiners	**Name**	**Signature**
Internal Examiner	.....	.....
External Examiner	.....	.....

DECLARATION

We, the undesigned students of 7th semester, Computer Science & Business Systems, KSSEM, declare that our Project Work Phase-2 entitled "**Multi-Model Blockchain based Voting System**", is a Bonafide work of ours. Our project is neither a copy nor by means a modification of any other engineering project.

We also declare that this project was not entitled for submission to any other university in the past and shall remain the only submission made and will not be submitted by us to any other university in the future.

Place: Bengaluru
Date :

Name and USN		**Signature**
PRANAV ARUN	**(1KG23CB038)**	
ROHITH R	**(1KG23CB044)**	
SUPREETH N	**(1KG23CB051)**	
SYED MOHAMMED NAQVI	**(1KG23CB052)**	

ABSTRACT

The increasing demand for secure, transparent, and trustworthy voting systems has exposed significant limitations in traditional methods such as paper ballots and electronic voting machines. These systems are often vulnerable to issues including voter impersonation, duplicate voting, centralized control, and a lack of transparency in vote counting, all of which can undermine election integrity and reduce public confidence in democratic processes. To address these challenges, this project proposes a Multi-Modal Biometric Blockchain-Based Voting System that combines advanced biometric authentication techniques, such as fingerprint and retina recognition, with decentralized blockchain technology. By leveraging multi-modal biometrics, the system ensures that each voter is uniquely and accurately identified, strictly enforcing the principle of one person, one vote. At the same time, blockchain technology provides a secure, immutable, and transparent ledger for recording votes, preventing tampering and unauthorized modifications. Cryptographic mechanisms are integrated to protect voter privacy while still enabling verifiable and auditable election results. This approach not only enhances security and reliability but also promotes accountability and trust in the electoral process. Overall, the proposed system offers a modern, efficient, and highly secure solution suitable for digital governance and future election infrastructures.

Keywords: Multi-modal biometrics, blockchain technology, biometric authentication, voting systems, election integrity, decentralized ledger, cryptography, digital governance, voter privacy, immutable ledger.

ACKNOWLEDGEMENT

We extend out heartfelt gratitude to the **MANAGEMENT, KAMMAVARI SANGHAM**, for their constant encouragement and for providing the necessary infrastructure and resources that enabled the successful completion of this program.

We extend our heartfelt gratitude to **Prof. Suresh Ramaswwamyreddy**, Principal/Director, K.S. School of Engineering and Management, for his visionary leadership and continued support throughout the execution of this program.

We are grateful to **Mr. Ramesh Babu N**, Head, Department of Computer Science and Business Systems at K.S. School of Engineering and Management, for his guidance and for facilitating a conducive environment for carrying out this work

We also deeply thankful to **Mrs. Jayashree LK**, Associate Professor, and Project Phase 2 coordinator, K.S. School of Engineering and Management, for her valuable guidance and boundless cooperation and help extended during this work.

We also express our heartfelt gratitude to **Mr. Ramesh Babu N**, Associate Professor and Project Phase 2 Guide, for her constant encouragement, insightful guidance, and unwavering support during the completion of this work.

Finally, we convey our heartfelt gratitude to our parents, faculty members, non-teaching staff of the department, and our friends for their encouragement, cooperation, and support, both direct and indirect, throughout our Project Phase - 2 journey.

PRANAV ARUN **(1KG23CB038)**

Date: **ROHITH R** **(1KG23CB044)**

Place: Bengaluru **SUPREETH N** **(1KG23CB051)**

SYED MOHAMMED NAQVI **(1KG23CB052)**

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	**INTRODUCTION**	**1-5**
1.1	Background	1
1.2	Problem Definition	1
1.3	Motivation	2
1.4	Objectives	3
1.5	Scope of the Project	3
1.6	Proposed System Overview	4
1.7	Significance of the Project	4
1.8	Organization of the Report	5
2	**LITERATURE SURVEY**	**6-19**
2.1	Helios Web Based Open Audit Voting [1]	6
2.2	Civitas Toward a Secure Voting System [2]	7
2.3	A Smart Contract for Boardroom Voting with Maximum Voter Privacy [3]	8
2.4	The Ballot Is Busted Before the Blockchain [4]	9
2.5	Going from Bad to Worse From Internet Voting to Blockchain Voting [5]	10
2.6	Real Time Eye Blink Detection Using Facial Landmarks [6]	11
2.7	One Millisecond Face Alignment with an Ensemble of Regression Trees [7]	12
2.8	Dlib ml A Machine Learning Toolkit [8]	13
2.9	MediaPipe A Framework for Building Perception Pipelines [9]	14
2.10	FaceNet A Unified Embedding for Face Recognition and Clustering [10]	15
2.11	Histograms of Oriented Gradients for Human Detection [11]	16
2.12	Voluntary Voting System Guidelines Version 2.0 [12]	17
2.13	Algorithms and Mechanisms Relevant to the Project	18
2.13.1	Face Detection and Landmark Estimation	18
2.13.2	Face Descriptor Comparison	18
2.13.3	Relative Blink Analysis	18
2.13.4	Random Credentials and Hashes	18
2.13.5	Contract State and Replay Prevention	19
2.14	Data Quality and Representative Evaluation	19
2.15	Evaluation and Validation	19
3	**SYSTEM REQUIREMENT SPECIFICATION**	**20-30**
3.1	Purpose of the System	20
3.2	Existing System	20
3.2.1	Limitations of the Existing System	20
3.3	Proposed System	21
3.4	Functional Requirements	22

Chapter No.	Title	Page No.
3.4.1	User Interface	22
3.4.2	Voter Registration	22
3.4.3	ID Details and Eligibility	22
3.4.4	Face Verification and Liveness Challenge	23
3.4.5	Input Validation	23
3.4.6	Session and Credential Issuance	23
3.4.7	Ballot Submission	23
3.4.8	Election State	23
3.4.9	Candidate and Election Management	24
3.4.10	Results and Confirmation	24
3.4.11	Report Generation	24
3.5	Non Functional Requirements	24
3.5.1	Performance	24
3.5.2	Reliability	24
3.5.3	Usability	24
3.5.4	Maintainability	25
3.5.5	Scalability	25
3.5.6	Security	25
3.5.7	Portability	25
3.5.8	Interpretability and Traceability	25
3.6	Hardware Requirements	25
3.7	Software Requirements	26
3.8	Software Architecture Requirements	27
3.8.1	Frontend	27
3.8.2	Backend	27
3.8.3	Biometric and Blockchain Layers	27
3.8.4	Data Layer	27
3.9	Feasibility Study	27
3.9.1	Technical Feasibility	27
3.9.2	Economic Feasibility	28
3.9.3	Operational Feasibility	28
3.9.4	Schedule Feasibility	28
3.9.5	Legal and Ethical Feasibility	28
3.10	User Roles and Responsibility Allocation	29
3.10.1	Separation During a Demonstration	29
3.11	Requirements Traceability and Acceptance Evidence	30
3.11.1	Change Control for Requirements	30
4	**SYSTEM DESIGN AND ARCHITECTURE**	**31-39**
4.1	System Architecture	31
4.2	High Level System Architecture	33
4.3	Frontend Architecture	33
4.3.1	User Interaction Layer	33
4.3.2	Registration and Camera Interface	33

Chapter No.	Title	Page No.
4.3.3	Voting and Results Interface	33
4.4	Backend Architecture	33
4.4.1	Request Handling	34
4.4.2	Image and Identity Processing	34
4.4.3	Credential and Transaction Coordination	34
4.4.4	Response Generation	34
4.5	Biometric and Contract Architecture	34
4.5.1	Stored Data and Logical Relationships	35
4.5.2	Runtime and Deployment Artifacts	35
4.6	Source Module Organization	36
4.6.1	Dependency Direction	36
4.7	API Request and Response Architecture	37
4.7.1	Response Interpretation	37
4.8	Private Database Dictionary	38
4.8.1	Schema Initialization and Evolution	38
4.9	Trust Boundaries and Data Exposure	39
4.9.1	Consequences for Review	39
5	**METHODOLOGY**	**40-49**
5.1	System Development Methodology	40
5.2	Voter Data Collection	41
5.3	Image Preprocessing	41
5.4	Face Descriptor Preparation	41
5.5	Independent Enrollment and Verification Sessions	42
5.6	Image Normalization and Identity Matching	42
5.7	Handling Unusable Camera Sequences	42
5.8	Authentication and Credential Processing	43
5.8.1	Session Authentication	43
5.8.2	One Time Voting Credential	43
5.9	Evaluation of the Authentication Workflow	43
5.10	Contract Preparation and Deployment	44
5.11	Ballot Submission Workflow	44
5.12	Election State Classification	45
5.13	Result Interpretation	45
5.14	Frontend and Backend Integration	45
5.15	Testing Methodology	46
5.16	Detailed Enrollment Procedure	47
5.16.1	Interpretation of the Consistency Rule	47
5.17	Temporal Blink Decision Procedure	48
5.17.1	Evaluation of Temporal Evidence	48
5.18	Voting Credential Lifecycle	49
5.18.1	Consumption and Public Replay Checks	49
6	**SYSTEM IMPLEMENTATION**	**50-56**
6.1	Frontend Implementation	50

Chapter No.	Title	Page No.
6.2	Backend Implementation	50
6.3	Registration Implementation	50
6.4	ID Details Processing	50
6.5	Enrollment Image Processing	51
6.6	Blink Sequence Processing	51
6.7	Face Recognition Integration	51
6.8	Vote Generation and Relay	51
6.9	Confirmation and State Persistence	51
6.10	Frontend and Backend Communication	52
6.11	Deployment Implementation	52
6.12	Error Handling	52
6.13	Session Restoration and Protected Navigation	53
6.13.1	Browser State and Server Authority	53
6.14	Input Validation at Component Boundaries	54
6.14.1	Validation Order and Error Meaning	54
6.15	Local Deployment and Configuration Agreement	55
6.15.1	Cookie and Origin Configuration	55
6.16	Error Responses and Recovery Guidance	56
6.16.1	Ambiguous Submission Outcomes	56
7	**TESTING AND RESULTS**	**57-63**
7.1	Software Testing	57
7.2	Unit Testing	57
7.3	Integration Testing	57
7.4	Functional Testing	57
7.5	Test Cases	57
7.6	Evaluation Metrics	58
7.7	Comparison of Evaluation Layers	58
7.8	Observed Source Validation	59
7.9	Existing Test Coverage and Gaps	59
7.10	Testing of Deployment	59
7.11	Result Analysis	59
7.12	Negative Authentication Test Design	60
7.12.1	Evidence and Isolation	60
7.13	Credential and Concurrent Submission Tests	61
7.13.1	Concurrency as a Separate Evaluation	61
7.14	Biometric Evaluation Protocol	62
7.14.1	Metrics and Denominators	62
7.14.2	Timing and Repeatability	62
7.15	Reproducibility and Evidence Packaging	63
7.15.1	Reporting Test Status	63
8	**RESULTS AND DISCUSSION**	**64-72**
8.1	System Output	64

Chapter No.	Title	Page No.
8.2	Authentication Performance	68
8.3	Comparative Analysis	68
8.4	Analysis of Face and Liveness Checks	69
8.5	Election State Analysis	69
8.6	Audit and Result Interpretation	69
8.7	Discussion of Findings	69
8.8	Results Table	69
8.9	Interpretation of Operational Scenarios	70
8.9.1	Roster Entry Without Face Enrollment	70
8.9.2	Recognized Voter Awaiting Approval	70
8.9.3	Expired Credential With an Active Session	70
8.9.4	Election Closed During Ballot Review	70
8.10	Reconciliation of Results and Transaction Evidence	71
8.10.1	Handling Disagreement	71
9	**APPLICATIONS AND BENEFITS OF THE SYSTEM**	**72-76**
9.1	Introduction to Applications	72
9.2	Applications.	72
9.3	Benefits of the Proposed System	74
9.4	Role of Machine Learning in Voter Authentication	75
9.5	Practical Example of System Usage	76
10	**FUTURE SCOPE AND ENHANCEMENTS**	**77-82**
10.1	Introduction	77
10.2	Improvement in Evaluation Data	77
10.3	Advanced Biometric Methods	77
10.4	Explainable Authentication Feedback	78
10.5	Election History and Reconciliation	79
10.6	Security and Data Privacy Enhancements	80
10.7	Deployment and Scalability	80
10.8	Summary of Proposed Enhancements	84
10.9	Prioritized Development Roadmap	81
10.10	Acceptance Criteria for Future Enhancements	82
11	**CONCLUSION**	**83-84**
12	**LIMITATIONS AND CHALLENGES**	**85-86**
13	**PUBLICATION OF JOURNAL**	**92-93**
13.1	Publication Position	92
13.2	Proposed Manuscript Title	92
13.3	Proposed Abstract	92
13.4	Contribution and Evaluation Scope	92
13.5	Conceptual Model for the Manuscript	93
13.6	Evidence Required Before Submission	93
	REFERENCES	**89-90**

APPENDICES

Chapter No.	Title	Page No.
	APPENDICES	**91-98**
A	**Frontend Source Code**	**91**
B	**Backend Source Code**	**93**
C	**Smart Contract Source Code**	**94**
D	**Database Source Code**	**95**
E	**Configuration and Reproduction**	**96**
F	**API Reference**	**97**
G	**Terminology and Interpretation**	**98**

TABLE OF ABBREVIATIONS

Abbreviation Full Form

Abbreviation	Full Form
ABI	Application Binary Interface
AI	Artificial Intelligence
API	Application Programming Interface
CLAHE	Contrast Limited Adaptive Histogram Equalization
CSS	Cascading Style Sheets
CVPR	Conference on Computer Vision and Pattern Recognition
EAR	Eye Aspect Ratio
ECDSA	Elliptic Curve Digital Signature Algorithm
HMAC	Hash-based Message Authentication Code
HOG	Histogram of Oriented Gradients
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ID	Identification / Identifier
IEEE	Institute of Electrical and Electronics Engineers
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
JWT	JSON Web Token
NOTA	None Of The Above
OCR	Optical Character Recognition
PDF	Portable Document Format
RPC	Remote Procedure Call
SHA	Secure Hash Algorithm

CHAPTER 1

INTRODUCTION

1.1 Background

An electronic voting application must coordinate voter registration, identity verification, eligibility, ballot submission and result presentation. A successful login alone does not establish that a person is entitled to vote. Similarly, a transaction recorded on a blockchain does not prove that the identity checks performed before submission were correct. These responsibilities require a clear separation between the voter registry, the authentication service and the election contract.

This is a Multimodal Blockchain Voting System developed as an academic prototype. The application combines a React and TypeScript web interface, a Python FastAPI service, a private SQLite voter registry and a Solidity voting contract. The browser supports registration and camera capture, while the backend verifies face information, manages sessions and submits authorized test ballots through a server-controlled relayer.

The term multimodal describes the combination of ID information, a facial template and a temporal blink signal in the voting workflow. The implemented system does not contain fingerprint or iris recognition. It also does not train an ensemble of independent voting classifiers. Its biometric component uses pretrained face-processing dependencies and compares a live face descriptor with an enrolled descriptor.

The blockchain component maintains candidate information, election state, consumed credential digests and public vote totals. It provides a record of accepted transactions on the configured local or test network. Identity review and credential issuance remain centralized responsibilities. The project therefore demonstrates integrated authentication and ballot recording, while retaining limitations in privacy, availability and presentation-attack resistance.

1.2 Problem Definition

The project addresses the need for a structured student-election demonstration in which approved voters can authenticate, select a candidate, submit one authorized ballot and inspect a transaction reference. The design must coordinate browser state, biometric processing, private eligibility records and contract rules without treating any single component as sufficient evidence of a trustworthy election. The proposed system integrates multimodal biometric authentication, including fingerprint and retina recognition, to verify voter identity before granting access to the voting interface.

The project addresses the following operational challenges:

1. Identity consistency: A submitted voter identifier must correspond to the stored face profile used during authentication.
2. Eligibility review: Enrollment must remain separate from an administrator's approval of the voter record.
3. Repeated submissions: A used voting credential must not cause a second increment in the candidate tally.
4. Session continuity: The browser must maintain an authenticated session without exposing the raw voting credential to application JavaScript.
5. Distributed state: A blockchain receipt and the associated database update must be handled consistently when a request fails or is retried.
6. Usable feedback: Camera errors, approval delays, expired credentials and transaction failures must produce understandable messages.

1.3 Motivation

The motivation is to connect several software-engineering topics in one observable workflow. A voting demonstration provides a practical setting for studying computer vision, API authorization, database transactions, blockchain state transitions and interface design. Each component has a distinct responsibility, making the project suitable for explaining how an application crosses multiple trust boundaries.

Face verification offers a way to compare a live user with an enrolled profile. A short blink sequence adds a temporal check to the submitted images. These signals can improve the structure of a demonstration, but their usefulness depends on camera conditions and the quality of enrollment. They must not be presented as certified protection against every photograph, replay or synthetic video.

The use of a contract also makes selected election rules explicit. Candidate creation is restricted to the owner, votes are accepted only during the active phase and previously consumed credential digests are rejected. Combining these rules with private eligibility checks allows the project to demonstrate both the value and the limits of blockchain-backed application design.

The primary motivation of this project is to integrate multiple software-engineering concepts into a single, observable workflow. A student-election demonstration provides a practical environment for studying computer vision, API authorization, database transactions.

1.4 Objectives

The main objective is to develop an integrated web application that verifies an enrolled voter, checks administrative eligibility and records an authorized test ballot through a blockchain contract.

The specific objectives are:

- To provide registration, voter login, administrator login, ballot and result interfaces.
- To collect multiple enrollment images and store a consistent 128-dimensional face descriptor.
- To require a fresh voter-bound challenge and a server-verified blink sequence during login.
- To separate face authentication from administrator approval of ID details.
- To maintain voter and administrator sessions using HttpOnly authentication cookies.
- To issue short-lived random voting credentials to eligible users.
- To restrict contract vote submission to the configured relayer and reject reused credentials.
- To display election state, candidate totals, turnout information and transaction references.
- To include regression tests for frontend, backend and contract behavior.

1.5 Scope of the Project

The project covers a local or testnet voting prototype, including the user interface, camera workflow, authentication API, eligibility registry, relayer, election contract and result presentation. The code includes local demo scripts, deployment configuration and an audit-report generator.

Included within the scope

- Voter registration and profile management: Provides a process for registering eligible voters and reviewing requests to replace existing face profiles.
- Administrator approval and election management: Allows authorized administrators to approve voter registrations, manage candidate information, and control the election lifecycle.
- Blink-based facial authentication and session management: Uses facial verification and blink-based checks to demonstrate identity validation.
- One-time credential validation and blockchain tally updates: Validates voting credentials before accepting a ballot and prevents the same credential digest from being reused. Successfully authorized transactions update the election contract's state, allowing the recorded vote count to be retrieved.
- Public analytics and administrator report export: Presents election statistics and voting results through a dashboard.

1.6 Proposed System Overview

Figure 1.6 specifies the application separates user interaction, identity processing and blockchain recording. The principal components and the direction of the voting workflow are shown below.

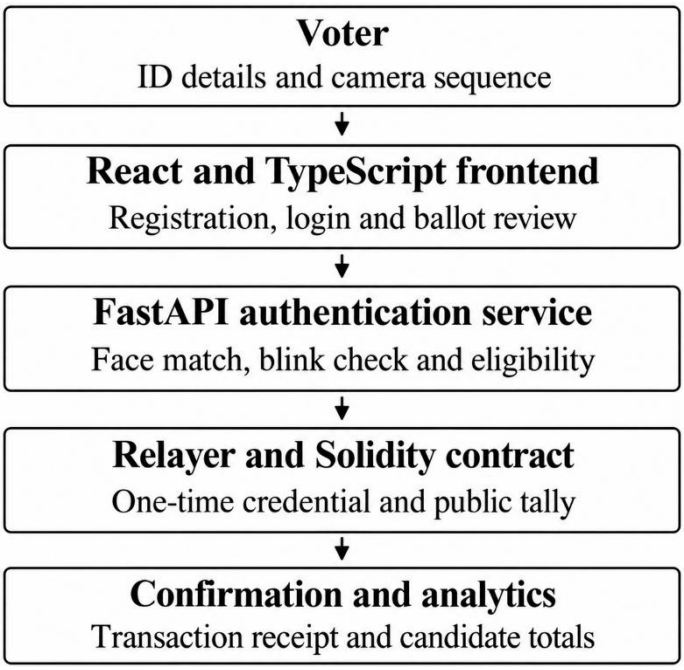


Figure 1.6: System Architecture of the Multimodal Blockchain Voting System

1.7 Significance of the Project

The project demonstrates how a browser-based application, computer-vision service, private database, and blockchain smart contract can work together to support a structured academic voting workflow. Each component performs a specific responsibility, including user interaction, biometric verification, voter eligibility management, transaction processing, and secure vote recording. Their integration provides a practical example of how modern software technologies can be combined to develop a modular digital voting application.

The significance of the project lies in its systematic handling of authorization, application state, data integrity, and user feedback. It demonstrates how access permissions can be checked before voting operations are performed and how predefined smart contract rules can govern ballot submission and vote counting. The application also provides transaction references and result summaries, helping users understand the outcome of their actions and follow the progress of the election process.

The potential benefits of the project include:

- For students: A practical example of full-stack development with biometric and blockchain components.
- For instructors: A demonstrable sequence from registration through authentication to a recorded transaction.
- For developers: An example of one-time credential handling, API authorization and contract access control.
- For researchers: A starting point for studying biometric errors, metadata privacy, recovery and user accessibility.
- For evaluators: A codebase in which documented claims can be compared with routes, tests and contract rules.

1.8 Organization of the Report

Chapter 2 reviews research and technical work relevant to verifiable voting, blockchain election protocols, face descriptors, facial landmarks and temporal blink analysis. The review distinguishes techniques implemented in the prototype from research directions that would require substantial additional development.

Chapter 3 specifies the functional requirements, non-functional requirements, development resources and feasibility considerations. Chapter 4 describes the architecture, frontend responsibilities, backend services, biometric processing, contract lifecycle and private data model. Chapter 5 explains the methodology from voter registration to transaction confirmation, including the separation between an authentication session and a voting credential.

Chapter 6 describes implementation details and the roles of the principal source files. Chapter 7 presents the existing test inventory and an acceptance-test plan. Chapter 8 discusses the implemented outputs and distinguishes source-based interface illustrations from measured execution results. No biometric accuracy, transaction throughput or unexecuted test pass rate is assumed.

Chapters 9 to 12 cover suitable academic applications, future development, conclusions and limitations. Chapter 13 presents the publication position and a possible manuscript outline. References identify the research and technical sources used, while the appendices reproduce selected implementation excerpts and configuration information from the project.

Finally, the References section identifies the research papers, technical documentation, and other sources used to develop the project.

CHAPTER 2

LITERATURE SURVEY

2.1 Helios Web Based Open Audit Voting

Authors: Ben Adida

Year: 2008

Helios describes a browser-based election system designed around public auditability. Its approach uses cryptographic election evidence so that observers can assess whether recorded ballots were tallied correctly. The paper focuses on accessible open-audit elections and explicitly considers settings with limited coercion risk.

Advantages

The work separates trust in a service operator from the evidence used to verify an election. It shows why a results page is less informative than a process that exposes meaningful verification material. Browser access also makes the system relevant to practical online participation.

Limitations

The paper's intended setting matters. Public auditability does not automatically resolve compromised client devices or pressure applied to voters. The guarantees depend on the cryptographic protocol and its assumptions; they cannot be transferred to an unrelated application simply because it publishes totals.

Future work and relevance to the project

For Prajatantra, Helios provides a useful distinction between an observable blockchain record and end-to-end election verification. The current contract increments public candidate counters after an authorized relayer call. It does not implement Helios encryption, ballot challenges or tally proofs. A future version would need a separate protocol design and verification interface before making comparable claims.

Conclusion

The project can use this work to structure its evaluation questions: what can a voter check, what can an independent observer check, and which components remain trusted? Its current transaction receipt proves only that a specified transaction was accepted on the configured chain. A broader audit claim requires additional evidence about ballot intent, eligibility and tally interpretation.

2.2 Civitas Toward a Secure Voting System

Authors: Michael R. Clarkson, Stephen Chong and Andrew C. Myers

Year: 2008

Civitas presents an electronic voting system concerned with coercion resistance, voter verification and universal verification. The design connects protocol-level reasoning with implementation analysis. The authors use security proofs and information-flow analysis and discuss the practical relationship among security, execution time and cost.

Advantages

The major advantage of Civitas is its systematic treatment of voter privacy, coercion resistance, and verifiability as distinct security properties. It demonstrates that protecting a voter's identity is not sufficient to guarantee that the vote remains private or that the election outcome can be independently verified. By connecting formal security objectives with implementation analysis, the research provides a valuable foundation for evaluating electronic voting architectures.

Limitations

Its guarantees depend on the stated assumptions and supporting election infrastructure. Reproducing the approach requires more than changing the data stored in a voting database. Complexity and operational requirements must be considered alongside the desired properties.

Future work and relevance to the project

This uses a random credential to avoid placing a direct voter identifier in the voting contract. That separation reduces direct identity exposure in contract fields, but the backend still knows the voter and records the transaction hash. The browser also displays a candidate selection and a transaction receipt. These facts do not establish the coercion-resistance properties studied by Civitas.

An appropriate research extension would define the adversary, identify trusted components and decide what evidence a voter can reveal to another person. Only after those questions are answered should a privacy protocol be selected and implemented.

Conclusion

Civitas provides a valuable research foundation for distinguishing account protection from ballot secrecy, coercion resistance, and election verifiability. Its principal relevance to the proposed project is the importance of defining security objectives explicitly and evaluating each objective against the implemented architecture.

2.3 A Smart Contract for Boardroom Voting with Maximum Voter Privacy

Authors: Patrick McCorry, Siamak F. Shahandashti and Feng Hao

Year: 2017

This work implements the Open Vote Network as an Ethereum smart contract. It investigates a self-tallying protocol for boardroom elections and evaluates the computational and financial costs of executing the protocol on an Ethereum test network. Privacy is obtained through the voting protocol rather than by hiding ordinary contract storage.

Advantages

The primary advantage of this work is its practical demonstration of a blockchain-based voting protocol with self-tallying capabilities. It illustrates how smart contracts can enforce election rules and support the verification of election outcomes. The study also considers gas consumption and transaction costs as measurable implementation factors, making it useful for evaluating the computational and financial feasibility of blockchain-based voting applications.

Limitations

The boardroom setting and protocol assumptions constrain the interpretation of the result. Its privacy guarantees are not a general property of Ethereum transactions, and a small-participant protocol should not be assumed suitable for unrestricted large elections.

Future work and relevance to the project

It has a simpler contract. It stores public candidate counts and accepts a candidate identifier together with an opaque credential digest from an authorized relayer. It does not implement the Open Vote Network's self-tallying cryptography. The two systems therefore share an execution platform while having different trust and privacy properties.

A meaningful extension would compare the prototype's transaction costs with the costs of a selected privacy-preserving protocol under the same network conditions. The comparison would require recorded receipts, gas usage, participant counts and a clear account of what each design guarantees.

Conclusion

The paper supports using Ethereum as an experimental platform for voting protocols. It also demonstrates why the protocol must be described explicitly. Public tally increments alone should not be described as private, self-tallying cryptographic voting.

2.4 The Ballot Is Busted Before the Blockchain

Authors: Michael A. Specter, James Koppel and Daniel Weitzner

Year: 2020

This security analysis examines the Voatz mobile voting application. The authors describe weaknesses through which different adversaries could affect vote integrity, availability or privacy. Their analysis also considers information leakage and the consequences of depending on third-party services for important application functions.

Advantages

The work examines the complete voting path rather than limiting its analysis to the blockchain. It provides an example of evaluating client behavior, communication and service dependencies together.

Limitations

The reported findings concern the system and version studied by the authors. They should not be treated as direct experimental findings about Prajatantra. The transferable lesson is the analysis method, not an assumption that all implementations share the same defects.

Future work and relevance to the project

It receives biometric frames from a browser, checks them in an API and submits votes with a server-held relayer key. A correct contract cannot independently verify that these earlier steps were legitimate. The authentication endpoint, enrollment process, administrator interface, session cookies and deployment settings therefore belong within the system's threat model.

Future evaluation should include altered client requests, missing cookies, expired credentials, unauthorized approvals and interrupted blockchain submissions. Such tests would complement contract replay checks and make the relationship between frontend feedback and backend authorization clearer.

Conclusion

The study motivates the adoption of a system-wide evaluation strategy for the proposed blockchain-based student voting system. Rather than evaluating the blockchain component in isolation, the entire application workflow must be examined to understand how the different modules interact and contribute to the overall system functionality. The system must establish a clear sequence through which voter identity is verified, eligibility is confirmed.

2.5 Going from Bad to Worse From Internet Voting to Blockchain Voting

Authors: Sunoo Park, Michael Specter, Neha Narula and Ronald L. Rivest

Year: 2021

The authors analyze security risks associated with internet voting and examine whether blockchain changes those risks. The article argues that client compromise, service disruption and other online threats remain significant and that a ledger does not automatically provide the evidence needed for a trustworthy public election.

Advantages

The work provides a broad assessment of assumptions that are sometimes overlooked in blockchain voting proposals. It encourages designers to ask which failures can be detected and corrected and whether convenience introduces unacceptable systemic risks.

Limitations

The article is a critical security analysis rather than a performance benchmark for this project. Its conclusions should guide scope and evaluation without being converted into invented test outcomes for the supplied code.

Future work and relevance to the project

This is appropriately treated as an academic local/testnet prototype. Its educational value comes from making authorization, biometric processing and contract behavior visible. It should not be represented as ready for binding public elections solely because candidate counts are stored on a blockchain.

An improved project evaluation would document assumptions about administrator honesty, relayer availability, camera input and database access. It would also examine whether failures leave enough evidence to determine what happened. Recovery from a transaction confirmed on-chain but not committed locally is a particularly relevant case.

Conclusion

The paper strengthens the report's scope boundary: a working demonstration is a useful engineering result, but operational election suitability requires separate evidence. The project should report concrete behavior and limitations rather than relying on broad claims of immutability or decentralization. The evaluation should therefore focus on observable system behavior and verifiable implementation evidence.

2.6 Real Time Eye Blink Detection Using Facial Landmarks

Authors: Tereza Soukupova and Jan Cech

Year: 2016

This paper estimates eye opening from facial landmarks using the eye aspect ratio, or EAR. It analyzes changes over a short sequence to detect blinks and uses a temporal classifier in the reported method. The central idea is that eye closure is a pattern across frames rather than a single static facial measurement.

Advantages

The representation is compact and interpretable. Landmark distances can describe changing eye geometry, allowing a standard camera sequence to provide a temporal signal without a separate depth sensor.

Limitations

Blink detection is not equivalent to comprehensive presentation-attack detection. Landmark quality, occlusion, lighting and facial geometry affect the measured sequence. The paper's classifier and results also cannot be attributed to a different threshold-based implementation.

Future work and relevance to the project

This calculates EAR from six points around each eye. Its backend requires a sufficiently clear bilateral close-and-reopen pattern, using neighboring open-eye samples as a baseline. The current function is a rule-based relative comparison, not the paper's trained temporal classifier. A successful blink is combined with face matching and a fresh server challenge.

Further work should evaluate genuine users and explicitly defined replay conditions. Test recordings should cover spectacles, narrow eye openings, camera angle changes and low light. Threshold changes should be evaluated against both false rejection and false acceptance rather than selected from successful demonstrations alone.

Conclusion

The paper supplies a relevant geometric foundation for temporal eye analysis. The project applies that foundation in a practical authentication sequence, while its actual resistance to replay remains an empirical question that requires separate measurement. The proposed project applies this concept through a practical authentication sequence in which facial features are analyzed and a short blink sequence is evaluated before access is granted.

2.7 One Millisecond Face Alignment with an Ensemble of Regression Trees

Authors: Vahid Kazemi and Josephine Sullivan

Year: 2014

This work uses an ensemble of regression trees to estimate facial landmark positions from image information. It studies efficient alignment, training choices and the relationship between data quantity and prediction quality. Facial landmarks provide structured locations for features such as the eyes and mouth.

Advantages

Efficient landmark estimation supports interactive computer-vision applications. The method illustrates how a sequence of learned shape refinements can produce useful geometric information without requiring a slow optimization process for every frame.

Limitations

The paper's timing and benchmark results describe its evaluation conditions. They are not measured response times for Prajatantra, which also performs image transfer, decoding, authentication and database operations. A complete application contains costs beyond landmark estimation.

Future work and relevance to the project

The backend calls the face_recognition library to obtain facial landmarks and extracts left-eye and right-eye point sets. These points are used for the blink calculation. The project does not contain a landmark-training dataset or a custom implementation of the research training procedure.

An appropriate performance study would measure landmark processing separately from face encoding and network latency. It should also report the number of usable frames, because fast execution is not useful if the resulting points are unstable or insufficient for the bilateral blink rule.

Conclusion

The work explains the role of landmark localization within the project. It supports treating geometry estimation as one processing stage, with its own quality and timing characteristics, rather than describing all biometric behavior as a single recognition algorithm. Within the proposed project, landmark localization acts as an intermediate processing stage between image acquisition and biometric verification. The camera captures facial images or video frames, and the computer-vision module identifies the relevant facial landmarks. The resulting coordinates can then be used to calculate geometric measurements, including eye aspect ratios.

2.8 Dlib ml A Machine Learning Toolkit

Author: Davis E. King

Year: 2009

The Dlib-ml paper introduces an open-source C++ toolkit for machine-learning software. It describes reusable numerical and learning infrastructure intended for engineering and research applications. The paper is relevant as a foundation for a dependency used by the project's face-recognition stack.

Advantages

A reusable toolkit allows application developers to build on established processing components. This makes it possible to focus on integration, data validation and user interaction without implementing every numerical method from the beginning.

Limitations

The toolkit paper is not an accuracy report for the specific pretrained face model used by this project. It also does not prove that a later application has correctly installed, configured or evaluated its dependencies. Those are separate implementation responsibilities.

Future work and relevance to the project

The backend requirements include dlib-bin and face-recognition-models, and the README gives installation guidance for the face_recognition wrapper. The application uses the wrapper's detection, landmark and encoding functions. It validates the returned descriptor shape before storing or comparing face information.

Reproducible evaluation should retain the versions of these dependencies and the model resources used for a test run. Replacing a model or library version may alter compatibility with existing enrolled descriptors, so a migration strategy and regression dataset would be necessary.

Conclusion

Dlib is part of the implementation foundation rather than a new learning method contributed by Prajatantra. The project's contribution is the surrounding registration, challenge, eligibility and voting workflow. Reporting these roles separately avoids attributing library capabilities or external benchmarks to the application itself. The primary contribution of Prajatantra lies in integrating these existing capabilities into a structured student-election workflow. The application connects voter registration, facial profile management, blink-based authentication challenges.

2.9 MediaPipe A Framework for Building Perception Pipelines

Authors: Camillo Lugaresi and collaborators

Year: 2019

MediaPipe describes a framework for combining perception components into processing pipelines. It addresses the practical work of building, evaluating and deploying applications that consume streams such as video. Modular components and performance analysis support iterative development across devices.

Advantages

The framework separates reusable processing operations and makes resource consumption a design concern. It is useful when a perception application must coordinate image capture, model inference and responsive feedback.

Limitations

A pipeline framework does not define the security policy of a voting application. Its outputs still need an application-specific interpretation, and client-side processing must not be treated as authoritative merely because it uses a recognized model toolkit.

Future work and relevance to the project

The repository includes a browser-side MediaPipe FaceLandmarker helper and an installation script for its assets. The helper can observe eye-blink expression scores and guide capture. However, the current VoterLogin page captures a frame sequence directly, and the server performs the decisive blink validation. The presence of the helper does not establish that every login executes its browser-side model.

Future development could consolidate the capture paths, measure resource use on representative devices and verify that local model files are packaged correctly. Any browser guidance should remain separate from the backend's independent authorization decision.

Conclusion

MediaPipe is relevant to the proposed Prajatantra student voting system because it provides computer-vision capabilities that can support responsive camera interaction and real-time facial landmark detection. Depending on the implementation, it can help identify facial features, track their positions across video frames, and provide visual guidance to users during the authentication process.

2.10 FaceNet A Unified Embedding for Face Recognition and Clustering

Authors: Florian Schroff, Dmitry Kalenichenko and James Philbin

Year: 2015

FaceNet studies mapping face images into an embedding space in which distances represent facial similarity. It uses a learned representation and a training objective based on relationships among matching and nonmatching images. Verification can then be expressed as a comparison between embeddings.

Advantages

The embedding viewpoint is useful because it separates feature extraction from the final comparison rule. It also provides a clear way to discuss the effect of a decision threshold and the difference between genuine and impostor comparisons.

Limitations

FaceNet's architecture, training data and published performance belong to that research system. does not include a FaceNet training pipeline or claim to reproduce its evaluation. Sharing an embedding dimension does not establish that two models are identical or compatible.

Future work and relevance to the project

This stores a 128-dimensional descriptor obtained through face_recognition and compares live and enrolled descriptors with the library's distance function. The default matching tolerance is 0.55. This value is an implementation setting, not an accuracy percentage or calibrated probability of identity.

Future evaluation should create separate enrollment and verification sessions, retain genuine and impostor labels, and study the threshold against both types of error. If an alternative face model is introduced, existing templates should not be mixed with its output without a demonstrated compatibility mechanism.

Conclusion

FaceNet provides an important conceptual foundation for understanding distance-based facial verification in biometric authentication systems. It represents facial images as numerical feature vectors, commonly called embeddings, in a feature space where images of the same person are expected to have smaller distances than images of different people. These representations can be compared using distance metrics to determine whether two facial images are sufficiently similar for a verification decision.

2.11 Histograms of Oriented Gradients for Human Detection

Authors: Navneet Dalal and Bill Triggs

Year: 2005

This work studies locally normalized histograms of image-gradient directions as features for human detection. It examines how descriptor construction and normalization affect detection performance. The original research task concerns human detection, and its results should not be reinterpreted as face-verification accuracy.

Advantages

The paper explains how local edge structure can be summarized in a useful image representation. It also shows that feature preparation and normalization are important parts of a detector, not merely implementation details after model selection.

Limitations

Detecting a face or a person is different from identifying an individual. A detection stage locates a region to process; it does not establish eligibility or match the image to a registered voter. Benchmark results from the original task cannot validate the project's authentication workflow.

Future work and relevance to the project

The backend explicitly requests the HOG detection mode from face_recognition. It reduces image dimensions for detection when appropriate and then rescales locations for landmark and encoding operations. More than one detected face causes a validation error in the main API path.

The project can evaluate whether image scaling improves response time without reducing the number of usable camera frames. Such an experiment should vary camera resolution, subject distance and illumination while retaining the same enrollment profiles and acceptance rules.

Conclusion

The study contributes background for the detector family used in the application. It reinforces the distinction among detection, landmark estimation, face encoding and identity comparison, each of which has a separate purpose in the implemented pipeline. The study also reinforces the importance of distinguishing four related but separate operations in a biometric authentication pipeline: face detection, facial landmark estimation, face encoding, and identity comparison. Although these operations may work together, each has a different purpose and produces a different type of output.

2.12 Voluntary Voting System Guidelines Version 2.0

Issuing body: U.S. Election Assistance Commission

Year: 2021

VVSG 2.0 is a technical framework for evaluating voting-system requirements. It covers areas such as system integrity, usability, accessibility and evidence needed for assessment. It is included here as a standards reference rather than as a research experiment or a certification held by this project.

Advantages

The guidelines encourage requirements that can be inspected and tested. Their value to an academic project is the discipline of connecting a claim to observable evidence rather than relying on interface appearance or general statements of security.

Limitations

The guidelines operate within their own jurisdiction and certification process. Referencing them does not certify Prajatantra, establish compliance with Indian election requirements or authorize use in a binding election. Such claims would require independent and applicable evidence.

Future work and relevance to the project

This includes accessible interface features, multiple language choices and explicit error states. These are useful starting points, but a full accessibility assessment would need keyboard testing, assistive-technology testing, language review and a supported alternative for users unable to complete the camera interaction.

The same evidence-based approach can guide reliability testing. Each requirement should identify the component responsible, the input conditions, the expected result and the evidence retained after execution. A screenshot of a successful demonstration is insufficient to establish behavior under faults.

Conclusion

The Voluntary Voting System Guidelines (VVSG) provide a useful reference for understanding the importance of structured requirements, systematic testing, accessibility, security, and reliability in electronic voting systems. They emphasize the need to define measurable system requirements and evaluate whether the implemented functionality satisfies those requirements under specified conditions.

2.13 Algorithms and Mechanisms Relevant to the Project

The application combines several mechanisms rather than using one algorithm to decide every aspect of voting. The source separates image processing, authentication, eligibility and contract state. Each stage must be evaluated according to its own inputs and purpose.

2.13.1 Face Detection and Landmark Estimation

The backend converts images into the required color representation, locates a face and obtains facial landmarks. It uses HOG mode for detection and rejects multiple detected faces. Landmark coordinates support the calculation of eye geometry across the submitted camera sequence.

2.13.2 Face Descriptor Comparison

A stored descriptor represents the enrolled profile. Live descriptors are validated and combined before comparison. The distance threshold determines whether the resulting descriptor is accepted as a match. A lower distance indicates greater similarity under the model; it is not a legal identity determination.

2.13.3 Relative Blink Analysis

The server compares both eyes during a proposed closure with neighboring open-eye samples. The rule requires enough valid observations and a substantial change in each eye. The implementation checks a temporal pattern, which is different from accepting a single frame with a small EAR value.

2.13.4 Random Credentials and Hashes

The service generates a random 32-byte credential and stores its SHA-256 digest privately. The relayer submits a Keccak digest of the raw credential to the contract. These digests have different roles and should not be compared as if they were the same value. A transaction hash retained by the service also creates an operational link to the public ballot record.

2.14 Data Quality and Representative Evaluation

The repository includes a student roster and application code, but it does not provide a labelled biometric benchmark with verified genuine and impostor trials. Roster rows are administrative records; they are not a dataset from which recognition accuracy can be calculated. Enrollment images and independent verification sessions must be collected under an appropriate consent and retention process.

Useful evaluation conditions include variations in lighting, camera resolution, subject distance, spectacles and repeated sessions. Enrollment images should not be reused as the only

verification samples. If the same images appear in both stages, the experiment can overstate how well the workflow performs on new camera captures.

The presentation-attack evaluation requires separately labelled conditions, including photographs, screen replays and other permitted test scenarios. The expected result and the number of attempts must be recorded for each condition. An isolated successful blink check does not establish a general resistance rate.

2.15 Evaluation and Validation

The biometric stage should report genuine acceptance, false acceptance, false rejection, failed acquisitions and processing latency. The application stage should report authorization behavior, session handling and recovery from failures. Contract evaluation should cover role restrictions, invalid candidates, election-state guards and duplicate credentials. Each result must identify the software version and test environment used.

Validation must distinguish an expected behavior from an observed result. A test case specifies what should happen; a test execution produces evidence about what actually happened. The existence of a test function in the source establishes coverage intent, while its pass or fail status requires an execution record.

The supplied project contains backend tests with mocked biometric operations, frontend tests with mocked dependencies and contract tests using a Ganache provider. These are valuable regression mechanisms. They do not independently measure real-camera performance, establish public-election suitability or prove end-to-end privacy.

A practical evaluation sequence begins with component checks and proceeds to integration. First, image validation and descriptor handling should be checked with controlled inputs. Next, enrollment, approval, login and credential issuance should be exercised through the API. Finally, the browser and relayer should be tested against a deployed contract while receipts and database state are retained.

Failure cases need equal attention. An RPC timeout may occur after a transaction has been submitted, and a local database failure may occur after chain confirmation. The evaluation must determine whether the system can identify the resulting state without allowing an accidental second ballot or reporting a false failure. The literature reviewed in this chapter leads to three design principles for Prajatantra. Authentication must be described independently of voting privacy. Blockchain recording must be distinguished from full election verifiability.

CHAPTER 3

SYSTEM REQUIREMENT SPECIFICATION

3.1 Purpose of the System

The purpose of is to provide an academic voting workflow that connects approved voter records to controlled blockchain submissions. The user interface should guide a participant through registration, camera verification, candidate review and confirmation. The administrator should be able to review eligibility and manage the election through the permissions available in the backend and contract.

The system must preserve the distinction between a face match, an eligible voter and an accepted ballot. A voter may have a registered face without an approved ID record. An authenticated voter may be unable to vote because a credential has expired or a ballot was already recorded. An election may also reject a valid credential when it is not active.

3.2 Existing System

Common voting demonstrations use either a centrally stored ballot or a direct wallet-to-contract transaction. A central application can enforce eligibility but requires trust in the operator's database and counting logic. A simple contract can expose a tally but does not independently establish the real-world identity or eligibility of the account submitting a transaction.

These observations describe architectural alternatives and are not measurements of a particular deployed election system. The purpose of the comparison is to identify the responsibilities that the proposed project must make explicit.

3.2.1 Limitations of the Existing System

1. Account identity alone may not establish that a person belongs to the permitted voter group.
2. A separately implemented registration process may not stay synchronized with ballot submission rules.
3. Public wallet transactions can expose relationships among accounts, candidate choices and timing.
4. A single successful demonstration may overlook repeated requests, expired sessions and partial failures.
5. Camera-based login can be inconvenient when lighting, device quality or user ability prevents a stable capture.

6. A frontend-only restriction can be bypassed unless the backend and contract enforce the same rule.
7. Result displays can be misleading when their data sources, timestamps or connection status are unclear.

3.3 Proposed System

The proposed system uses separate layers for presentation, authentication, private records and election state. The React frontend collects input and displays progress. FastAPI validates requests, verifies the face sequence, checks approval and manages credentials. SQLite stores private voter and credential records. The relayer submits the selected candidate and an opaque digest to the voting contract.

The contract independently limits who can submit votes and when they are accepted. It validates the candidate identifier, rejects an empty credential and rejects a digest already marked as used. A successful call increments the selected candidate count and emits a VoteCast event. Administrator functions are protected by a separate owner check.

The proposed workflow consists of the following stages:

1. Register the voter's details and a valid multi-image face profile.
2. Review the ID record and approve eligibility through the administrator interface.
3. Request a fresh login challenge and capture the blink sequence.
4. Verify the face, issue the session and prepare a voting credential when permitted.
5. Select and review a candidate in the active election.
6. Validate the credential, relay the transaction and display its returned hash.

The design is deliberately described as a prototype. It provides no independent proof that the administrator's eligibility decision is correct and does not encrypt the candidate choice on the public chain.

The smart contract acts as an independent enforcement layer for the election rules encoded within it. Backend authentication alone does not override the contract's restrictions: transactions must satisfy the applicable caller authorization, election-state, candidate-validation, and credential-reuse conditions. A successful transaction updates the contract's election state and produces a Vote Cast event. However, the transaction hash primarily identifies the blockchain transaction; it does not independently prove that every preceding.

3.4 Functional Requirements

3.4.1 User Interface

The system shall provide public home and result pages, registration and login forms, a protected voter page and a protected administrator page. It shall show loading, failure and completion states so that users can distinguish an incomplete operation from an accepted one. Route protection shall be supported by server-side authorization for restricted API actions.

3.4.2 Voter Registration

The system shall accept a normalized voter identifier, profile details and multiple camera images. The enrollment function shall require at least five submitted images and at least three usable, consistent face descriptors. The stored descriptor shall contain 128 finite numeric values and a nonzero norm.

Self-registration shall assign the user role. If an active face profile already exists, a public replacement request shall create a pending face update for administrative review rather than immediately replacing the active template. Deleted voter records shall not be silently restored through this route.

3.4.3 ID Details and Eligibility

The interface shall allow review of ID-related fields such as name, USN, branch, validity and date of birth. The project includes browser OCR support and an optional server OCR route. Extracted text shall be treated as assistance for data entry; it shall not constitute independent verification of an official identity document.

An authorized administrator shall approve or revoke the ID-verification flag. Successful biometric login shall not override this approval requirement. The voter eligibility endpoint shall explain whether the user is approved, has already voted and has a usable credential for the current election.

The system shall retain the distinction between roster membership and completed enrollment. A roster entry with an empty face profile shall direct the user to registration rather than attempt a meaningless face comparison.

The voter eligibility endpoint shall return a clear status indicating whether the voter is approved, whether a vote has already been recorded, and whether a usable voting credential exists for the current election. These status indicators shall help the frontend determine the appropriate next action, such as continuing to candidate selection, completing registration.

3.4.4 Face Verification and Liveness Challenge

The system shall issue a voter-bound, single-use challenge with a 30-second lifetime. Login shall submit at least eight camera frames. The backend shall require sufficient bilateral eye observations, a close-and-reopen pattern and a matching face descriptor. The current route uses a blink challenge; side-to-side head movement is not a required login step.

3.4.5 Input Validation

The API shall validate identifiers, challenge IDs, image decoding and descriptor structure. Enrollment shall check image resolution, brightness and sharpness. Multiple faces shall be rejected by the main detection path. Client validation shall support usability while server validation remains authoritative.

3.4.6 Session and Credential Issuance

The backend shall issue an HttpOnly authentication cookie after a successful login. An eligible voter may also receive a random one-time voting credential in a separate HttpOnly cookie. The default credential lifetime is 15 minutes; the authentication lifetime is configured separately.

3.4.7 Ballot Submission

The browser shall send the candidate identifier to the authenticated cast endpoint. The service shall check credential ownership, election binding, expiry, revocation, prior consumption and current eligibility before relaying the vote.

Once all checks have been successfully completed, the backend shall coordinate the transaction relayer to submit the candidate identifier and the associated credential digest to the election smart contract. Table 3.1 states the contract shall independently enforce its own rules, including authorized-caller restrictions, election state, candidate validity, and prevention of credential-digest reuse. After successful transaction submission and confirmation, the service shall return the appropriate transaction reference and status to the browser.

Table 3.1: Election State and Permitted Operations

State	Permitted contract behavior
NotStarted	Owner may add candidates and start the election.
Active	Relayer may submit valid unused credentials; owner may end the election.
Ended	Candidate totals remain readable; further votes are rejected.

3.4.8 Candidate and Election Management

The owner shall add candidates before the election begins. Candidate names must be nonempty and satisfy the contract's length checks. The owner shall start an election only after at least one candidate exists. Ending an election shall be permitted only from the active state. The current contract does not provide an in-place reset for another election.

3.4.9 Results and Confirmation

The system shall display candidate information, election state and available totals. After successful casting, it shall display the transaction hash returned by the backend. Public analytics shall distinguish a leading candidate during voting from the winner field returned after the election has ended.

3.4.10 Report Generation

The administrator interface shall support audit-data retrieval and report export. Reports shall be interpreted according to their data source and signature algorithm. The current audit implementation contains fallback records when chain logs are unavailable; these records must not be treated as verified blockchain evidence.

3.5 Non Functional Requirements

3.5.1 Performance

The interface should remain responsive during image capture, verification and transaction confirmation. Response times must be measured on stated hardware and network conditions. A configured timeout is not a measured service-level guarantee.

3.5.2 Reliability

The application should report failures without falsely confirming a vote. Database and blockchain state should be reconciled after interruptions. A retry should not produce a second accepted use of the same credential digest.

3.5.3 Usability

The application shall provide a clear, consistent, and user-friendly interface for voters and administrators. All input fields shall include descriptive labels, and the interface shall provide concise instructions for camera access, face positioning, and completion of the blink-based authentication process. Instructions shall help users understand each step before proceeding. The system shall display actionable error messages when an operation fails, such as when camera permission is denied, facial verification is unsuccessful.

3.5.4 Maintainability

Frontend components, API code, configuration and contract logic should have clear ownership. Changes to credential semantics should be reflected in the contract ABI, relayer code, frontend integration, tests and technical documentation.

3.5.5 Scalability

Capacity planning should consider camera processing, SQLite write serialization, relayer nonces and RPC response time. Redis-based rate limiting can share abuse counters, but it does not by itself make the entire application safe for multiple workers.

3.5.6 Security

Restricted routes shall enforce role checks on the server. Deployment should protect private keys, session secrets, face templates and database backups. Secure transport and suitable cookie settings are required outside local development. Frontend environment variables must not contain relayer secrets.

3.5.7 Portability

The project should be installable using the documented Node and Python environment. Native face-recognition dependencies and model assets require platform-specific attention. The report distinguishes declared dependency versions from a verified installation on a particular computer.

3.5.8 Interpretability and Traceability

The system should explain whether a failure concerns image capture, authentication, approval, credential validity or blockchain submission. A face distance is not a confidence percentage. A displayed transaction reference should be associated with the correct contract and network.

3.6 Hardware Requirements

The following development resources are suggested for a local demonstration. They are planning recommendations rather than measured minimum specifications. Final requirements depend on the number of concurrent users, camera resolution and the processing time of the installed dependencies.

These resources should be validated through practical testing before the system is used for demonstrations involving multiple participants. Table 3.2 suggests the performance testing should record relevant measurements such as biometric-processing time, API response latency, transaction confirmation time, and memory usage.

Table 3.2: Suggested Hardware Requirements

Component	Suggested development provision
Processor	Modern four-core CPU for browser, API and local chain processes.
Memory	8 GB RAM or more for a comfortable single-machine demonstration.
Storage	Several GB of free space for dependencies, model assets, logs and database backups.
Camera	Working webcam; enrollment checks require images of at least 320 by 240 pixels.
Display	Laptop or desktop display suitable for reviewing forms and candidate cards.
Network	Local connectivity for the demo; reliable internet for remote RPC or package retrieval.

3.7 Software Requirements

The versions below describe the package declarations or configured tools in the supplied project. They should be checked for mutual compatibility during installation rather than assumed to prove a successful build. Table 3.3 lists the versions listed below represent the package declarations, dependency specifications, and development tools configured in the supplied project. They provide a reference for reproducing the development environment and identifying the software components intended for use.

Table 3.3: Software Components in the Project

Layer	Software	Role
Frontend	React 19; TypeScript; Vite 8	Pages, routing, development server and production bundle.
Styling	Tailwind CSS; local CSS	Layout, colors, responsive states and accessibility styling.
API	Python; FastAPI; Uvicorn	Validation, authentication, eligibility and relayer endpoints.
Vision	face_recognition; dlib; OpenCV; NumPy	Face detection, landmarks, descriptors and image processing.
Storage	SQLite; optional Redis	Private registry and shared rate-limit storage.
Blockchain	Solidity 0.8.24; Truffle; Ganache; Web3	Compile, deploy, test and interact with the contract.
Testing	pytest; Vitest; Testing Library	Backend and frontend regression suites.

3.8 Software Architecture Requirements

3.8.1 Frontend

The frontend shall use shared contexts for authentication, language and blockchain state. Candidate selection and progress states shall be controlled consistently so that repeated button presses do not deliberately initiate concurrent submissions from the same interface. Server and contract checks remain necessary even when the button is disabled.

3.8.2 Backend

The FastAPI service shall validate payloads, normalize voter IDs, maintain sessions and restrict administrative operations. It shall load configuration from the settings layer and use the private database for eligibility and credential state. Optional services shall return explicit errors when unavailable.

3.8.3 Biometric and Blockchain Layers

Biometric processing shall obtain usable image information and compare it with the stored profile. The relayer shall submit only the candidate identifier and the opaque digest required by the active contract. The contract shall independently enforce its access and replay rules.

3.8.4 Data Layer

Private profile information, approval metadata and credential records shall remain separate from public candidate and tally storage. Election identity shall distinguish credentials associated with different contract deployments. Database backup and deletion procedures must account for both active profiles and pending updates.

3.9 Feasibility Study

3.9.1 Technical Feasibility

The project is developed using established web development frameworks, database management systems, and blockchain technologies to provide an integrated and structured application environment. It includes an integrated demo launcher designed to simplify the process of starting the required components and demonstrating the system's functionality. The overall architecture connects the frontend interface, backend services, database layer, and blockchain components to support communication between the different modules. From a technical perspective, the architecture is suitable for a controlled academic demonstration, as it follows a modular approach and uses commonly adopted development tools and technologies.

3.9.2 Economic Feasibility

The local demonstration can use an existing computer, webcam and a development blockchain. Software dependencies are generally available through their respective open-source projects, subject to their licenses. A realistic cost assessment must still include development time, hardware maintenance, support and any paid hosting or RPC services selected for a future deployment.

Gas expenditure on a local chain is simulated. Testnet demonstrations do not establish the monetary cost of a production election. A cost comparison would require transaction measurements and the fee conditions of the network selected for the experiment.

3.9.3 Operational Feasibility

The interface organizes the voter workflow into understandable stages, but operation depends on trained administrators, correct enrollment and reliable camera access. Support procedures are needed for rejected captures, incorrect records and unavailable services. A supervised demonstration can evaluate these procedures before broader trials.

3.9.4 Schedule Feasibility

Work can be organized into registration, authentication, credential handling, contract integration and result evaluation. Each milestone should have acceptance evidence before the next stage depends on it. Future privacy or accessibility features should be planned as separate improvements rather than assumed to be small additions to the current implementation.

3.9.5 Legal and Ethical Feasibility

Biometric and identity data require informed participation, restricted access and a defined retention policy. An academic trial should use consenting participants and clearly identify its purpose. The interface and report must not imply government approval or election certification without evidence. Applicable institutional and legal review would be needed before any use involving binding decisions; no such approval is claimed here.

The feasibility assessment supports a controlled educational prototype while recognizing that broader use depends on evidence and governance beyond software implementation alone. Biometric and identity-related data are sensitive and must be handled responsibly throughout the development, testing, and demonstration of the proposed system. Appropriate safeguards should be implemented to ensure informed participation, restricted data access, secure data handling, and a clearly defined data retention and deletion policy.

3.10 User Roles and Responsibility Allocation

The application distinguishes permission to use a screen from permission to perform an operation. A visitor may open public pages, but a restricted API request must carry a valid session. An authenticated voter must also satisfy eligibility and credential checks. Administrative approval and contract ownership are separate forms of authority, even when one demonstrator operates both accounts.

The role field in a browser session supports navigation and presentation. The server decodes the signed authentication token before accepting restricted requests. Table 3.4 shows the contract independently checks the transaction sender against its immutable owner or relayer address. Consequently, changing a browser variable cannot legitimately grant contract ownership or satisfy a server role dependency.

Table 3.4: Roles and Enforcement Responsibilities

Role or component	Permitted responsibility	Enforcement location
Public visitor	Read public election information	Public API and read-only contract calls
Authenticated voter	Request eligibility and submit a candidate choice	Voter dependency and cast endpoint
API administrator	Review voters and manage private records	Administrator dependency in FastAPI
Contract owner	Add candidates and start or end an election	onlyOwner modifier in Voting.sol
Server relayer	Submit an authorized ballot transaction	Backend validation and onlyRelayer modifier

3.10.1 Separation During a Demonstration

The operator should record which account is acting as API administrator and which wallet owns the deployed contract. A successful administrator login does not establish that the connected wallet may start the election. Similarly, a correctly configured relayer key does not authorize its holder to change candidate records through owner-only methods.

The voter interface should explain an approval delay as an eligibility condition rather than a face-recognition failure. This distinction follows directly from the separate responsibilities: the biometric service establishes a profile match, while the administrator determines approval. Tests should exercise these responsibilities independently before combining them in a complete voting scenario.

3.11 Requirements Traceability and Acceptance Evidence

Traceability connects a requirement to the implementation that enforces it and the evidence needed to assess it. This makes the report useful during review: an evaluator can move from a statement such as one credential must not be reused to the relevant database checks, contract mapping and repeat-submission test.

The matrix below identifies implementation evidence in the supplied project and the acceptance evidence that a controlled run should retain. Table 3.5 has the final column describes required observations; it does not assert that those observations have already been collected.

Table 3.5: Requirement to Implementation Traceability

Requirement	Implementation anchor	Acceptance evidence to retain
Consistent enrollment	get_enrollment_embedding in main.py	Mixed-frame rejection and accepted profile record
Fresh login challenge	liveness_challenge and verify_face	Expired, reused and wrong-voter challenge responses
Approved eligibility	issue_voting_credential and cast_vote	Approval change followed by eligibility and cast checks
One accepted ballot	Credential checks and credentialUsed mapping	One tally increment across repeat submissions
Controlled lifecycle	startElection and endElection	Owner success and unauthorized rejection receipts
Clear confirmation	API response and Voting.tsx	Matching UI reference, receipt and database tx_hash

3.11.1 Change Control for Requirements

Each revised requirement should identify the component affected and the test evidence that must be refreshed. For example, changing the credential lifetime requires checks at issuance, cookie delivery, expiry validation and user feedback. Editing only a displayed countdown would leave the server's decision unchanged.

A requirement is complete when its success condition and relevant failure conditions are observable. If an evaluator cannot distinguish a blocked request from a failed network connection, the evidence is incomplete. The project should therefore retain request conditions, responses and transaction references together, with private biometric data excluded from ordinary review records. A requirement should be considered complete only when its success criteria and relevant failure conditions can be observed and verified through repeatable tests.

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture

The proposed system follows a layered architecture consisting of a browser-based frontend, an authentication and election API, a private database, and an Ethereum-compatible voting smart contract. These components work together to support voter registration, authentication, eligibility verification, ballot submission, and election monitoring. Although the system distributes responsibilities across multiple components, authority is not equally decentralized. The administrator controls voter eligibility approval, the API manages credential issuance and validation, and the relayer is responsible for submitting blockchain transactions. These trust boundaries must be considered when evaluating the system's security and reliability.

The frontend provides the home, registration, login, ballot, analytics and administrator pages. It coordinates user interaction and obtains information from the API and blockchain context. The API performs the checks that cannot safely depend on browser state, including role authorization, credential ownership and expiry.

The private database stores voter records and their face descriptors. A separate credential table records issuance, expiration, consumption, revocation and transaction references. The contract stores public candidate records, election state and used credential digests. The distinction between these stores is central to the design.

The normal ballot path starts with face authentication and eligibility checking. The browser then sends a candidate identifier to the backend. The backend reads the credential cookie and validates its private record. The relayer signs a contract transaction and waits for a successful receipt before the database is updated and a success response is returned.

The application also has an administrative path. An authenticated administrator may review voter records through the API, while contract ownership is checked for candidate and lifecycle actions. Being logged in as an API administrator does not itself make a wallet the contract owner.

This design supports explicit boundaries between identity, application authorization and chain execution. It also identifies failure cases that require careful handling, especially when a blockchain operation and a database transaction do not complete together. The blockchain layer provides a verifiable transaction history.

4.2 High Level System Architecture

The following Figure 4.1 summarizes the principal layers and the information passed through the authorized voting workflow.

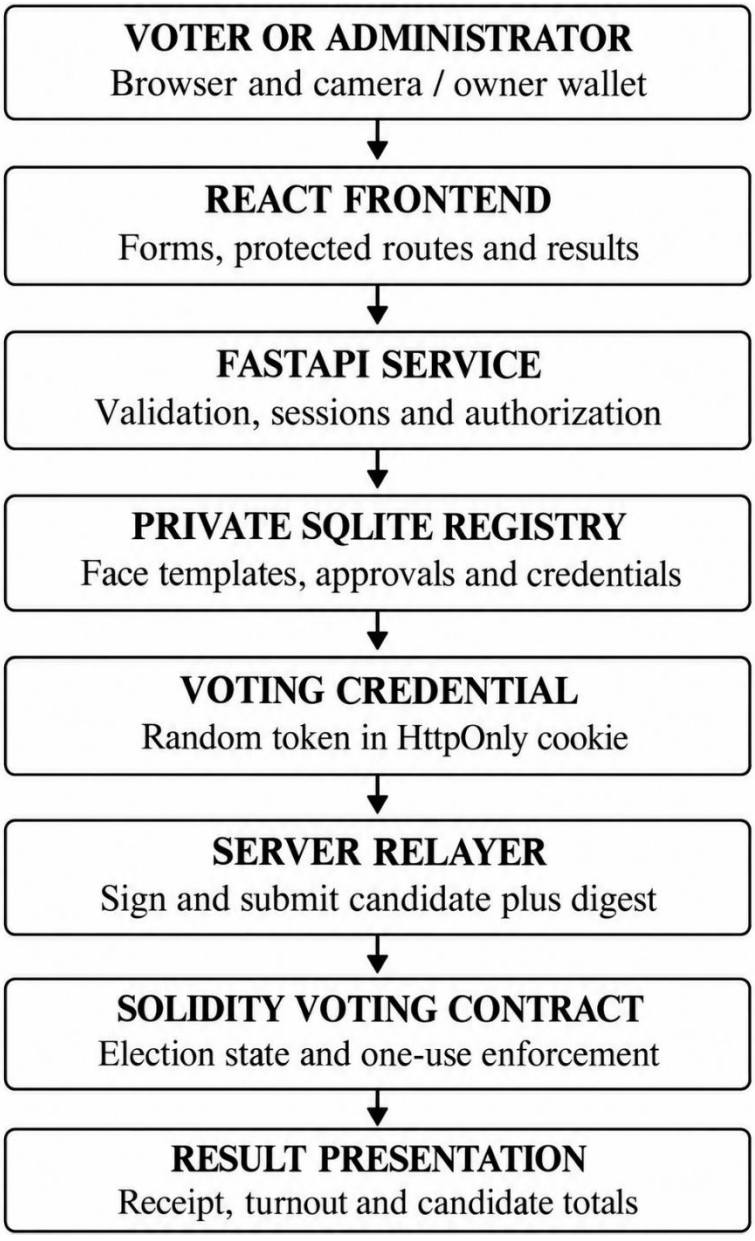


Figure 4.1: High Level Architecture of Prajatantra

The blockchain layer provides a verifiable transaction history, but it does not automatically guarantee voter anonymity, correct eligibility decisions, or end-to-end election integrity.

4.3 Frontend Architecture

4.3.1 User Interaction Layer

App.tsx defines the routes. AuthContext maintains the current session, LanguageContext supplies interface text and Web3Context manages the configured blockchain connection. ProtectedRoute separates user and administrator pages. These components organize navigation and presentation; they do not replace backend authorization.

4.3.2 Registration and Camera Interface

Registration.tsx collects ID details and captures enrollment samples. VoterLogin.tsx requests a challenge and records a short sequence for the verification endpoint. The camera interface displays instructions and errors so that the user can correct a missing profile or poor capture condition before trying again.

4.3.3 Voting and Results Interface

Voting.tsx obtains eligibility and candidate data, then moves through choose, review, sealing and sealed states. Analytics.tsx displays election status and candidate totals. The audit-report service formats administrator data for export. The figures in Chapter 8 illustrate these interfaces using example values, not measured election results.

4.4 Backend Architecture

The main FastAPI module exposes routes for registration, authentication, eligibility, election metadata, casting and reporting. Pydantic models validate request structure. Figure 4.2 talks about settings module controls secrets, cookie behavior, face tolerance, database location and blockchain configuration.

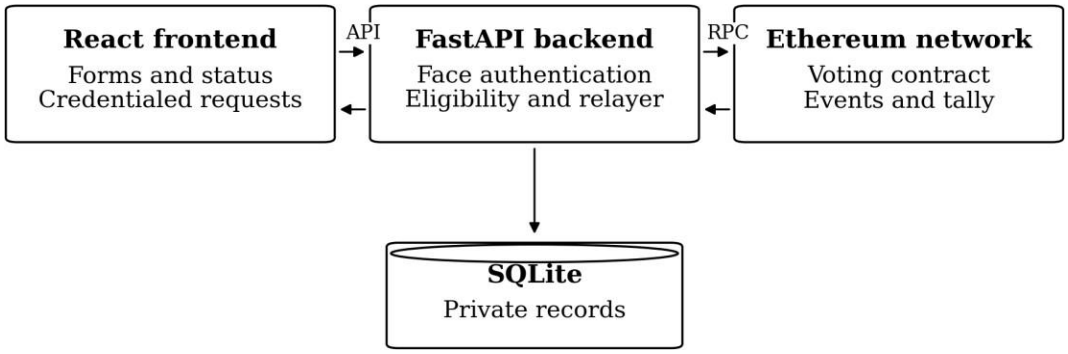


Figure 4.2: Frontend Backend and Blockchain Communication

These properties depend on the contract implementation, credential design, administrative controls, and the information exposed through public transactions.

4.4.1 Request Handling

Requests pass through payload validation and, where required, role dependencies. Rate limits apply to sensitive endpoints such as face verification and ballot casting. Allowed browser origins are configured separately. Authentication data is obtained from the cookie or supported authorization mechanism rather than trusted from a client-supplied role field.

4.4.2 Image and Identity Processing

The service decodes submitted images, obtains facial information and checks the relevant profile. Enrollment applies image-quality checks and averages consistent descriptors. Login evaluates the blink sequence and compares the live descriptor with the stored descriptor. A normalized-lighting path is supported, with a retry on original pixels for older enrollments.

4.4.3 Credential and Transaction Coordination

The cast endpoint starts an immediate SQLite transaction, validates the credential and checks the voter record. It then calls relay_vote. After a successful chain receipt, it marks the credential consumed and stores the transaction hash. This order must be considered in crash recovery because the chain operation cannot be rolled back by a later SQLite failure.

4.4.4 Response Generation

The API returns profile fields, eligibility states, transaction references or structured errors according to the route. Raw voting credentials are delivered in HttpOnly cookies and are not returned in the ordinary authentication JSON body. The frontend uses the response to update the visible stage.

4.5 Biometric and Contract Architecture

The biometric layer is responsible for identifying an enrolled voter profile and evaluating a limited temporal signal as part of the authentication process. It supports identity verification by comparing the presented biometric input with the enrolled profile and checking the relevant signal conditions. Its effectiveness depends on the quality of enrollment data, matching accuracy, camera availability, and the reliability of the implemented verification logic. These checks should be evaluated using authorized test participants or simulated data, with appropriate privacy safeguards and clearly defined acceptance criteria.

The smart contract layer serves a different purpose by enforcing the rules governing blockchain-based voting operations. It validates contract ownership and relayer permissions, verifies the election state, checks candidate validity, and prevents the reuse of credential digests through single-use enforcement.

4.5.1 Stored Data and Logical Relationships

The private data model establishes a logical relationship between voter records and their associated credential records. A single voter may have multiple credential records over time, reflecting credential issuance, expiration, consumption, or revocation. This structure supports credential lifecycle management while maintaining a distinction between voter identity information and authentication credentials.

Figure 4.3 shows election metadata provides descriptive information, such as the human-readable name associated with a configured election deployment. This information helps the application identify and present the relevant election to users and administrators.

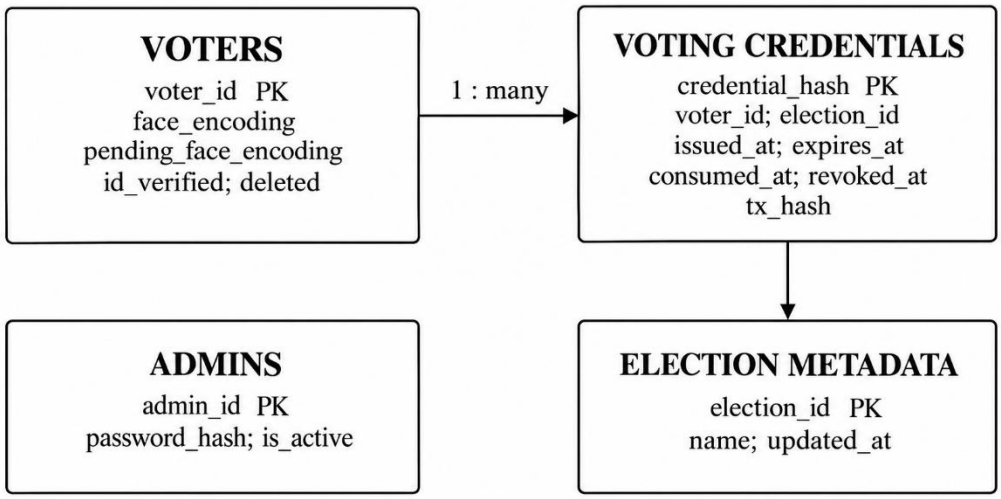


Figure 4.3: Logical Data Model of the Private Registry

4.5.2 Runtime and Deployment Artifacts

Table 4.1 understands the file boundaries helps a developer locate the implementation before changing a requirement.

Table 4.1: Important Runtime and Deployment Artifacts

Artifact	Purpose
SQLite database	Voter profiles, approvals, admins, credentials and election metadata.
Contract ABI and address	Connect the frontend and relayer to the deployed voting contract.
Face model dependencies	Provide pretrained detection, landmark and descriptor operations.
MediaPipe assets	Support the optional browser helper when that capture path is used.
Environment configuration	Select origins, network, database, secrets and relayer settings.

4.6 Source Module Organization

The repository groups the application into a browser frontend, a face-authentication service and a blockchain component. Table 4.2 separation is visible in its directory structure, although a large share of backend behavior remains concentrated in main.py. Understanding the file boundaries helps a developer locate the implementation before changing a requirement.

Table 4.2: Source Modules and Their Responsibilities

Module or directory	Responsibility
src/pages	Registration, login, voting, administration and analytics screens
src/context/AuthContext.tsx	Session lookup, periodic refresh and local logout state
src/context/Web3Context.tsx	Browser blockchain connection and contract interaction context
src/services/api.ts	Credentialed JSON requests and ballot submission helper
server/face-recognition/main.py	Routes, database operations, image processing and relay coordination
server/face-recognition/config.py	Environment-based settings and selected validators
server/blockchain/contracts/Voting.sol	Active election state and permission rules
scripts/start-demo.mjs	Local service startup and deployment coordination

4.6.1 Dependency Direction

Pages call shared services and contexts rather than maintaining separate authentication protocols. The API receives the user's request, checks private records and invokes the relayer. The contract receives a candidate identifier and credential digest; it does not call the browser or query the private database.

Changes should respect this dependency direction. A presentation change belongs in a page component, while an authorization change must be enforced in the API or contract that controls the operation. When a rule appears in more than one layer, both copies require review. Frontend feedback can make a failed operation understandable, but it cannot replace enforcement at the point where the record changes.

The archive also contains a legacy contract outside the configured blockchain contract directory. The Truffle configuration identifies the active source directory, so implementation reviews should follow that configuration rather than selecting a file only because it has the same name. Therefore, referential integrity must be maintained through application-level validation.

4.7 API Request and Response Architecture

The browser communicates with the authentication service through routes under /api/v1. The shared API helper removes duplicate path prefixes, sends JSON and includes browser credentials. A successful response is decoded into the expected frontend type; an unsuccessful response becomes a displayed error using the server's textual detail where available.

Different requests carry different evidence. Registration provides profile fields and enrollment images. Face verification provides a voter identifier, a challenge identifier and a frame sequence. Ballot submission provides only candidate_id in the JSON body. Figure 4.4 authentication and voting credentials accompany the request through cookies, keeping the raw voting token outside the ordinary application response.

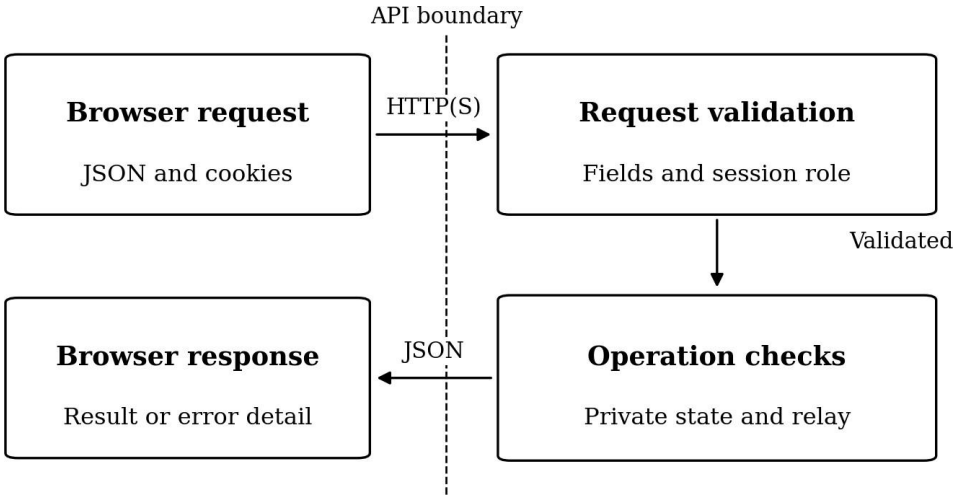


Figure 4.4: Request Validation Across the Browser and API Boundary

4.7.1 Response Interpretation

An HTTP success code must be interpreted in the context of the route. A successful profile request shows that a session can be read; it does not show that a ballot has been recorded. A successful cast response includes a transaction hash after the relay and private persistence steps complete.

Errors should retain the distinction between authentication, permission, conflicting state and unavailable infrastructure. The frontend may simplify the wording for a user, but diagnostic records should preserve the operation and status code. This allows the evaluator to determine whether an attempted ballot was rejected before relay, failed at the blockchain stage or reached an uncertain state after submission. Particular attention should be given to uncertain transaction outcomes. A request may fail before the relayer submits a transaction, fail during blockchain execution, or experience a timeout after submission when the transaction's final status is not yet known.

4.8 Private Database Dictionary

The private database contains more than the face descriptor. It stores the approval decision, credential lifecycle and links needed to associate the authenticated voter with an accepted transaction. Table 4.3 fields support application consistency, while also creating sensitive identity-to-transaction relationships.

Table 4.3: Selected Private Data Fields

Table	Field or field group	Meaning in the implementation
voters	voter_id; role	Normalized identity key and application role
voters	face_encoding; pending_face_encoding	Active descriptor and replacement awaiting review
voters	id_verified; verified_by; verified_at	Administrative approval and reviewer metadata
voters	deleted; voted; last_voted_election_id	Account and election-related status fields
voting_credentials	credential_hash; voter_id; election_id	Private token digest and ownership scope
voting_credentials	issued_at; expires_at	Credential validity interval
voting_credentials	consumed_at; revoked_at; tx_hash	Use, withdrawal and transaction reference
deleted_voters	voter_id; deleted_at	Deletion marker used during roster initialization
election_metadata	election_id; name; updated_at	Private display metadata for the configured election

4.8.1 Schema Initialization and Evolution

The initialization routine creates missing tables and adds selected columns when an older voters table is present. It checks the existing column list before adding several newer fields. This supports local upgrades, but it is not a substitute for a versioned migration process with backups and recovery tests.

The source uses parameterized values for the operational queries described here. Logical relationships connect the tables, but the shown schema does not declare foreign-key constraints. Application code therefore carries responsibility for ownership checks, deletion behavior and consistent election identifiers. Tests should examine these relationships explicitly, especially after a restart or schema change.

4.9 Trust Boundaries and Data Exposure

The system crosses four principal boundaries: browser to API, API to private storage, relayer to blockchain and operator to administrative controls. Each boundary carries different information and relies on a different authority. A complete architecture review must identify both the information transferred and the component allowed to change it.

At the browser boundary, profile fields and camera frames are supplied by the client. The API must validate them before relying on their content. A challenge identifier limits reuse within the implemented protocol, but it does not itself prove that uploaded pixels came from an untampered camera. The face and blink decisions therefore depend on more than the existence of an identifier.

At the storage boundary, the service can read face descriptors, approval fields and transaction references. The face descriptor is serialized as JSON in SQLite. This representation is an implementation format, not encryption. A backup of the database inherits the sensitivity of the original records and should be handled with the same access restrictions.

At the blockchain boundary, the relayer signs a call containing a candidate identifier and a digest of the credential. The contract verifies sender permissions and state conditions. It cannot independently repeat the face match or the administrator's identity review. The correctness of those earlier decisions remains part of the service's trusted responsibilities.

4.9.1 Consequences for Review

Public transaction data reveals the candidate choice accepted by the contract. The private transaction reference can connect that choice to a voter record for an operator with database access. Omitting a name from the contract consequently reduces direct disclosure without establishing ballot secrecy.

A useful review records who can read each data store, who can change each decision and what independent evidence exists. The current architecture supports a traceable academic demonstration. Stronger privacy or resistance to a compromised service would require changes to the voting protocol and its verification model, rather than a change to the appearance of the interface.

A comprehensive security review should therefore examine the system's access-control boundaries, data flows, and accountability mechanisms. It should identify who can read each data store, who has authority to modify eligibility and election settings, who can issue or revoke credentials, and who can authorize blockchain transactions. In conclusion, transparency of transaction records and confidentiality of voter identity are distinct properties.

CHAPTER 5

METHODOLOGY

5.1 System Development Methodology

The project follows an integration-oriented methodology in which registration, biometric verification, eligibility, credential handling and blockchain recording form distinct stages. Figure 5.1 states has an input, a responsible component and an expected output.

The methodology consists of the following steps:

1. Define voter and administrator responsibilities.
2. Prepare the private registry and enrollment workflow.
3. Validate camera samples and store face descriptors.
4. Implement challenge-based face login and session handling.
5. Review eligibility and issue a one-time voting credential.
6. Configure and deploy the election contract.
7. Connect candidate selection to the authenticated cast API.
8. Record confirmation data and display election results.
9. Test component rules, integration and recovery conditions.
10. Configure Session Management.
11. Implement Role-Based Access Control.
12. Review Eligibility Before Credential Issuance.

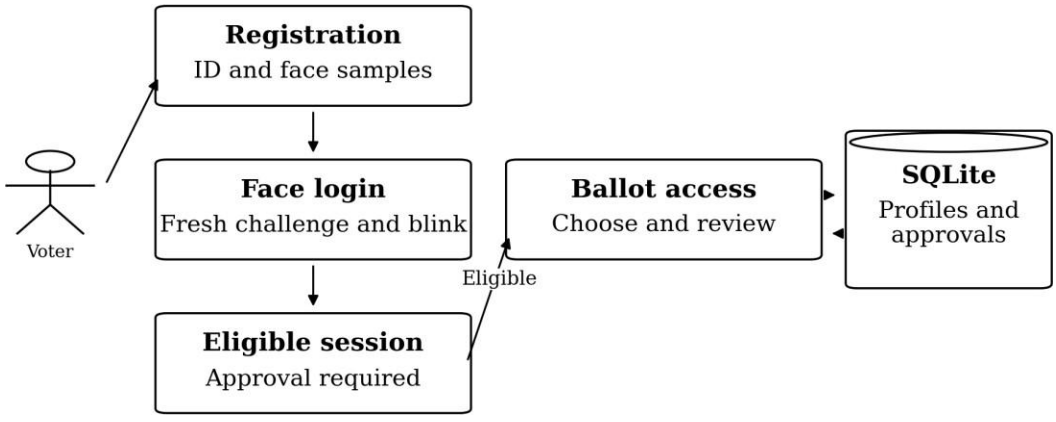


Figure 5.1: Model Diagram of Registration and Authentication

5.2 Voter Data Collection

Registration collects the voter identifier, ID-related profile fields and multiple camera samples. The registry can contain roster entries before a face is enrolled. An empty roster template is not sufficient for login, so the verification route returns a clear registration requirement in that case.

Data collection must be limited to what the workflow needs. The source includes a roster for the intended student context; individual roster details are not reproduced in this report. Example interface figures use illustrative values.

5.3 Image Preprocessing

Enrollment decodes the image, checks its dimensions and examines brightness and sharpness. Figure 5.2 configured checks reject images smaller than 320 by 240 pixels, very dark or overexposed images, and low Laplacian variance. Usable frames then pass through face detection, landmark extraction and descriptor generation.



Figure 5.2: Model Diagram of Enrollment and Image Processing

5.4 Face Descriptor Preparation

The enrollment function requires at least five input images and three valid descriptors. It compares each usable descriptor with the initial reference, retaining a consistent group at the enrollment distance limit of 0.60. The mean descriptor is validated before storage. This procedure combines samples from one enrollment session; it is not training a new face-recognition model.

Each valid descriptor is compared with an initial reference descriptor using the configured distance metric. The function retains a consistent group of descriptors that satisfy the enrollment distance threshold of 0.60. Samples that exceed this limit are excluded from the selected group to reduce inconsistencies among the enrolled facial representations. The mean descriptor is then calculated from the retained group and validated. A useful review records who can read each data store, who can change each decision and what independent evidence exists. The current architecture supports a traceable academic demonstration.

5.5 Independent Enrollment and Verification Sessions

A meaningful experiment should capture enrollment and verification in different sessions. This reduces the chance of evaluating the same images twice. Figure 5.3 gives genuine trials should be distinguished from impostor comparisons and presentation-attack trials, with the identity and conditions recorded privately for evaluation.

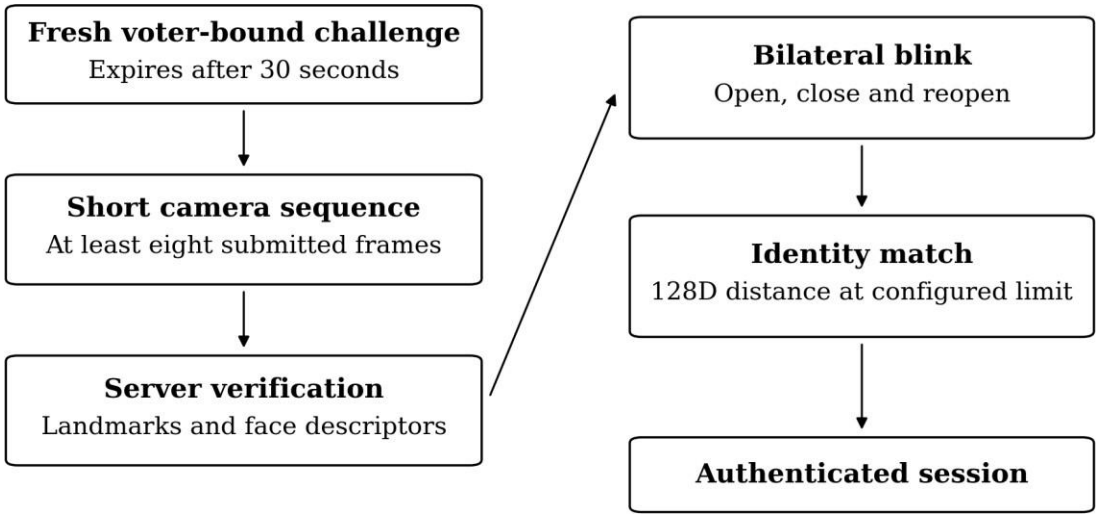


Figure 5.3: Model Diagram of Blink Based Face Verification

5.6 Image Normalization and Identity Matching

During login, the server can normalize illumination using CLAHE on the lightness channel. It obtains identity encodings from separated frames and averages consistent encodings. If the normalized result does not match an older profile, the route can retry the original pixels from the same sequence before deciding.

5.7 Handling Unusable Camera Sequences

The sequence must contain enough usable observations of both eyes. The system rejects a missing blink or an unmatched profile. A failed challenge is consumed, so the next attempt needs a fresh challenge. Repeated failures should lead to supervised assistance, not automatic approval.

Repeated authentication failures should trigger a controlled assistance procedure, such as administrator-supervised verification or an approved alternative identity-verification process. The system must not automatically approve a voter merely because the biometric checks repeatedly fail. Any assistance procedure should follow documented authorization rules, maintain appropriate audit records, and protect the participant's biometric information.

5.8 Authentication and Credential Processing

5.8.1 Session Authentication

The backend creates a signed JWT containing the voter identifier and role after successful verification. The browser receives the token in an HttpOnly cookie. AuthContext checks the session on initialization and periodically attempts refresh while a session is active. Refresh does not replace eligibility checking at the point of ballot submission.

The configured JWT lifetime defaults to two hours. The session cookie and the voting credential are different objects with different purposes. A valid login session may exist without a usable credential, for example when the voter is awaiting approval or has already cast a ballot.

5.8.2 One Time Voting Credential

The service creates 32 random bytes and stores the SHA-256 digest with the voter and election identifiers. The raw token is encoded for delivery in the credential cookie. Its default lifetime is 15 minutes. Issuance checks that the voter exists, is not deleted, is approved and has not already completed a ballot for that election.

When an unused valid credential is replaced, the earlier active record is revoked. The cast endpoint rejects an expired, revoked or consumed credential and verifies that the supplied credential belongs to the authenticated voter. These checks are applied server-side even if the interface already reports eligibility.

5.9 Evaluation of the Authentication Workflow

The workflow should be evaluated with valid, missing, mismatched and expired inputs. Important cases include a challenge issued for another voter, a replayed challenge, an empty profile, insufficient frames and re-login after a recorded ballot. The expected behavior is denial with an understandable error and no unauthorized contract submission.

The authentication and voting workflow should be evaluated using a structured set of positive and negative test cases to verify that only valid, authorized requests can proceed to blockchain transaction submission. Testing should cover valid, missing, mismatched, and expired inputs across the biometric, session, credential, and smart contract layers. Each test case should define the initial conditions, input values, expected response, and observable evidence of the outcome. Particular attention should be given to challenges issued for a different voter, replayed challenges, empty biometric profiles, insufficient camera frames, and attempts to log in again after a ballot has been recorded.

5.10 Contract Preparation and Deployment

The Truffle configuration selects the contract directory under server/blockchain and writes build artifacts into the frontend contract-artifact directory. The active Voting contract is compiled with Solidity 0.8.24 and deployed with a nonzero relayer address. A different legacy contract exists elsewhere in the archive, so the configured source path must be used consistently.

The deployed address, chain identifier and relayer configuration must agree across the browser and API. The local demo launcher coordinates Ganache, contract deployment, FastAPI and Vite. Remote configuration requires separate verification of network access, keys and funds for transaction fees.

5.11 Ballot Submission Workflow

The browser sends a candidate ID to the cast endpoint. After validation, the relayer signs a transaction containing the candidate ID and Keccak digest of the raw credential. The backend waits for a successful receipt, stores the transaction reference and clears the credential cookie in the success response.

After successful validation, the configured relayer signs and submits a transaction to the voting smart contract. Figure 5.4 talks about the transaction contains the candidate ID and the Keccak digest of the raw voting credential.

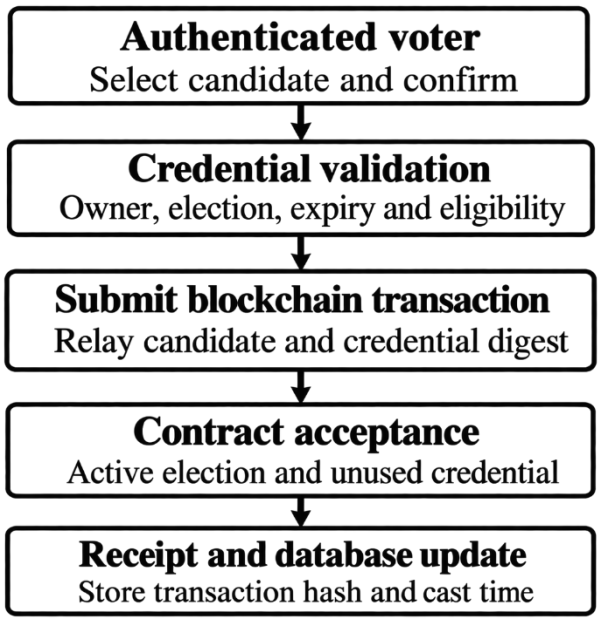


Figure 5.4: Model Diagram of Vote Submission and Confirmation

5.12 Election State Classification

The contract represents election state with an enum. Table 5.5 states control accepted operations and should not be confused with the browser's choose, review, sealing and sealed presentation stages.

Table 5.5: Contract Election States

Value	State	Meaning
0	NotStarted	Candidates may be added by the owner.
1	Active	Valid relayer submissions may increment a candidate tally.
2	Ended	Voting is closed and stored totals remain available.

5.13 Result Interpretation

Candidate counts are read from the contract. The analytics route also reads the number of consumed credentials and currently uses the larger of the two totals. This fallback can support a display during partial failures but does not prove that both stores agree. A proper final evaluation must reconcile them explicitly.

5.14 Frontend and Backend Integration

The shared API helper sends JSON requests with credentials included. The server resolves the session and credential cookies, while the frontend displays response states. Candidate and lifecycle interactions also depend on the configured Web3 connection.

The server remains responsible for determining whether the session is valid, whether the requested operation is permitted, and whether the associated voting credential satisfies its lifecycle requirements. Figure 5.5 has frontend processes the returned responses and displays appropriate states, such as successful authentication, validation errors, access denial, pending operations, and transaction failures.

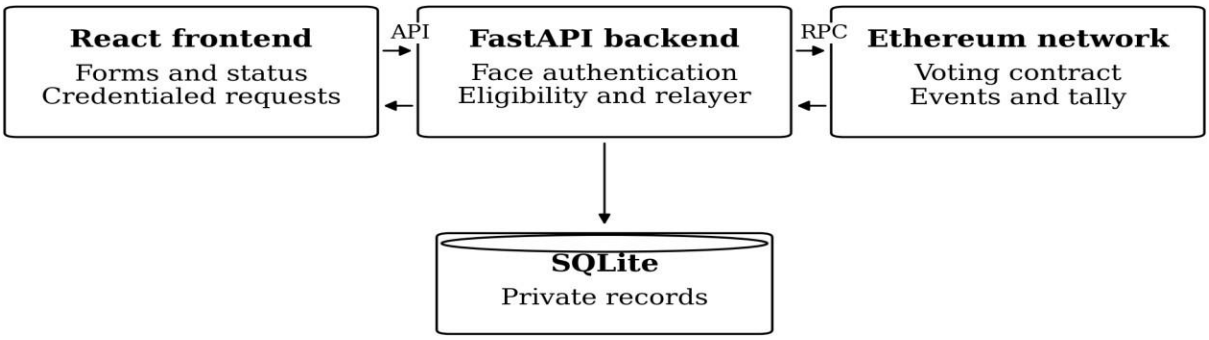


Figure 5.5: System Architecture of the Integrated Web Application

5.15 Testing Methodology

Testing should proceed from controlled component inputs to a complete local election. The unit stage checks descriptor validation, image rules, route authorization and contract guards. The integration stage checks whether the actual browser, API, registry, relayer and contract agree on the outcome of a ballot.

The principal user interactions are summarized below. Figure 5.6 is voter functions and administrative functions have different permissions. Public results can be viewed without granting access to private voter records or administrative audit endpoints.

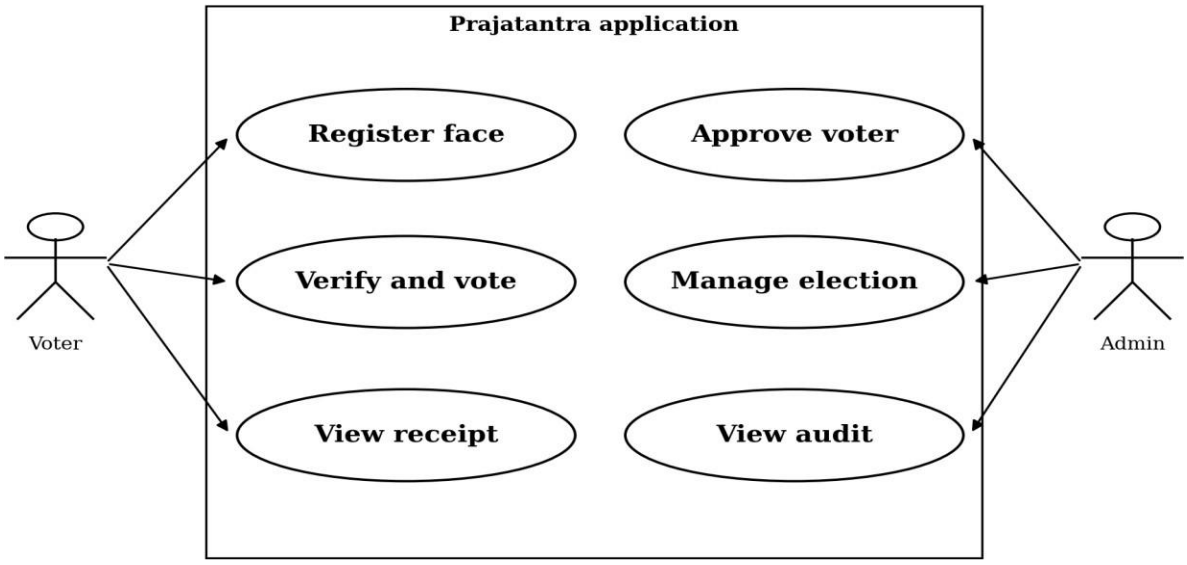


Figure 5.6: Use Case Diagram of the Voting System

The acceptance run should retain the deployed contract address, software version, test identities, candidate configuration and receipts. It should include both successful casting and expected rejections. Once the owner ends the election, subsequent submissions must be rejected while the final counts remain readable.

A separate fault-injection stage should test unavailable RPC services, interrupted requests and application restarts. The current one-use challenge store is in process memory, so worker routing and restarts must be considered when testing a multi-process deployment.

This acceptance and fault-injection methodology provides a structured way to evaluate the prototype under both normal and abnormal operating conditions. Passing these tests would provide evidence of the specific behaviors covered by the test suite, but would not independently establish production readiness, ballot secrecy, or certification for real elections.

5.16 Detailed Enrollment Procedure

Enrollment constructs the reference descriptor used by later authentication. The implementation requires at least five submitted images, then evaluates whether enough of those images produce usable and mutually consistent face descriptors. The number of submitted frames and the number of retained descriptors are therefore different quantities.

For each image, the service decodes the Base64 payload, checks image quality and attempts face processing. The quality check requires a minimum width of 320 pixels and height of 240 pixels. Mean grayscale brightness must remain between the configured code thresholds of 35 and 235, and the Laplacian variance must be at least 35. These are implementation thresholds, not measured guarantees about recognition performance.

The face-processing path rejects multiple visible faces and validates each descriptor as a finite, nonzero vector with exactly 128 elements. Frames without a usable descriptor are omitted from the candidate set. Enrollment fails if fewer than three usable descriptors remain, even when the user supplied five or more images.

The first usable descriptor becomes the reference for the consistency check. Other descriptors are retained when their distance from that reference is no greater than 0.60. At least three descriptors must survive this stage. The service averages the retained descriptors and validates the resulting vector before storing it.

5.16.1 Interpretation of the Consistency Rule

The rule reduces the influence of inconsistent frames within one enrollment attempt. It does not create an independently verified identity label. It also depends on the first usable frame: if that frame is atypical, the retained group may differ from a group selected around another reference.

A practical enrollment evaluation should record submitted frames, usable frames, retained descriptors and rejection reason. Those counts explain whether a failure arose from capture quality, face detection or consistency. An approved profile replacement should preserve an auditable administrative decision, because replacing the reference changes the basis of all subsequent comparisons for that voter.

A practical enrollment evaluation should maintain a record of the number of frames submitted, the number successfully processed, the number of descriptors retained and the reason for any rejection. These measurements help distinguish failures caused by poor capture conditions, unsuccessful face detection, invalid descriptors or insufficient consistency among samples.

5.17 Temporal Blink Decision Procedure

The eye aspect ratio measures eye opening from six landmark points. For an ordered eye contour, two vertical point distances form A and B, while the horizontal eye width forms C. The implemented calculation is EAR = (A + B) / (2C). Both eyes are evaluated independently before the sequence can be accepted.

The blink helper searches for a closure with enough observations before and after it. It omits the immediately adjacent samples when estimating the open-eye baseline, allowing a natural blink to occupy several frames. Figure 5.7 has each side of the baseline must contain at least two finite observations for each eye.

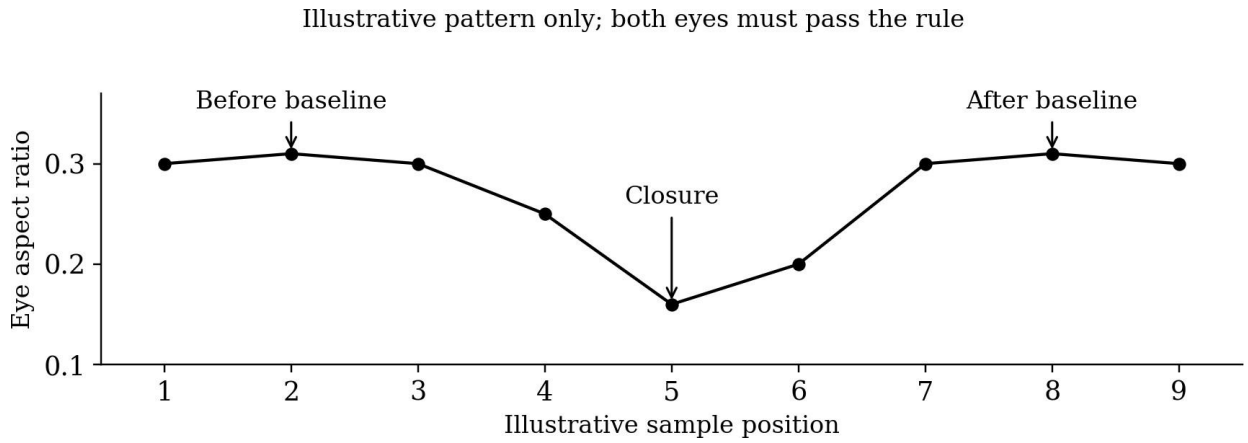


Figure 5.7: Temporal Samples Used by the Bilateral Blink Rule

The open level is the smaller of the before and after medians. It must be at least max(0.18, 0.70 × EAR_MIN_OPEN). A candidate closure must be no greater than 85 percent of that open level, and the absolute drop must be at least 0.03. Both eyes must satisfy the conditions at the same candidate position.

5.17.1 Evaluation of Temporal Evidence

These conditions describe a relative geometric change. They should not be converted into a percentage accuracy claim. Missing landmarks, an incomplete reopen period or a blink outside the captured interval may cause rejection even for an enrolled user. Evaluation should retain the observed failure category and capture conditions, while avoiding storage of raw face sequences beyond the agreed research purpose. Authentication may be rejected even when the enrolled user is genuine. Such false rejections can occur when facial landmarks cannot be detected reliably, the eye does not complete the expected reopening sequence, or the blink occurs outside the captured observation interval.

5.18 Voting Credential Lifecycle

The voting credential is a short-lived random value associated with a voter and the configured election. It is separate from the authentication session. A valid session identifies the user to the API, whereas the credential records whether the service may submit an unused ballot authorization for that election.

Issuance creates 32 random bytes. The private table stores a SHA-256 digest of those bytes, together with issue and expiry times. Figure 5.8 is browser receives the raw value in an HttpOnly cookie. The default lifetime is 15 minutes. Issuance checks the voter record and declines to create a new usable authorization after an accepted vote is recorded.

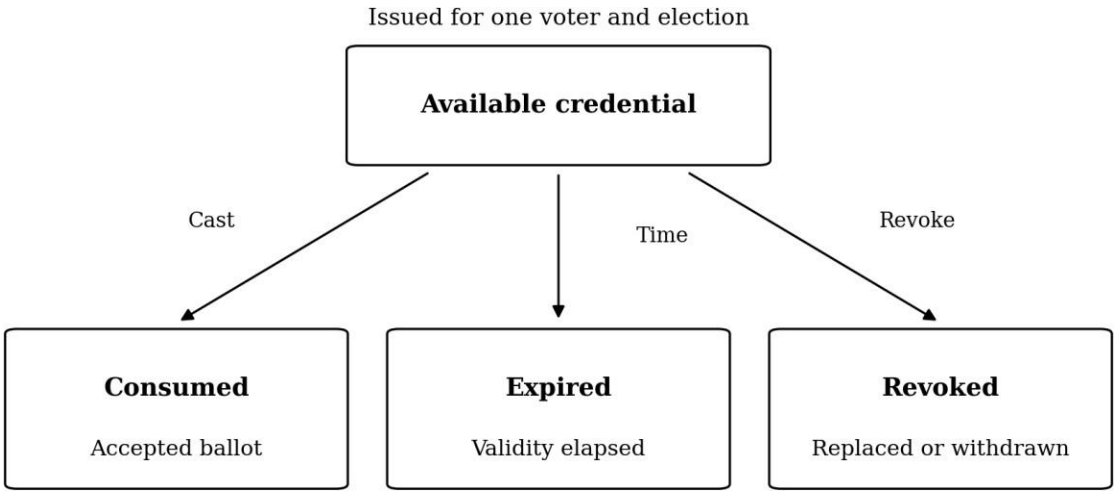


Figure 5.8: Logical States of a Voting Credential

The database represents these states through timestamps rather than a single state column. A credential is unusable when expired, revoked or consumed. When a current unused credential is replaced, the issuance logic revokes the earlier current credential. Logout also attempts to revoke the applicable unused credential and clears the relevant cookies.

5.18.1 Consumption and Public Replay Checks

Before relay, the cast endpoint validates ownership, election binding, expiry, revocation, consumption and current eligibility. The contract uses a Keccak digest of the raw bytes for its own used-credential mapping. The private SHA-256 digest and public Keccak digest therefore have different roles and should not be compared as identical values. Successful processing records the transaction hash and consumption time locally after the contract receipt is obtained. In summary, blink detection is a geometric challenge-validation mechanism whose performance must be evaluated empirically.

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 Frontend Implementation

The frontend uses React and TypeScript with Vite. App.tsx defines the application routes, including registration, login, voting, administrator pages and public analytics. Shared contexts manage session data, language selection and blockchain connectivity. Components use local CSS and Tailwind classes for layout and state presentation.

The voting page separates selection from confirmation. Its handleVote function checks the current stage, selected candidate, prior vote state, submission state and credential readiness before calling the shared API helper. A failed request returns the interface to review with an error message; a successful response records the transaction hash and displays the sealed state.

6.2 Backend Implementation

The FastAPI application is implemented primarily in main.py. Its request models, dependencies, database helpers and route handlers coordinate registration, authentication, voter management and election reporting. The settings module reads environment configuration and supplies defaults for local operation.

FastAPI supports typed request validation and generated API documentation. In this project, Pydantic models also normalize or constrain selected fields. These facilities make the API contract explicit, but application-specific checks remain in the route and helper logic.

6.3 Registration Implementation

The public registration route forces the user role and validates a multi-image face profile. For a new profile it writes the descriptor and submitted ID fields. For an existing enrolled user it queues a pending replacement, preserving the active profile until administrative review.

6.4 ID Details Processing

The interface supports manual review and OCR assistance. The optional server OCR route uses EasyOCR when available and extracts recognized USN and date patterns. Some returned fields remain dependent on the recognition result or manual entry. OCR output does not bypass approval. Privacy protection must also be considered when collecting evaluation evidence. Raw facial images and video sequences should not be retained beyond the agreed research purpose and retention period.

6.5 Enrollment Image Processing

Images are decoded from Base64 and checked before enrollment. The API validates the descriptor shape as exactly 128 values and rejects nonfinite or zero-norm data. It retains a consistent group of face descriptors and stores their mean as JSON in the voter record.

6.6 Blink Sequence Processing

The current VoterLogin capture function limits the maximum frame dimension to 720 pixels and collects 14 JPEG frames with a nominal 100-millisecond delay between samples. It instructs the user to blink during the sequence. These are capture settings, not measured end-to-end latency.

The backend obtains landmarks on usable frames and computes eye aspect ratios independently for both eyes. Its blink rule compares a closure with nearby open-eye baselines, requiring a relative drop and a minimum absolute change. Head motion is not required by this route, although auxiliary scripts in the archive contain other liveness routines.

6.7 Face Recognition Integration

The face_recognition dependency performs detection, landmark extraction and encoding. The main route requests HOG detection and rejects multiple visible faces. Matching uses the configured default tolerance of 0.55. No custom face-model training or measured biometric benchmark is included in the implementation evidence.

6.8 Vote Generation and Relay

The API helper submits only candidate_id in the request body. The backend reads the raw credential from the cookie and derives the chain digest. The relayer checks configuration, estimates gas, signs the transaction and waits up to its configured receipt timeout.

6.9 Confirmation and State Persistence

After a successful receipt, the API stores consumption time and transaction hash and updates the voter's election marker. The success response returns the reference used by the receipt interface. Recovery remains necessary if local persistence fails after chain confirmation.

A critical recovery scenario occurs when the blockchain confirms the transaction but local database persistence fails. In this situation, the ballot may already have been recorded on-chain even though the application has not updated the credential’s consumption status or the voter’s election marker. Treating the request as an ordinary failure and blindly resubmitting it could create inconsistent application state or trigger a duplicate-submission attempt.

6.10 Frontend and Backend Communication

The API base is centralized in apiBase.ts and api.ts. Requests include cookies and use JSON payloads where appropriate. The helper parses response details and converts unsuccessful responses into errors for the interface. AuthContext performs session checking, refresh and logout, while restricted API routes independently validate the session.

The project includes five language codes: English, Hindi, Kannada, Tamil and Malayalam. The presence of a language dictionary does not establish that every dynamic message is translated. The interface also includes status announcements and a voice-assistance helper, which should be evaluated with assistive technologies.

6.11 Deployment Implementation

The documented local demo starts Ganache on port 7545, FastAPI on port 8000 and Vite on port 8080. It deploys the configured contract and supplies the resulting address to the services. Docker and reverse-proxy configuration are included for alternative environments, and the Truffle configuration contains a Sepolia target.

A configured remote target is not evidence of a currently running deployment. The source also requires attention to optional deployment dependencies and model assets. Secrets must remain server-side, and HTTPS deployments must enable suitable cookie and origin settings rather than reuse permissive local assumptions.

6.12 Error Handling

The backend returns errors for missing profiles, invalid sequences, expired credentials, denied roles and failed relay operations. The frontend displays feedback and avoids intentionally repeating a submission while it is in progress. These measures support ordinary interaction but do not remove the need to reconcile ambiguous transaction outcomes.

The report generator is implemented with jsPDF and table-formatting utilities. Its current fallback data and official-looking labels require correction before an exported document can be treated as an evidence-based audit record. Export formatting alone does not validate the underlying election data.

The key requirement is consistency between the blockchain record and the private application database. Because these systems do not share a single atomic transaction, explicit reconciliation, safe retry handling and observable error states are necessary to maintain reliable voting records.

6.13 Session Restoration and Protected Navigation

AuthContext maintains the browser's view of the current session. On startup, it requests /auth/me with credentials included. The interface keeps a session-checking flag until this lookup completes, allowing a protected route to distinguish loading from an unauthenticated state. A failure to reach the service does not prevent the public landing page from loading.

When a session exists, the context starts a refresh interval of ten minutes. The refresh request uses POST /auth/refresh and replaces the local session fields on success. The context stores the voter identifier, role and selected profile fields; it does not expose the raw authentication cookie to ordinary component state.

ProtectedRoute checks the session and compares its role with the route's allowed role. It displays a loading indicator while the initial session check is pending. An unauthorized administrator route redirects to the administrator login page, while an unauthorized user route redirects to the public home page.

6.13.1 Browser State and Server Authority

These checks support navigation but remain a browser-side convenience. The server dependencies independently decode the token and verify the required role before sensitive operations. A route that displays correctly must still handle an expired or rejected request, because local session state can become stale after the page is opened.

Logout sends a request to the service and clears local session state in a finally block. This ensures that the browser interface leaves the signed-in state even when the request fails. It does not establish that an unreachable server has revoked every credential. A recovery test should distinguish the visible logout action from the server-side changes that actually occurred.

Session tests should therefore cover first load, valid restoration, failed refresh, role mismatch, network loss and a second tab retaining older state. Each case should verify both the displayed route and the response of a restricted API endpoint.

The logout workflow sends a request to the authentication service and clears the browser's local session state within a finally block. This design ensures that the interface exits its signed-in state even if the logout request fails or encounters a network error. Nevertheless, clearing local state does not prove that the server has invalidated the session or revoked the associated credential. If the server is unreachable, the browser may appear logged out while a server-side credential remains valid until its expiry or another revocation mechanism takes effect.

6.14 Input Validation at Component Boundaries

Validation occurs at several levels because a value can be structurally valid while still being unsuitable for an operation. Request models check selected field shapes and lengths. Image helpers check decoded content. Database logic checks ownership and lifecycle state. Table 6.1 states the contract checks sender authority, election state and candidate validity.

Table 6.1: Examples of Implemented Validation Boundaries

Boundary	Implemented condition	Reason for the check
Face verification model	Nonempty voter ID of at most 100 characters	Avoid an absent or oversized identity key
Challenge field	Nonempty identifier of at most 128 characters	Require a bounded challenge reference
Image sequence	Nonempty supplied list	Reject an absent capture sequence
Face descriptor	128 finite values with nonzero norm	Reject malformed biometric representations
Election name	Trimmed nonempty value of at most 120 characters	Bound private display metadata
Candidate record	Nonempty name and byte-length limits	Bound contract candidate fields
Cast endpoint	Cookie decodes to exactly 32 bytes	Require the expected credential representation

6.14.1 Validation Order and Error Meaning

The CastVoteRequest model accepts an integer candidate_id, while the active contract enforces the valid candidate range. An evaluator should therefore test the complete path rather than assuming that a frontend selection control is the only possible source of candidate identifiers.

Likewise, accepting a JSON payload does not mean that its frames pass quality checks or that a challenge remains valid. Validation tests should identify the expected rejection layer and confirm that later state changes do not occur. This is especially important for ballot requests, where an invalid input must not consume a valid credential or increment a candidate total.

Evaluation should therefore test the complete request path, from frontend input through API validation and credential verification to transaction submission and contract execution. Test cases should include valid candidate identifiers, identifiers outside the configured range, negative values, unexpected data types, missing fields and malformed JSON. These cases help establish which component rejects each invalid request.

6.15 Local Deployment and Configuration Agreement

The project includes a demo launcher and Makefile targets for the local development stack. The launcher searches for a suitable Python environment and resolves the required Node command-line tools. Figure 6.1 tells truffle compiles the active Solidity source and places its build artifacts in the frontend contract directory.

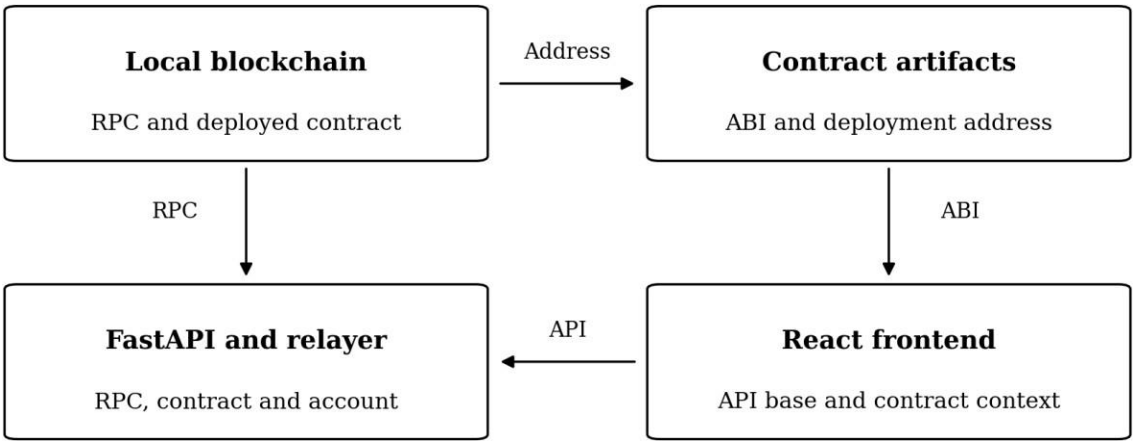


Figure 6.1: Configuration Agreement in the Local Demonstration Stack

The development network uses the local RPC endpoint on port 7545. The API is started on port 8000, while the documented demonstration frontend uses port 8080. A working environment must align the deployed contract address, network, owner and relayer information across the browser and service. A listening port alone is insufficient evidence that the intended contract is deployed.

6.15.1 Cookie and Origin Configuration

The apiBase.ts module standardizes the API base URL used by the frontend. It normalizes local hostnames so that an application opened through localhost does not inadvertently send requests to 127.0.0.1 or another hostname that the browser treats as a different origin. Although these addresses may resolve to the same development machine, cookies and browser security policies can treat them differently. This distinction is important when authentication depends on session cookies being included in subsequent API requests.

The module also ensures that the API base path consistently ends in /api/v1. Standardizing this path reduces integration errors caused by inconsistent URL construction across frontend components. However, hostname normalization alone does not guarantee that cookies will be accepted or transmitted. The browser's origin rules, credential settings on frontend requests, server-side CORS configuration and cookie attributes.

6.16 Error Responses and Recovery Guidance

The implementation distinguishes several failure classes in the cast path. A missing, malformed, expired or revoked credential is an authentication problem. A credential owned by another voter or an ineligible record is a permission problem. Table 6.2 is prior consumption is a conflict with existing election state. Infrastructure failures arise separately when the relayer cannot complete its work.

Table 6.2: Selected Cast and Relay Error Conditions

HTTP status	Example condition in the source	Appropriate interpretation
401	Missing, invalid, expired or wrong-election credential	Obtain a valid authorization for the current context
403	Credential ownership mismatch or ineligible voter	Resolve identity or approval state before retrying
409	Credential or voter already recorded as used	Inspect existing record; do not create a second ballot
503	Unavailable node or invalid relayer configuration	Operator must restore the required service configuration
502	Relay exception or unsuccessful transaction receipt	Inspect transaction evidence before assuming no vote occurred

6.16.1 Ambiguous Submission Outcomes

The relayer waits up to 120 seconds for a receipt after sending the signed transaction. A timeout after broadcast is different from a failure before broadcast. The generic relay error alone cannot establish that the network never accepted the transaction. Recovery should inspect the known chain state before attempting another authorization.

The current implementation obtains a pending account nonce and sends transactions directly; it does not implement the durable submission queue proposed in Chapter 10. Diagnostic records should preserve the operation stage and available transaction reference without recording private keys or raw credentials. User-facing feedback should explain whether to authenticate again, wait for approval or seek assistance, rather than offering an unrestricted retry for every error.

Direct nonce acquisition and submission can introduce coordination challenges, particularly when multiple requests are processed concurrently or a previous transaction remains pending. A transaction may also be broadcast successfully even if the application subsequently loses its connection to the RPC service. Without durable submission records and reconciliation mechanisms, the application may have limited information available after a process restart or an interrupted request.

CHAPTER 7

TESTING AND RESULTS

7.1 Software Testing

Testing evaluates whether the application enforces the requirements described in Chapter 3. A useful test records its inputs, expected result, actual result and environment. The supplied test files establish regression coverage intent; their existence does not establish a pass rate or biometric performance.

7.2 Unit Testing

The backend test module contains 16 test functions. Their subjects include health, enrollment, authorization, login, persistence, rate limiting, logout, image quality, descriptor validation, challenges and reporting. Biometric operations are mocked in important tests, so those checks evaluate application behavior rather than real-camera recognition accuracy.

7.3 Integration Testing

Integration testing should exercise registration, approval, login and casting against a deployed contract. The database, credential cookie, receipt and public event must refer to the same test ballot. A complete run also checks restart behavior and the distinction between an API administrator and the contract owner.

7.4 Functional Testing

The frontend includes 10 test cases across VoterLogin, Voting and Analytics. They cover form input, candidate loading, eligibility-related presentation, the relayer API workflow and results display. The contract suite contains five tests covering owner and relayer configuration, owner-only candidate creation, relayer-only voting, replay rejection and event emission.

7.5 Test Cases

Table 7.1 describes acceptance cases for the integrated system. These are expected outcomes to verify in a controlled run. They are not presented as completed execution results. Additional cases should be added whenever a new failure mode or an inconsistency between components is discovered. These cases represent expected outcomes rather than completed execution results. Each test must be executed using a documented configuration, with the observed response and supporting evidence recorded separately. Table 7.1 include API responses, application logs, transaction hashes, blockchain receipts and relevant before-and-after database states.

Table 7.1: Integrated Acceptance Test Cases

ID	Test condition	Expected behavior
T1	New voter submits sufficient valid enrollment images.	Store a valid profile; require eligibility approval before voting.
T2	Existing voter submits a replacement face.	Queue pending update; preserve active template until approval.
T3	Fresh challenge, matching face and valid blink.	Create authenticated session; issue credential if eligible.
T4	Challenge is expired, replayed or bound to another ID.	Reject verification and require a new challenge.
T5	Authenticated voter is not ID-approved.	Deny ballot submission and show approval requirement.
T6	Credential is expired, revoked or belongs to another voter.	Reject before submitting a contract transaction.
T7	Eligible voter selects a valid candidate during Active state.	Return confirmed transaction hash and persist consumption.
T8	Same credential digest is submitted again.	Contract rejects reuse without a second tally increment.
T9	Non-owner tries candidate or lifecycle administration.	Contract rejects the unauthorized call.
T10	RPC fails or application stops around confirmation.	Do not assert an unverified success; reconcile chain and database.

7.6 Evaluation Metrics

Biometric evaluation should record false accepts, false rejects, genuine acceptance and failure-to-acquire rates. Operational evaluation should record capture duration, API latency, confirmation time and recovery outcomes. Vote-count correctness should be assessed by reconciling successful test transactions with candidate totals and private consumption records.

7.7 Comparison of Evaluation Layers

Mocked unit tests isolate application logic; contract tests isolate chain rules; live integration tests evaluate their interaction. None of these layers alone establishes public-election readiness. Each metric must identify its denominator, test conditions and retained evidence.

Metrics should also distinguish between test cases designed, test cases executed and test cases passed. An unexecuted test must not be counted as a successful result. Similarly, a high pass rate across a small or narrowly selected test set does not establish general reliability or security.

7.8 Observed Source Validation

All seven Python source files in the archive were syntax-checked successfully during report preparation. This establishes that those files parse under the preparation runtime. It does not import their external dependencies, start the API or execute the supplied pytest suite.

The full frontend, backend and blockchain test suites were not executed for this report. Their required application dependencies, live camera setup and deployed test election were not present in the preparation environment. Consequently, no full-suite pass rate, recognition accuracy or transaction-performance result is asserted.

7.9 Existing Test Coverage and Gaps

The contract tests cover several important permission and replay conditions. Additional integration tests should exercise credential expiry, revocation, election binding, repeated login after a vote, concurrent submissions and interruption after chain confirmation. Real-camera trials are necessary to measure whether users can consistently complete the blink sequence.

The repository's security review describes a previous Slither scan, but the raw output file referenced by that review is not included in the archive. Its reported detector counts are therefore not reproduced as independently verified results in this report.

7.10 Testing of Deployment

A deployment check should verify the actual contract address and ABI, the relayer account, the RPC network, cookie settings, allowed origins and model resources. It should also test the effect of restarting a process while challenges or credentials are active.

7.11 Result Analysis

The available evidence supports a detailed description of implemented controls and test coverage. It does not support a numerical claim that the system is accurate, scalable or fully secure. The next evaluation milestone is a reproducible local election run with retained receipts and explicit successful and unsuccessful cases.

The next evaluation milestone is a reproducible local election run using a documented software version, a known smart contract deployment, a defined candidate configuration and controlled test identities. The run should cover both successful and unsuccessful scenarios, including valid ballot submission, invalid candidate identifiers, expired or reused credentials, unauthorized requests, election closure and transaction timeouts. The evaluation should distinguish between implemented functionality, expected test outcomes and experimentally verified behavior.

7.12 Negative Authentication Test Design

Negative tests examine whether invalid requests are rejected without creating an authenticated or eligible state. They should vary one condition at a time so that the observed failure can be attributed to the intended check. Table 7.2 is valid baseline profile and a controlled capture fixture are useful for isolating challenge behavior from image-processing failures.

Table 7.2: Proposed Negative Authentication Cases

Case	Controlled input condition	Expected observation
N1	Reuse a challenge after an earlier verification request	The second request is rejected as missing or used
N2	Use a challenge issued for another voter ID	Verification is rejected before a session is granted
N3	Submit after the challenge validity interval	The expired challenge is rejected
N4	Use a roster entry with an empty face template	The response directs the user to registration
N5	Supply frames with multiple visible faces	The face-processing path rejects the capture
N6	Supply no valid bilateral close-and-reopen pattern	No successful blink-based authentication is granted
N7	Use a valid blink with a nonmatching identity	The face comparison prevents authentication

7.12.1 Evidence and Isolation

Each record should include the case identifier, fixture version, challenge issue time, request time, response status and whether a session cookie was issued. Raw biometric samples should be held separately under the evaluation's retention policy. Test logs can reference a fixture identifier without duplicating the face images.

The verification route removes the challenge from its store before proceeding with later checks. A failed attempt can therefore require a fresh challenge. Tests must not reuse the same challenge across unrelated cases, because an earlier consumption would obscure the condition under investigation. Existing mocked tests and the additional real-camera evaluation should be reported separately. Raw biometric samples should be stored separately from routine test logs and managed according to the evaluation's approved retention policy. Rather than duplicating facial images or video sequences in diagnostic records, each test should reference a unique fixture identifier. Access to the fixture repository should be restricted, and retention and deletion.

7.13 Credential and Concurrent Submission Tests

Credential tests should inspect both private state and public contract state. A rejected API response is not sufficient evidence if the candidate tally still changes. Table 7.3 conversely, a successful contract transaction without a corresponding private record identifies a consistency failure even when the UI eventually reports an error.

Table 7.3: Proposed Credential and Recovery Cases

Case	Scenario	Evidence to compare
C1	Submit the same unused credential twice sequentially	One accepted transaction and one tally increment
C2	Submit the same credential concurrently	Responses, chain events and final consumption record
C3	Use a credential with another voter's session	Ownership rejection with no accepted ballot
C4	Change the configured election before submission	Election-binding rejection and unchanged new-election tally
C5	Revoke approval after credential issuance	Current eligibility check blocks the request
C6	Interrupt after transaction broadcast	Receipt and contract mapping checked before recovery
C7	Interrupt before the local commit after confirmation	Chain acceptance compared with absent or incomplete private update

7.13.1 Concurrency as a Separate Evaluation

BEGIN IMMEDIATE serializes relevant SQLite write access, but the database transaction remains open while the relay waits for its receipt. Concurrent requests can therefore experience waiting or contention. The relayer also needs coordinated account nonces if requests are processed by multiple workers or separate service instances.

The evaluation should record request start and completion times, lock-related failures, transaction hashes and final candidate counts. An acceptance criterion should be defined before the run: the controlled batch must produce the intended number of accepted ballots with no duplicate use. Any unexplained discrepancy should remain visible in the results rather than being hidden by the larger of two displayed totals. Relayer coordination is another important consideration. The current implementation obtains a pending account nonce and submits transactions directly. When requests are processed concurrently by multiple workers or separate service instances, independent nonce acquisition may result in competing transactions.

7.14 Biometric Evaluation Protocol

A biometric evaluation needs labelled attempts that are independent of the enrollment images. Each participant should enroll once for the selected experiment, then return in separate capture sessions. Genuine attempts compare the correct participant with their profile; impostor attempts compare a different consenting participant with that profile. Presentation-attack trials require their own clearly defined conditions.

The evaluation should record lighting, camera type, image dimensions, spectacles, distance from the camera and whether a complete blink sequence was captured. These conditions explain failures and prevent a single aggregate percentage from hiding uneven behavior. Samples used to select a threshold must be separated from the samples used for the final reported result.

7.14.1 Metrics and Denominators

The false acceptance rate is the number of accepted impostor attempts divided by the number of evaluated impostor attempts. The false rejection rate is the number of rejected genuine attempts divided by the number of evaluated genuine attempts. Failure to acquire should be recorded separately when the system cannot obtain usable input for the intended comparison.

For every rate, the report should state the numerator, denominator, threshold and treatment of unusable captures. A result based on a small number of trials should retain those counts rather than presenting an unsupported impression of precision. Threshold 0.55 is a distance decision setting in this implementation; it is not a probability or an accuracy percentage.

7.14.2 Timing and Repeatability

Timing should distinguish camera capture, frame transfer, server processing and subsequent blockchain work. The frontend's 100-millisecond interval between captured frames is a requested browser delay, so it does not measure complete login latency. Report observed timings from repeated trials using the same hardware and configuration, then identify any changed condition in comparative runs.

This protocol defines future evaluation work. The supplied archive does not contain the labelled trial records needed to calculate these metrics for Prajatantra. Results should be added only after the corresponding experiment and evidence review are complete.

For comparative experiments, the evaluator should document every material change, including camera hardware, lighting, frame rate, browser version, network conditions, server configuration, database settings and blockchain response conditions.

7.15 Reproducibility and Evidence Packaging

A repeatable test run begins with an identified source revision or archive checksum, dependency information and a description of the execution environment. Table 7.4 tells the backend, frontend and contract suites test different layers, so their outputs should be retained separately before an integrated summary is written.

Table 7.4: Evidence Package for a Controlled Test Run

Evidence item	Minimum information	Purpose
Source record	Archive identifier or revision and relevant configuration	Identify the exact implementation examined
Environment record	Operating system, runtimes, dependencies and camera	Explain reproducibility conditions
Test execution log	Command, timestamp, result and failure detail	Distinguish execution from test inventory
Chain record	Network, contract, receipt and relevant events	Verify the accepted public operation
Private state extract	Sanitized credential status and transaction reference	Check local persistence and reconciliation
Issue record	Reproduction steps and affected requirement	Connect a failure to corrective work

7.15.1 Reporting Test Status

The Makefile exposes separate frontend, backend and contract targets, together with a combined test target. Their names are useful reproduction instructions, but invoking a command is not equivalent to obtaining a successful result. Missing dependencies, unavailable services and failed assertions must remain distinct in the execution record.

The existing report records the successful syntax parsing of seven Python files and identifies the supplied test inventory. Those observations should remain unchanged until additional execution evidence is available. Any later test result should state the environment and command used, identify skipped cases and link failures to the relevant requirement rather than replacing the entire history with a single pass label.

The Makefile supports repeatable test invocation, while reliable evaluation depends on recording what actually happened during each execution. The existing evidence—successful syntax parsing of seven Python files and identification of the supplied test inventory—must remain unchanged until further results are verified.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 System Output

The project provides a complete set of interface modules for registration, verification, administration, voting and result presentation. Figures 8.1 to 8.10 are source-based interface illustrations with example data. They explain the intended screens and states; they are not screenshots from an executed election or evidence of measured performance.

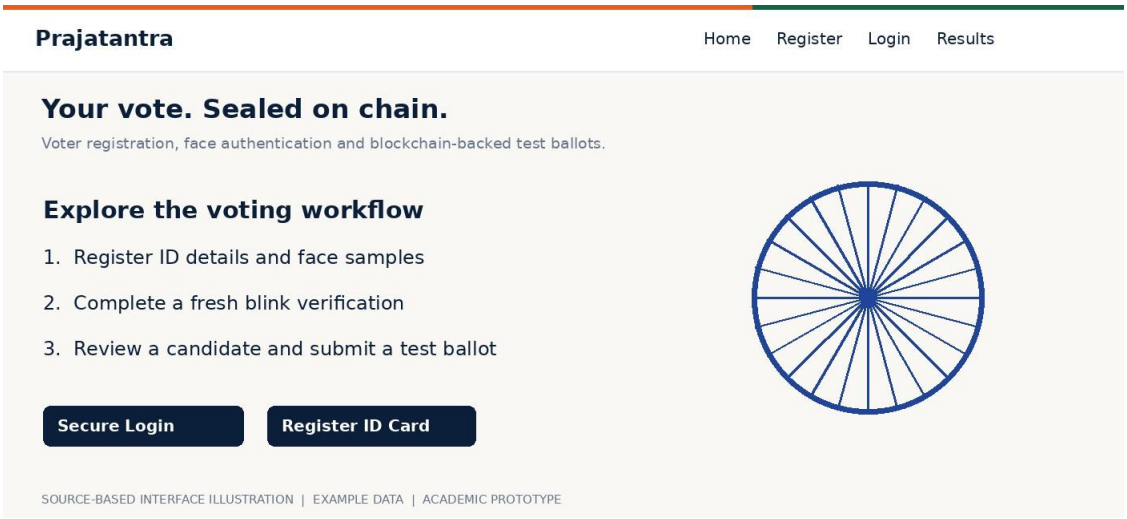


Figure 8.1: Home Interface Illustration of Prajatantra

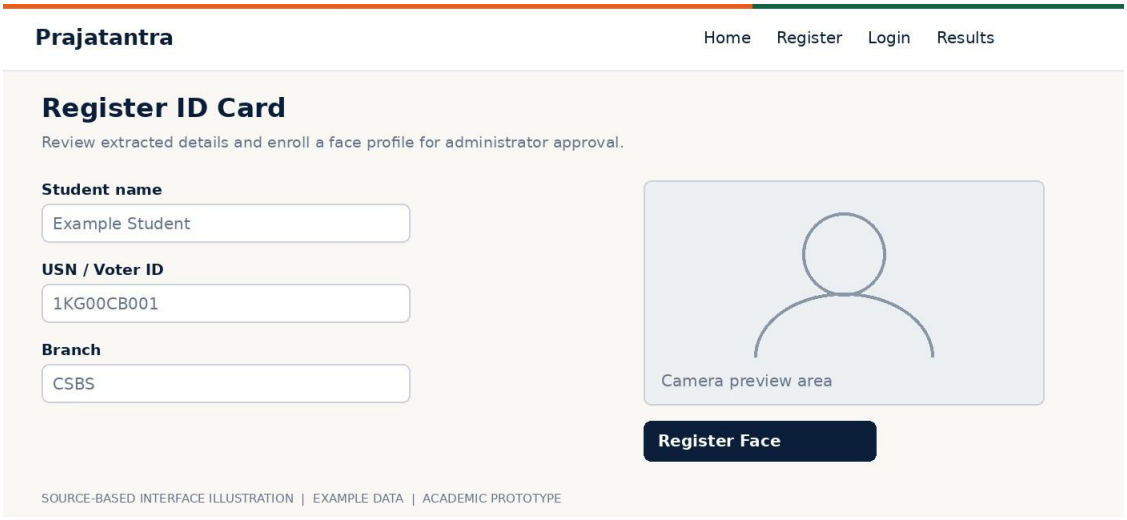


Figure 8.2: Voter Registration Interface Illustration

The home interface directs users to registration, login and results. Registration combines reviewed ID fields with a camera enrollment workflow. The current code separates this enrollment step from administrator approval.

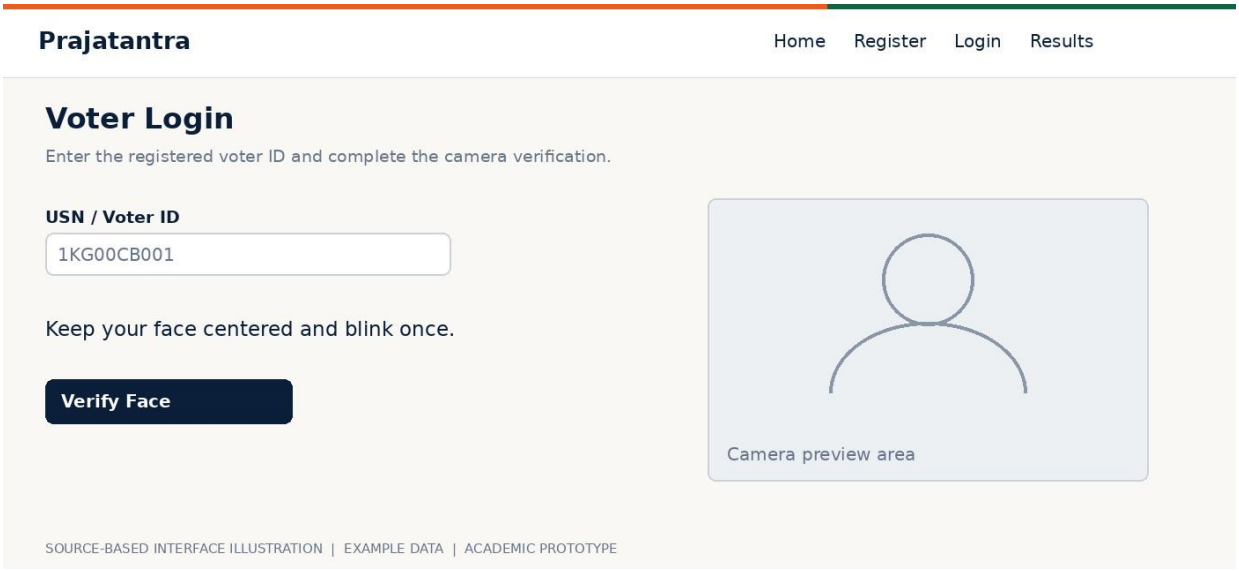


Figure 8.3: Face Login and Blink Capture Interface Illustration

The voter enters a registered identifier and starts a short camera capture. The backend consumes a fresh challenge and checks the sequence. A valid face profile can authenticate successfully while still awaiting administrative approval to vote.

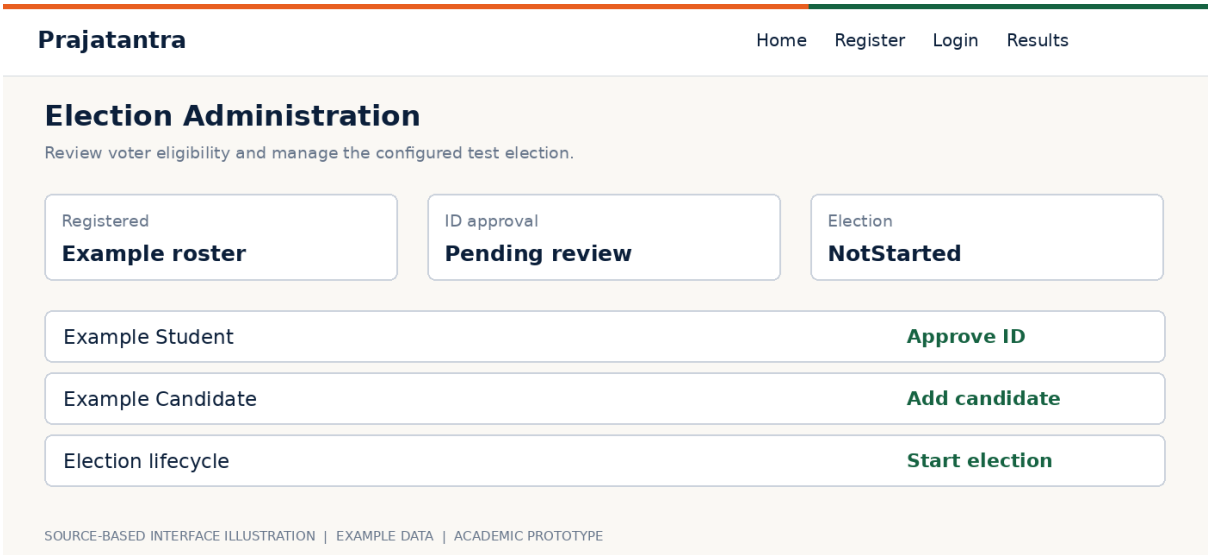


Figure 8.4: Administrator Interface Illustration

The administrative interface supports several operations required to manage voter eligibility, biometric profile updates and election configuration. These include reviewing submitted identity information, approving pending facial-profile replacements and maintaining election metadata. These functions are governed by application-level authentication and role-based authorization, ensuring that only appropriately authorized administrators can access the corresponding administrative workflows. The distinction is important because an authenticated application administrator is not necessarily the contract owner.

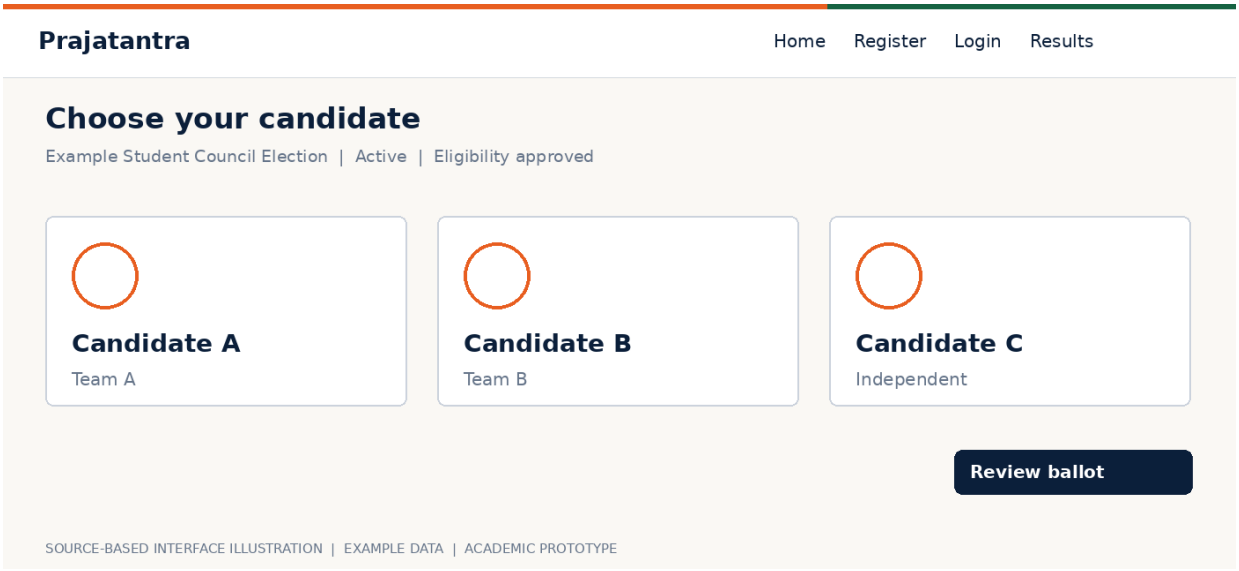


Figure 8.5: Candidate Selection Interface Illustration

The ballot displays candidates returned by the configured contract. A NOTA option must exist as an actual candidate record if the administrator wants it available. Creating a client-only candidate identifier would fail contract validation.

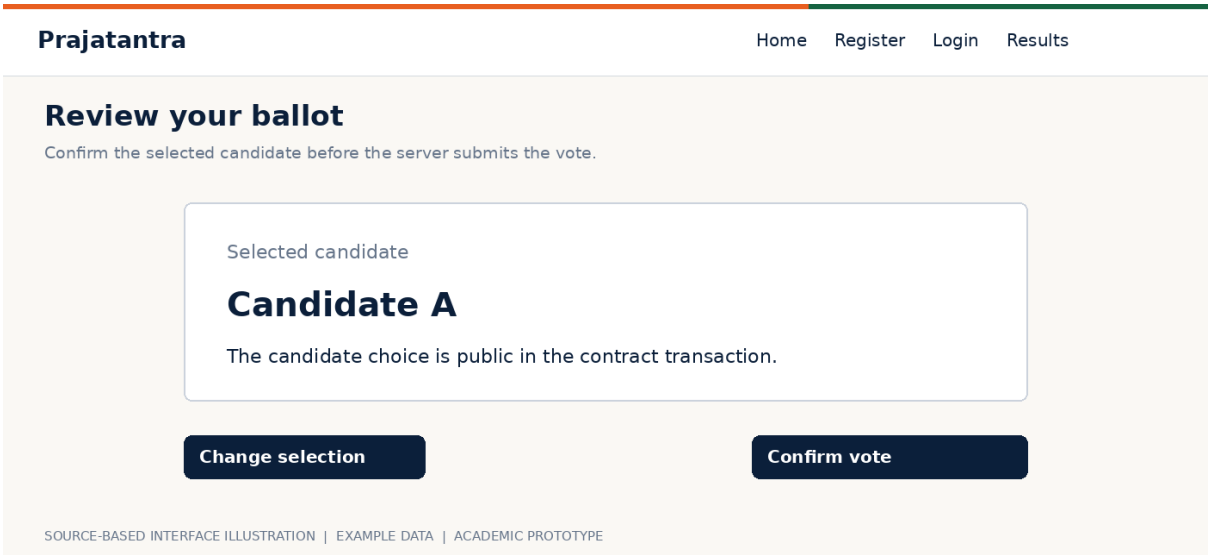


Figure 8.6: Ballot Review Interface Illustration

The review stage separates selection from submission. The interface checks whether a credential is ready and whether a request is already in progress. The server then repeats the authorization checks before relaying a transaction.

The frontend also uses submission-state indicators to manage the interaction. When a request is being processed, the interface can disable repeated submission attempts and display an appropriate status message. However, these client-side checks serve primarily as usability safeguards. They cannot guarantee that a request is valid or that duplicate submissions are impossible, since requests.

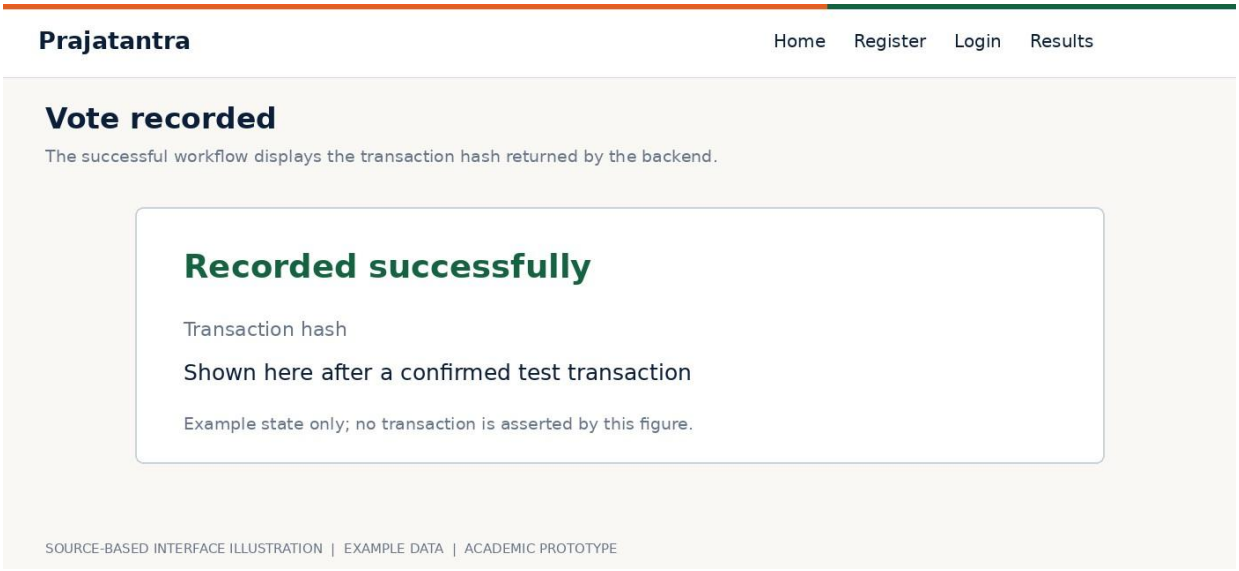


Figure 8.7: Transaction Confirmation Interface Illustration

A successful cast response contains the transaction hash. The interface uses this reference to show confirmation. A receipt is associated with the configured chain and contract; it does not independently demonstrate anonymous or coercion-resistant voting.

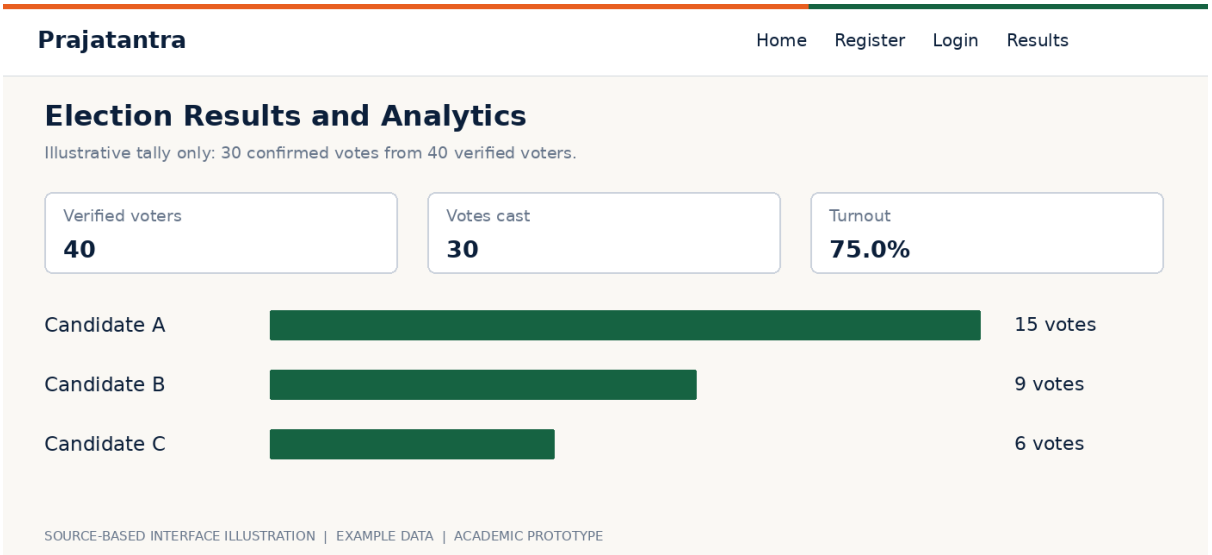


Figure 8.8: Results and Turnout Interface Illustration

The example above uses 30 votes and 40 verified voters to explain the display. These values are illustrative only. Actual candidate counts; denominator values and connection status must be obtained from a recorded test election. The distinction between recorded votes and verified voters must also be maintained. The number of verified voters represents the population defined by the application's eligibility or verification criteria, whereas the recorded vote count represents ballots accepted by the voting contract. These quantities are not necessarily equal because a verified voter may not cast a ballot, and a voter may be verified without having completed.

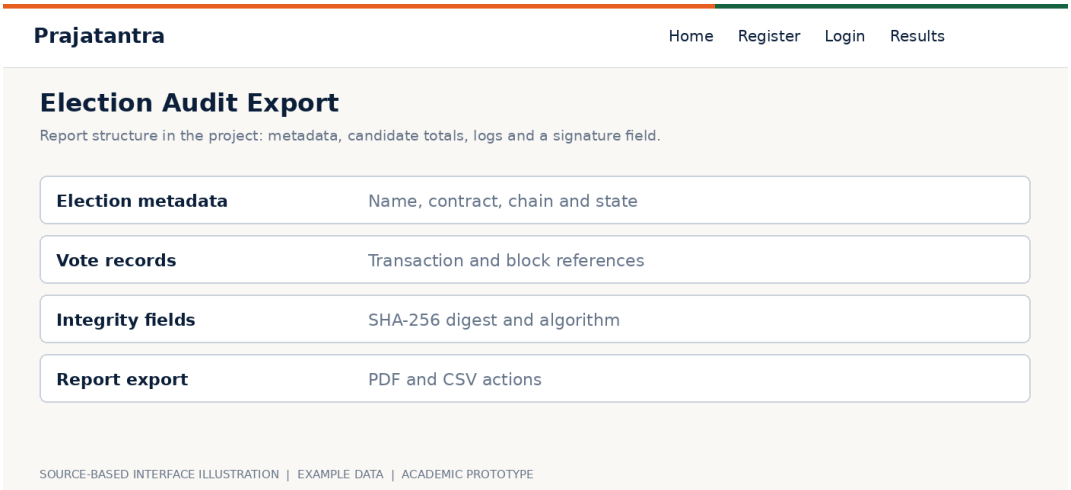


Figure 8.9: Administrator Audit Export Interface Illustration

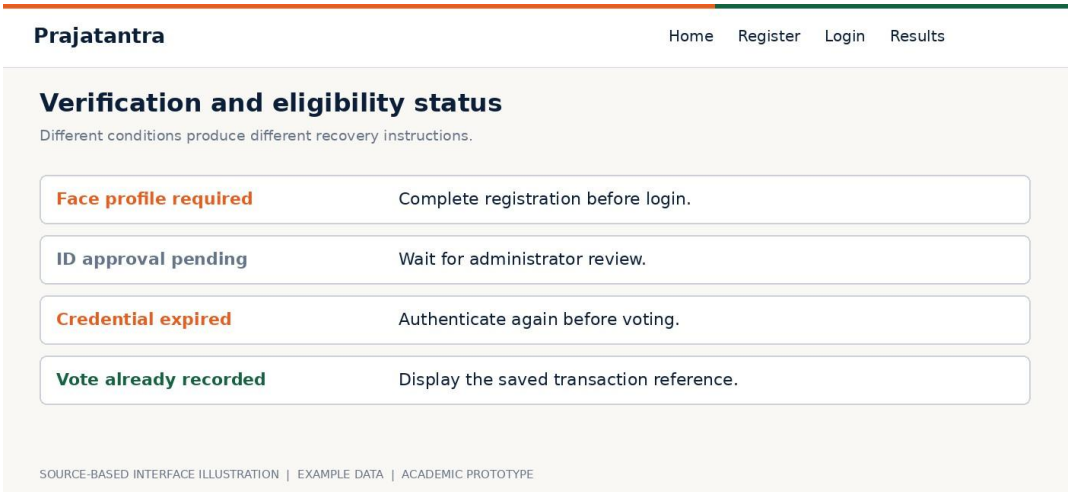


Figure 8.10: Verification and Eligibility Message Illustrations

8.2 Authentication Performance

No measured false-acceptance, false-rejection or presentation-attack results accompany the implementation. The configured thresholds describe decision rules, not empirical accuracy. A valid performance report needs independently labelled trials and stated capture conditions.

8.3 Comparative Analysis

Compared with a password-only demonstration, the project adds face enrollment and temporal verification. Compared with direct wallet voting, it introduces private eligibility checks and a relayer. These differences redistribute trust and operational responsibilities; they do not automatically establish superior security or usability. Compared with a direct wallet-based voting approach, Prajatantra introduces private eligibility checks through the application backend and uses a relayed to submit blockchain transactions. In a direct wallet model, the voter typically interacts with the blockchain using a connected wallet, while the proposed architecture separates application-level eligibility validation from transaction submission.

8.4 Analysis of Face and Liveness Checks

Face matching addresses similarity to an enrolled profile, while the blink rule checks a temporal pattern. Their combination is meaningful only when the enrollment belongs to the correct person and the submitted frames are trustworthy. Replays and synthetic inputs require explicit evaluation.

8.5 Election State Analysis

The contract allows one forward lifecycle from setup to active voting to closure. It does not implement scheduled automatic opening or an in-place election reset. A new deployment and consistent configuration are needed for a separate election.

8.6 Audit and Result Interpretation

Public events include the candidate ID and credential digest. The backend retains transaction references, so identity-to-ballot correlation remains possible for an operator with access to private records. The audit endpoint also contains fallback records that cannot be treated as authentic block evidence.

8.7 Discussion of Findings

The main implemented result is an integrated prototype with explicit approval, session, credential and contract checks. The important unresolved work concerns measured biometric behavior, recovery across database and chain failures, trustworthy audit exports and supported alternatives to camera login. The approval mechanism provides an administrative checkpoint before eligible participation, while session and credential checks help prevent unauthorized access and repeated use of voting credentials. The smart contract independently enforces configured voting rules, including election-state restrictions, candidate validity, and single-use credential digests.

8.8 Results Table

Table 8.1 evaluation should record both roster coverage and usable-profile coverage

Table 8.1: Evidence Available for the Project

Area	Evidence and interpretation
Python syntax	Seven files parsed successfully; dependency execution was not tested.
Regression inventory	16 backend, 10 frontend and 5 contract test cases found; not executed here.
Biometric performance	No measured benchmark supplied; no accuracy percentage asserted.
Blockchain performance	Contract and relay code inspected; no throughput or gas benchmark asserted.
Interfaces and reports	Source modules present; chapter figures are labelled illustrations.

8.9 Interpretation of Operational Scenarios

The same person can encounter different interface outcomes as their profile and election state change. Evaluating these scenarios helps determine whether the application explains its decisions consistently. The scenarios below describe expected behavior derived from the source, rather than measured completion rates.

8.9.1 Roster Entry Without Face Enrollment

A roster entry may contain identity metadata and an empty face template. Login detects this condition and instructs the user to complete registration. The existence of the roster record should therefore not be counted as completed biometric enrollment. An evaluation should record both roster coverage and usable-profile coverage when describing readiness for a demonstration.

8.9.2 Recognized Voter Awaiting Approval

A face match and blink sequence concern authentication. They do not replace the administrator's approval of identity details. The interface should preserve this distinction so that an enrolled user does not repeatedly recapture their face to resolve an approval delay. The correct next action is review of the eligibility record.

8.9.3 Expired Credential With an Active Session

The session lifetime and credential lifetime differ. A user can remain recognized by the application while their ballot authorization has expired. The cast endpoint checks the credential's expiry independently. This scenario tests whether the interface can recover through the appropriate authentication or eligibility workflow without implying that the earlier session guaranteed a ballot.

8.9.4 Election Closed During Ballot Review

The browser may display a candidate while the owner ends the election in another session. A later submission must still satisfy the contract's current state. The displayed review page is not a reservation of voting permission. A controlled demonstration should exercise this transition and ensure that the UI reports the rejection without showing a false transaction confirmation.

Together, these scenarios expose the difference between a page's displayed state and the authoritative state at submission time. They also provide concrete cases for evaluating status messages, recovery instructions and the freshness of data returned by the service. The frontend should handle this rejection without presenting a false success message. It should distinguish between a ballot that has been successfully recorded, a transaction that has reverted, a request rejected before transaction submission, and a submission whose final outcome is still unknown.

8.10 Reconciliation of Results and Transaction Evidence

The project presents candidate totals and turnout information derived from more than one component. A final evaluation should reconcile those values against accepted contract transactions and private credential consumption. Table 8.2 tells matching totals are useful evidence of consistency, but the evaluator must also confirm that they refer to the same network, contract and election interval.

Table 8.2: Cross Checks for an Accepted Test Ballot

Evidence source	Expected information	Cross check
Browser confirmation	Transaction reference returned by the API	Matches the receipt being inspected
Transaction receipt	Successful status on the intended contract	Belongs to the configured network and operation
VoteCast event	Candidate identifier and credential digest	Consistent with the selected test candidate
Contract state	Used digest and updated candidate total	Shows the expected single accepted increment
Private credential row	Consumption time and tx_hash	References the same accepted transaction
Voter record	Election-related voted status	Agrees with the credential record for that election

8.10.1 Handling Disagreement

If the contract indicates acceptance but the database lacks consumption, the discrepancy should be recorded as a recovery issue. If the database indicates consumption without the expected chain evidence, the operator should verify the network and contract selection before concluding that a ballot exists. A user-facing total alone cannot resolve either case.

The analytics code contains fallback behavior, and the audit endpoint can produce fallback records when chain logs are unavailable. These behaviors support interface continuity but should not be used to replace missing transaction evidence in an experimental result. A reconciled result should identify which values came from direct chain inspection and which came from private application records.

The report should preserve unresolved differences and their investigation status. This produces an assessable result even when a trial discovers a defect, and avoids presenting a visually complete dashboard as proof of a fully consistent election. It also allows invalid requests to be rejected before transaction submission whenever the relevant failure can be identified.

CHAPTER 9

APPLICATIONS AND BENEFITS OF THE SYSTEM

9.1 Introduction to Applications

is best suited to controlled academic demonstrations and research exercises. It can help participants understand how registration, biometric verification, eligibility and transaction recording fit together. Its application value is strongest when the event is clearly identified as a test and when participants understand how their data will be handled.

The project should be evaluated in a setting where administrators can assist with enrollment, camera access and failed attempts. The interface alone cannot resolve disputed eligibility or determine whether a person should be allowed to vote. Those decisions require a process outside the software.

9.2 Applications in Student Council Demonstrations

A supervised student-council mock election can demonstrate the complete workflow. An administrator prepares candidates, reviews voter records and starts the test election. Participants register, authenticate and cast test ballots, then compare the final candidate totals with the retained transaction records.

This setting supports discussion of practical questions: why does a registered face still need ID approval, why can a credential expire, and why does a confirmed chain transaction require a corresponding database update? These questions connect technical behavior with operational responsibilities.

9.3 Applications

The project can be used to teach separation of concerns and interface contracts. Students can trace a request from a React component through FastAPI, SQLite and Web3 to a Solidity function. They can then identify which layer is responsible for validation, authorization, persistence and presentation.

Classroom exercises can also compare intended behavior with actual implementation. For example, documentation describing head movement should be checked against the active blink-only route. This encourages evidence-based technical writing and makes maintenance an explicit part of software development.

Clubs and student societies can use a nonbinding demonstration to evaluate the usability of online registration and result presentation. The trial should use a controlled participant group and avoid claims of anonymity that the current protocol cannot satisfy. Feedback can reveal confusing instructions, incomplete translations and difficulties with camera permissions.

The demonstration can also compare administrative workloads. Approval queues, replacement face profiles and deleted voter records create tasks that must be supported by a clear process. Measuring time spent on these tasks would provide more useful operational evidence than counting only successful login attempts.

The Voting contract illustrates constructor configuration, immutable addresses, access-control modifiers, state enums, mappings and events. A local Ganache environment allows students to inspect accepted and rejected transactions without using real funds. Candidate counts and replay guards can be examined directly through the contract interface.

The project is especially useful for explaining the boundary between on-chain and off-chain logic. The chain rejects a reused credential digest, but it does not decide whether the issuing service enrolled the correct person. The relayer permission makes this trust relationship visible in the source.

The face and blink workflow can serve as a controlled experimental platform after an appropriate evaluation dataset is collected. Researchers can compare capture conditions, enrollment procedures and decision thresholds while retaining the same voting workflow.

Such experiments must separate detection failures, nonmatching faces and rejected blink sequences. Reporting these outcomes separately helps identify whether a problem comes from image quality, template comparison or temporal observation. The application should not attribute every failed login to an incorrect identity.

Students can investigate how candidate counts change following accepted votes, how election-state restrictions affect submissions, and how credential-digest replay guards prevent the same digest from being accepted more than once. They can also examine transaction receipts and contract events to understand how state transitions are represented on the blockchain.

A local development environment is suitable for controlled experiments, but it does not by itself establish security, production readiness, or compatibility with every deployment environment. Figure 9.1 is test outcomes should be documented alongside the contract version and deployment configuration.

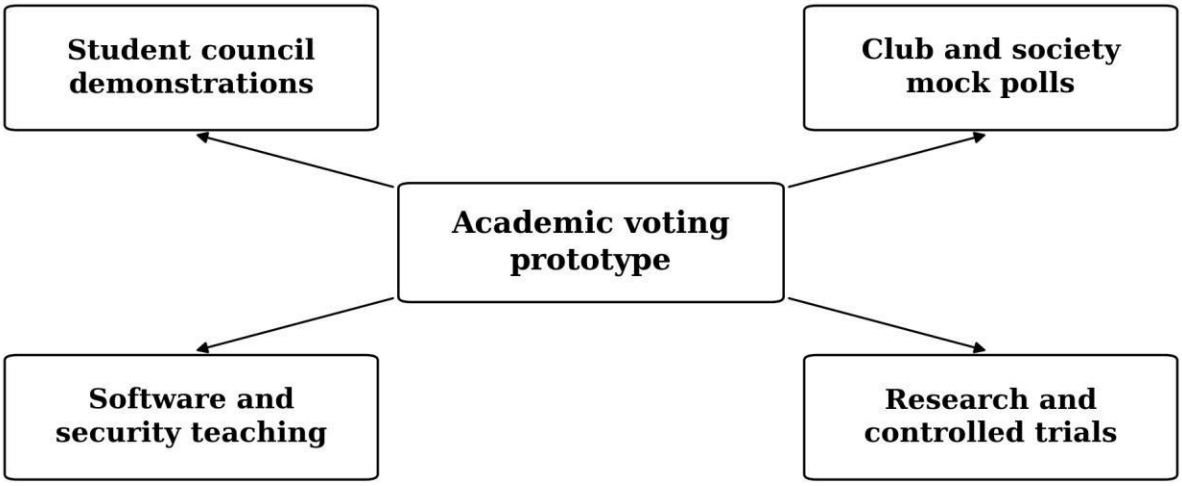


Figure 9.1: Major Academic Application Areas of Prajatantra

9.3 Benefits of the Proposed System

The application provides an explicit route from submitted details to an enrolled profile and an approved voter record. This reduces ambiguity about whether a person has merely registered or has permission to participate in the configured election. It also gives administrators a place to review replacement face templates.

The browser, API and contract are connected through defined requests and states. A participant can move through the process without manually constructing a blockchain transaction. This makes the demonstration more accessible while still allowing developers to inspect the underlying operations.

The same credential validation and contract checks are applied to supported submissions. A disabled frontend button is supplemented by backend ownership, expiry and eligibility checks and by the contract's one-use mapping. Consistency is valuable, although it remains dependent on correct implementation and deployment.

The contract does not receive the voter's name, USN or face descriptor. It records an opaque credential digest and public election information. This separation avoids directly placing the private profile fields in the contract, while the service's transaction mapping remains sensitive.

The benefit must be described accurately. Removing a direct identifier from contract storage does not prevent an authorized operator from relating a voter record to its transaction reference. A stronger privacy objective would require a different protocol and an independent analysis of metadata.

The frontend presents registration, verification, candidate review, submission and confirmation as separate stages. This structure helps users understand what they are waiting for and where an error occurred. Specific messages can distinguish a missing face profile from an unapproved ID or a failed transaction.

The project also provides a foundation for accessibility improvements through language selection, status announcements and readable controls. These features need evaluation with representative users and assistive technologies before claiming comprehensive accessibility.

The demo launcher, test files and configuration examples make it easier to repeat a local workflow. Reproducibility improves when the operator records dependency versions, the deployed contract address, candidate configuration and test evidence. A repeatable demonstration allows another evaluator to inspect both ordinary behavior and controlled failures.

The strongest benefit is educational traceability: a visible interface action can be connected to a specific API route, database update, contract function and result. Figure 9.2 creates a concrete basis for explaining the system and for identifying where future improvements should be made.

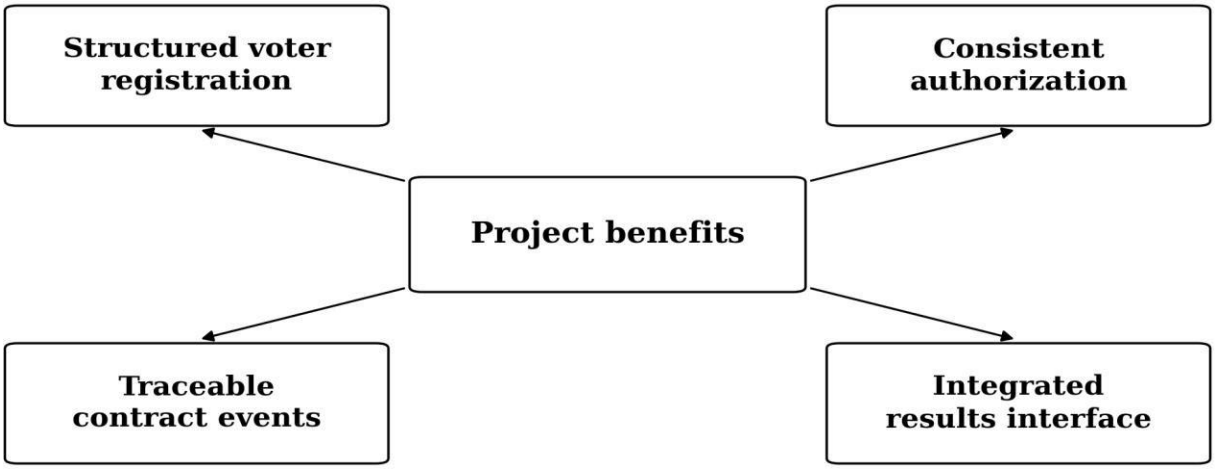


Figure 9.2: Key Benefits of the Proposed System

9.4 Role of Machine Learning in Voter Authentication

Machine learning contributes pretrained face-processing capabilities through the installed dependencies. The application uses these components to locate a face, estimate facial structure and produce a descriptor. It then applies configured comparison rules within a larger authentication workflow. The relayer's permissions make this trust relationship particularly visible. The contract can restrict transaction submission to an authorized relayer.

The project does not learn voter eligibility from historical records. Eligibility is an administrative decision stored in the registry. Likewise, the contract does not use a learned model to select a winner; it increments explicit candidate counts for accepted submissions.

This distinction makes the design easier to evaluate. Recognition quality belongs to the biometric stage, while vote-count correctness belongs to the transaction and state logic. Improvements in one stage do not automatically validate the other.

9.9 Practical Example of System Usage

A demonstration organizer prepares a test contract and candidate list, then enrolls and approves consenting participants. Each participant completes a fresh login challenge, reviews a candidate and submits a ballot. Figure 9.3 is about after the election closes, the organizer compares contract totals, successful receipts and private consumption records. The workflow is summarized on the next page.

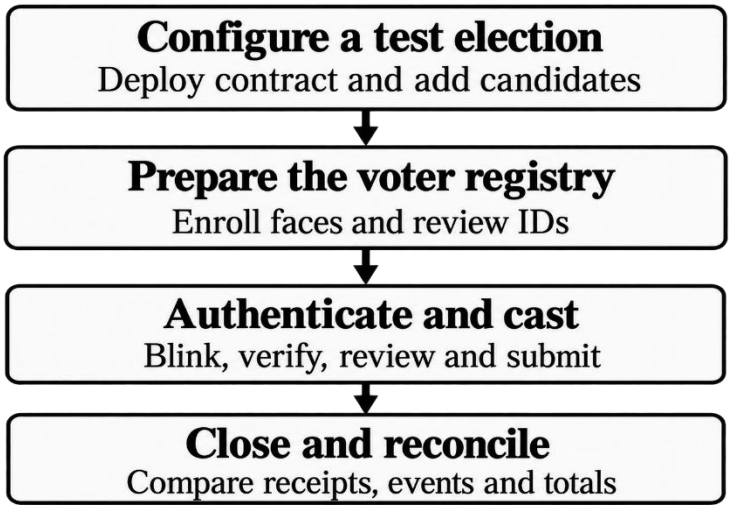


Figure 9.3: Practical Workflow for a Controlled Test Election

A supervised demonstration should use a local test election, labelled sample candidates and consenting participants. Its record should identify the software configuration and distinguish illustrative data from records produced during the session.

First, start the configured services and confirm the deployed contract address, network and owner. Check that the relayer has the intended address and that the API can reach the node. Open the registration interface and enroll a participant using multiple usable images. Explain why an initial roster entry alone does not contain a biometric profile.

CHAPTER 10

FUTURE SCOPE AND ENHANCEMENTS

10.1 Introduction

Future development should be driven by evidence from controlled tests and by a clearly defined intended use. The current prototype already connects the main workflow components. The next steps should improve the reliability of that connection, measure biometric behavior and make privacy and accessibility objectives explicit.

An enhancement should identify the limitation it addresses, the code or process it changes and the evidence required to assess success. This avoids treating a new library or a redesigned interface as sufficient proof of a better voting system.

10.2 Improvement in Evaluation Data

A consent-based evaluation dataset should contain separate enrollment and verification sessions. It should record capture conditions and distinguish genuine attempts, impostor attempts, acquisition failures and permitted presentation-attack trials. The dataset should represent the devices and users expected in the demonstration setting.

Threshold selection and final evaluation should use separate data. The team should retain the number of attempts, the selection procedure and uncertainty around reported rates. Biometric data should remain private, with access limited to the evaluation team and a documented deletion schedule.

10.3 Advanced Biometric Methods

Alternative face descriptors or presentation-attack detectors could be assessed against the existing baseline. A replacement should demonstrate an improvement under comparable conditions rather than rely on a benchmark from a different dataset. Model size, latency, acquisition failures and accessibility effects should be evaluated along with recognition errors.

The application would also need a template-migration strategy. Existing stored descriptors cannot be assumed compatible with the output of a different model. A controlled re-enrollment procedure may be safer than silently comparing incompatible representations. All metrics should be defined before testing, with sample counts, test conditions, thresholds, and uncertainty reported where appropriate. Results should distinguish recognition errors from image-acquisition failures and presentation-attack detection outcomes.

10.4 Explainable Authentication Feedback

Future feedback should explain the operational reason for a rejected attempt without exposing sensitive matching details. Examples include insufficient camera frames, poor visibility of both eyes, a missing profile and expired session state. Figure 10.1 is internal diagnostic records should support evaluation while avoiding unnecessary disclosure of biometric information.

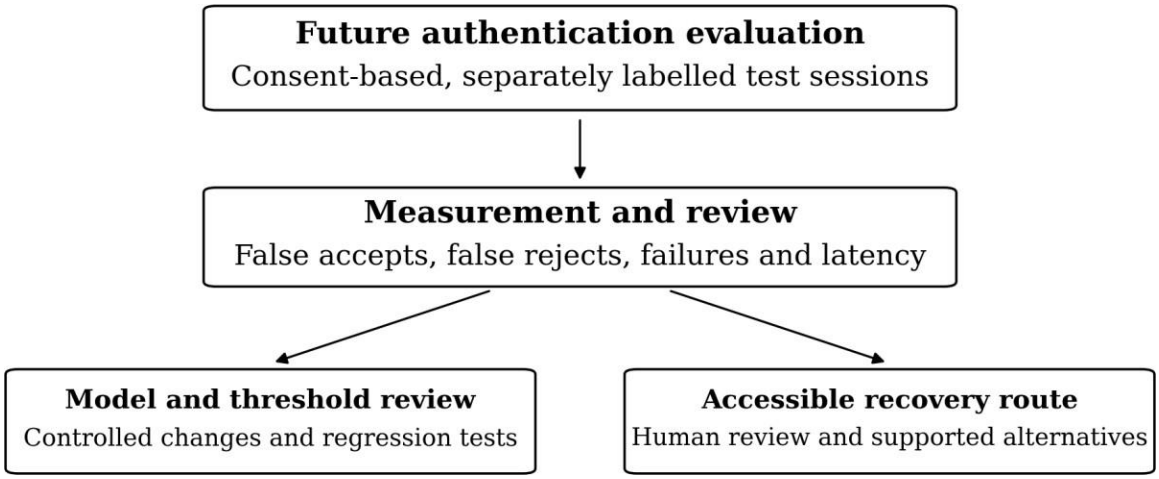


Figure 10.1: Proposed Framework for Authentication Evaluation

The evaluation process should connect measured failures to a review of capture instructions, thresholds or supported alternatives. It should not automatically relax authentication after repeated failures. Any exception pathway must preserve the administrator's eligibility responsibility and retain an appropriate record.

OCR can be improved by separating text extraction from field validation and human confirmation. The current optional server route uses limited patterns and does not verify document authenticity. Future versions could provide clearer confidence cues for extracted fields and require explicit review before saving a profile.

Face-replacement requests should also provide administrators with enough context to make a careful decision. The active profile should remain protected while a pending update is reviewed, and approval actions should be traceable to the administrator who performed them. The current Prajatantra implementation includes an optional server-side OCR route that uses limited pattern-based checks to extract information from submitted documents. Although this functionality can assist with data entry, text extraction alone does not establish that the extracted information is accurate or that the source document is genuine.

The relayer should use a durable submission queue and explicit nonce coordination when handling concurrent requests. A stored intent identifier can help associate a browser action with its eventual transaction. Recovery should distinguish an unsent request, a submitted transaction, a confirmed receipt and a failed local update.

A reconciliation process should inspect chain receipts after timeouts or restarts. It should avoid issuing a replacement voting opportunity merely because a client did not receive a response. This is particularly important because the current database transaction remains open while the relayer waits for confirmation.

10.5 Election History and Reconciliation

Future data management could retain separate election records with explicit contract addresses and archival status. Candidate totals, VoteCast events and credential consumption records should be compared before a final result is released. Any discrepancy should be reported rather than hidden by selecting the larger available total.

The interface should show clear data-source status and consistent translations across static and dynamic messages. Keyboard navigation, screen-reader feedback and alternatives to biometric interaction should be tested. Tie handling and empty-election states should also have explicit behavior.

Audit exports should remove official-looking claims that are not supported by project authorization. Synthetic fallback block data should be removed or unmistakably separated from verified chain records. The report should state whether it contains an ECDSA signature or an HMAC, because those support different verification arrangements.

Monitoring should track failures and timing without retaining unnecessary raw camera content. Model, threshold and deployment changes should trigger a regression evaluation with the same controlled cases, allowing the team to identify changes in both usability and enforcement. Monitoring records should capture relevant events across the frontend, backend, biometric authentication module, database, relayer, and smart contract. Each event should contain sufficient information to identify the stage of processing and determine whether the operation succeeded, failed, or remains unresolved.

A simple change record can make this process repeatable. Each enhancement should identify its objective, affected components, preserved behavior, acceptance criteria, test status, evidence location, and documentation updates.

10.6 Security and Data Privacy Enhancements

The private registry should have a defined retention policy, protected backups and restricted administrative access. Key-management procedures should address relayer compromise and session-secret rotation. Because owner and relayer addresses are immutable in the current contract, replacing those roles requires a carefully managed new deployment.

A secret-ballot objective would require a different voting protocol, potentially involving encrypted ballots, proofs and an appropriate tallying process. Merely hashing another identifier would not provide the required privacy property. The threat model should explicitly include the service operator and transaction metadata.

Multi-worker operation requires shared or coordinated challenge storage, safe database access and relayer transaction ordering. Load testing should measure processing and queue behavior under realistic camera requests. A successful local demonstration does not establish capacity for a large number of simultaneous voters.

10.8 Summary of Proposed Enhancements

The main improvement areas are shown below. They should be treated as separate workstreams with their own acceptance evidence. Each regression run should identify the software version, model version, threshold, deployment configuration, test dataset or fixture version, and execution environment. Results should be compared with a recorded baseline. Figure 10.2 change should not be considered an improvement solely because the number of successful logins increases.

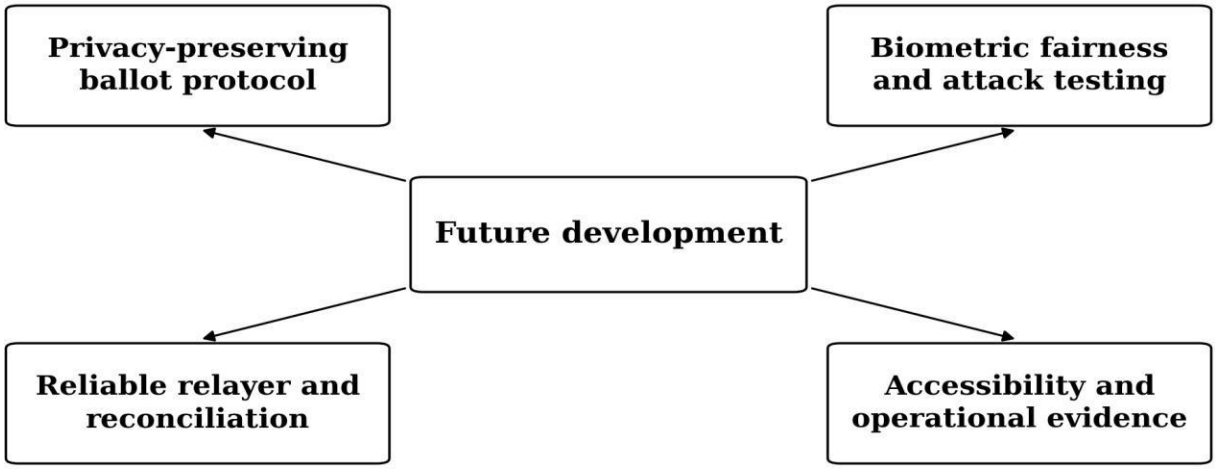


Figure 10.2: Major Future Enhancement Areas for the System

10.9 Prioritized Development Roadmap

Future work should be ordered by the dependencies between improvements. A new interface or biometric model cannot compensate for an unresolved transaction recovery path. The following roadmap groups related changes into stages that can each produce reviewable evidence before the next stage begins.

Create a documented environment in which the supplied frontend, backend and contract tests execute consistently. Record the exact dependencies, model assets and contract configuration. Add the missing integration cases identified in Chapter 7, including wrong-election credentials, concurrent submissions and interrupted requests. This establishes a baseline against which later modifications can be assessed.

Introduce durable tracking for ballot submission attempts and coordinate relayer nonces. Separate transaction broadcast, confirmation and private persistence into observable states. Add a reconciliation process that identifies incomplete operations without granting an accidental second authorization. Correct report generation so that unavailable chain evidence remains explicitly unavailable.

Collect an appropriately consented evaluation dataset with independent sessions. Measure genuine and impostor outcomes, acquisition failures and timing under stated conditions. Review the blink capture and error messages with participants who use different devices or need assistance. Design an authorized recovery route that preserves eligibility checks when camera login is unsuitable.

Define the required ballot privacy and verification properties before selecting a new protocol. Evaluate how identity approval can be separated from ballot content, what an observer can verify and which parties remain trusted. Any proposed encryption or proof system should include a threat model and a practical verification procedure.

Each stage should produce an implementation record, a test record and a statement of remaining limitations. Completing one stage should not be used to imply that later privacy or election-governance objectives have already been achieved. Privacy-preserving monitoring and repeatable regression testing provide a structured way to evaluate both usability and enforcement in Prajatantra. Recording operational metadata rather than unnecessary raw biometric content supports troubleshooting while reducing privacy exposure. Repeating the same controlled tests after relevant changes helps identify regressions.

10.10 Acceptance Criteria for Future Enhancements

An enhancement is assessable when its intended behavior is stated before implementation and connected to evidence. The criteria below provide a starting point for planning, while keeping performance targets open until the project has a measured baseline. Table 10.1 is numerical targets should be justified by the intended demonstration environment and evaluation design.

Table 10.1: Proposed Enhancement Acceptance Evidence

Enhancement	Required behavior	Evidence before completion
Submission recovery	Identify broadcast and confirmed operations after interruption	Fault-injection logs and reconciled chain and database records
Relayer coordination	Avoid conflicting nonces under controlled concurrency	Concurrent request results and accepted transaction sequence
Trustworthy audit export	Distinguish retrieved chain evidence from unavailable data	Export compared with independently inspected receipts
Biometric evaluation	Report outcomes with defined labels and denominators	Independent trial records and threshold-specific counts
Accessible recovery	Provide a usable supported path while preserving approval	Task observations and authorization tests
Privacy protocol	Meet explicitly stated privacy and verification goals	Protocol analysis, implementation tests and verification procedure

Every enhancement should identify which existing behavior must remain valid. Changes to descriptor processing require an enrollment compatibility decision. Changes to credential handling require replay and ownership tests. Changes to exports require review of both the underlying evidence and the labels shown in the generated document.

The report should be updated from completed evidence rather than from planned features. Proposed work belongs in this chapter until implementation and validation support moving it into the architecture, methodology and results chapters. Maintaining that distinction allows a reviewer to understand the current system while also assessing a concrete path toward improvement.

A disciplined enhancement process should preserve existing controls, define measurable acceptance criteria, and require appropriate regression evidence. Maintaining a clear separation between implemented functionality, completed evaluation, and proposed work will make the Prajatantra report more accurate, reproducible, and useful for technical review. The report should retain unresolved issues and failed tests.

CHAPTER 11

CONCLUSION

It is a Multimodal Blockchain Voting System that integrates a web interface, a face-authentication service, a private voter registry and an Ethereum-compatible contract. The project organizes registration, eligibility approval, login, credential issuance, ballot submission and results into a connected academic workflow.

The architecture makes several responsibilities explicit. The backend verifies a registered face and a blink sequence, the administrator controls ID approval, the credential service limits voting permission and the contract applies selected election rules. This separation is the central engineering contribution of the prototype.

The source implements registration and login pages, protected voter and administrator routes, candidate management, one-time credential handling and public analytics. The active contract restricts administrative operations to its owner and ballot submissions to its relayer. It accepts votes only in the active state and rejects reuse of a credential digest.

The project also includes automated test files and a local demo launcher. These support maintainability and future evaluation. The report's observed validation is limited to source inspection and successful syntax checking of seven Python files; complete execution and biometric measurement remain separate activities.

Pretrained face-processing components supply descriptors and landmarks used in authentication. The application contributes the surrounding enrollment, challenge and authorization workflow. It does not train a new face-recognition model or establish a benchmark accuracy result.

Machine learning therefore supports identity comparison, while deterministic application and contract rules control eligibility and vote recording. Describing these roles separately makes the system easier to explain, test and improve.

The project provides a concrete setting for learning how browser applications interact with biometric services, private databases and smart contracts. A developer can trace a selected candidate from the review screen to the cast endpoint, the relayer transaction and the resulting contract event. An evaluator can also identify where the workflow relies on administrator decisions or private service state.

This traceability is useful for both demonstrations and maintenance. It exposes differences between documentation and implementation, such as the active blink-only login path and the presence of an older contract in a separate folder. It also makes unresolved recovery and privacy issues visible before a broader application is considered.

The design should be used in controlled academic trials with clear participant expectations. Its public candidate data and service-held transaction references mean that secret-ballot guarantees are outside the current implementation. Camera-related exclusion and administrator support also require attention.

demonstrates a coherent prototype for approved voter authentication and blockchain-backed test-ballot recording. The principal implemented controls include face-template matching, a one-use liveness challenge, administrative eligibility, short-lived random credentials, relayer-only submission and contract replay prevention.

The system's capabilities should be stated at this level of evidence. It is not an independently certified election platform, and no unmeasured accuracy or scalability result is claimed. The most valuable next steps are a reproducible end-to-end evaluation, robust transaction reconciliation, trustworthy audit exports and a defined privacy and accessibility strategy.

With those improvements, the project can become a stronger research and teaching platform. Its current contribution is the integration of clearly identifiable components and rules, together with a practical foundation for examining how identity, authorization and public transaction records interact. The current architecture does not establish a secret-ballot guarantee. Candidate selections are recorded through public blockchain transactions, and service-held transaction references may connect transaction activity to private voter records. Therefore, the system should not be represented as providing guaranteed voter anonymity or ballot secrecy.

Maintaining this traceability also helps identify risks before the system is considered for broader use. The implementation should be evaluated against the actual authentication workflow, credential lifecycle, contract permissions, database records, and transaction-handling behavior rather than relying solely on architectural descriptions.

The system is not an independently certified election platform, and no unmeasured accuracy or scalability result is claimed. Its next stage should focus on reproducible evaluation, reliable cross-system recovery, verifiable audit reporting, and a clearly defined privacy and accessibility strategy.

CHAPTER 12

LIMITATIONS AND CHALLENGES

The supplied archive contains a roster and implementation files but no independent labelled benchmark for the complete camera-authentication workflow. Consequently, the report cannot estimate genuine acceptance, false acceptance, false rejection or attack resistance from the available files. A roster count is not a recognition-test sample size.

Lighting, camera quality, pose, spectacles and individual eye geometry can affect the usable observations. The relative blink rule attempts to account for a changing baseline, but its effect across user groups has not been measured here. An evaluation must examine failures to acquire usable images as well as final matching decisions.

Users who cannot perform the instructed blink or obtain an adequate capture need a supported route for assistance. Treating every acquisition failure as an authentication failure can create avoidable exclusion and obscure the real source of the problem.

The application expects a valid 128-dimensional descriptor from its configured dependencies. A vector with the right length is not automatically compatible with a different model or model version. Library changes require regression testing and may require re-enrollment.

OCR assists with reading selected fields but does not establish that an ID document is genuine or belongs to the person submitting it. The optional server extractor uses limited text patterns and may leave fields unresolved. Administrative approval remains a trusted manual step, and incorrect approval can authorize the wrong profile despite later successful face matching.

A fresh challenge limits straightforward replay of the same challenge identifier, while the eye sequence requires a temporal change. However, the challenge is fixed to blink_once and the server receives ordinary image data from the client. The protocol does not cryptographically bind an untampered live camera stream to the challenge. A capable replay or synthetic input therefore requires explicit evaluation.

Computer-vision processing, SQLite writes and chain confirmation can all delay the workflow. The cast route holds an immediate database transaction while awaiting the relay result. This can serialize write activity and requires analysis under concurrent load. The current relayer path also needs coordinated nonce handling for multiple simultaneous requests.

The default tolerance of 0.55 is a comparison threshold. It is not 55 percent accuracy, a calibrated identity probability or proof of a person's legal identity. Enrollment consistency checks use another threshold for a different purpose. Both require controlled evaluation before changing them.

The audit endpoint creates fallback transaction and block-like records when chain logs are unavailable. It also constructs a canonical digest rather than a complete independently verified election proof. ECDSA signing is attempted with the relayer key; the fallback is HMAC-SHA256, which depends on a shared secret.

These behaviors limit how exported reports can be interpreted. An authentic-looking PDF cannot make synthetic block fields authentic, and a signature does not validate the truth of the signed inputs. Reliable audit reporting must preserve evidence provenance and distinguish unavailable data from verified chain records.

Audit exports should therefore identify the source and verification status of each record. Verified chain data, application-database records, derived summaries and synthetic fallback values must be clearly distinguished. When data is unavailable, the report should state that limitation rather than silently substituting a record that resembles confirmed blockchain evidence.

A reliable audit process should preserve relevant transaction hashes, verified receipts, block references, timestamps, record-generation methods and verification outcomes. Signature-verification procedures and key-management practices should also be documented. These controls would make exported reports more transparent and useful for subsequent review.

The identified limitations do not negate the educational value of Prajatantra, but they define the boundaries of what can currently be claimed. The implementation combines biometric processing, eligibility management, challenge-based authentication, database records and blockchain-backed ballot submission. However, the available evidence does not establish measured biometric accuracy, comprehensive presentation-attack resistance, performance under substantial concurrent load or independently verifiable election-wide auditability.

The next stage should prioritize a reproducible end-to-end evaluation, accessibility and assisted-authentication procedures, regression testing for model changes, coordinated relayer operation, transaction reconciliation and evidence-preserving audit exports. Until these areas have been implemented and assessed with documented results, the system should be presented as an academic prototype for controlled demonstrations and research.

CHAPTER 13

PUBLICATION OF JOURNAL

13.1 Publication Position

Publication of this project has not been established by the available project records. No acceptance letter, published article identifier or publication certificate is asserted in this report. A manuscript can be prepared from the implementation and from reproducible evaluation evidence when that evidence is available.

13.2 Proposed Manuscript Title

Multimodal Voter Authentication and Blockchain Based Ballot Recording

13.3 Proposed Abstract

This project presents an academic voting prototype that integrates a React and TypeScript interface, a FastAPI authentication service, a SQLite voter registry and a Solidity election contract. Registration stores a multi-image face descriptor, while login combines a fresh server challenge with a temporal blink check and face matching. Administrative approval remains separate from biometric authentication. Eligible users receive a short-lived random credential, and a server-controlled relayer submits a candidate identifier and credential digest to the contract. The contract restricts election administration, enforces its active state and rejects reused digests. The implementation includes result displays, report export and regression-test files. The design illustrates the interaction of identity, authorization and public transaction records, while retaining centralized trust and limitations in ballot secrecy, liveness assurance and failure recovery. A complete evaluation should measure biometric errors, transaction behavior and end-to-end reconciliation under controlled conditions.

13.4 Contribution and Evaluation Scope

A suitable manuscript should focus on the integrated workflow and the separation of application responsibilities. Any claim of improved accuracy, performance or privacy must be supported by an appropriate comparison and retained evidence. The implementation should not be presented as a new face-recognition model or a certified public-election protocol.

Diagnostic records should preserve the operation stage and available transaction reference without recording private keys or raw credentials. User-facing feedback should explain whether to authenticate again, wait for approval or seek assistance, rather than offering an unrestricted retry for every error.

13.5 Conceptual Model for the Manuscript

The central design distinction is between private identity processing and public election recording. Figure 13.1 following model can support a manuscript discussion of the trust boundary and its limitations.

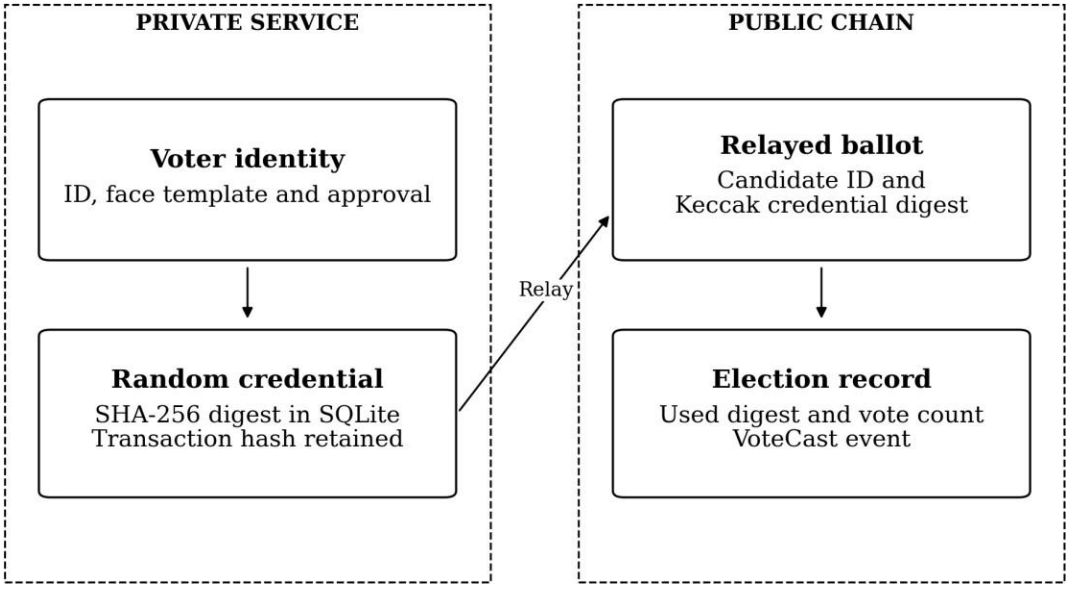


Figure 13.1: Authentication and Ballot Recording Trust Boundaries

13.6 Evidence Required Before Submission

- A fixed source version, dependency environment and deployed contract reference.
- A documented evaluation protocol with consent and data-retention arrangements.
- Genuine, impostor and presentation-attack results with stated trial counts.
- Executed regression and integration results, including interrupted submissions.
- A clear account of public candidate visibility and private transaction linkage.
- Verified bibliographic information and an accurate author-contribution statement.

Publication metadata and any issued certificates should be added only after they are received and verified. The conceptual model above is a project diagram, not evidence of journal acceptance or publication.

REFERENCES

Adida, B. (2008). Helios: Web-based open-audit voting. In *Proceedings of the 17th USENIX Security Symposium*. USENIX Association.

Clarkson, M. R., Chong, S., & Myers, A. C. (2008). Civitas: Toward a secure voting system. In *2008 IEEE Symposium on Security and Privacy*. IEEE.

McCorry, P., Shahandashti, S. F., & Hao, F. (2017). A smart contract for boardroom voting with maximum voter privacy. In *Financial Cryptography and Data Security*. Springer.

Specter, M. A., Koppel, J., & Weitzner, D. (2020). The ballot is busted before the blockchain: A security analysis of Voatz, the first internet voting application used in U.S. federal elections. In *Proceedings of the 29th USENIX Security Symposium* (pp. 1535–1553). USENIX Association.

Park, S., Specter, M., Narula, N., & Rivest, R. L. (2021). Going from bad to worse: From internet voting to blockchain voting. *Journal of Cybersecurity, 7*(1), tyaa025.

Soukupová, T., & Čech, J. (2016). Real-time eye blink detection using facial landmarks. In *Proceedings of the 21st Computer Vision Winter Workshop*.

Kazemi, V., & Sullivan, J. (2014). One millisecond face alignment with an ensemble of regression trees. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 1867–1874). IEEE.

King, D. E. (2009). Dlib-ml: A machine learning toolkit. *Journal of Machine Learning Research, 10*, 1755–1758.

Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.

Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*. IEEE.

U.S. Election Assistance Commission. (2021). *Voluntary voting system guidelines version 2.0*.

Multimodel Voting System Project. (2026). *README.md and package.json* [Source code]. Supplied Multimodel-Voting-System-master source archive. Inspected October 27, 2026.

Multimodel Voting System Project. (2026). *Server face-recognition modules: main.py, config.py, face_utils.py, and test_main.py* [Source code]. Supplied source archive.

Multimodel Voting System Project. (2026). *Voting.sol, deployment configuration, and Voting.test.js* [Source code]. Supplied source archive.

Multimodel Voting System Project. (2026). *src/pages, src/context, src/services, and src/lib* [Source code]. Supplied source archive.

APPENDICES

Appendix A Frontend Source Code

A.1 Shared API Request and Ballot Submission

The following excerpt from src/services/api.ts shows credentialed JSON requests and the candidate-only ballot body. The raw voting credential is read by the backend from its HttpOnly cookie; it is not added to this JSON object.

```
export async function apiRequest<T>(
  path: string,
  init: RequestInit = {},
): Promise<T> {
  const cleanBase = API_BASE_URL.replace(/\/+$/, "");
  const cleanPath = path.startsWith("/") ? path : `/${path}`;
  const url = cleanPath.startsWith("/api/v1")
    ? `${cleanBase}${cleanPath.slice(7)}`
    : `${cleanBase}${cleanPath}`;

  const response = await fetch(url, {
    ...init,
    credentials: "include",
    headers: {
      "Content-Type": "application/json",
      ...(init.headers || {}),
    },
  });

  return parseResponse(response) as Promise<T>;
}

export function castVote(candidateId: number) {
  return apiRequest<{
    success: boolean;
    transaction_hash: string;
    tx_hash?: string;
  }>("/api/v1/voter/cast", {
    method: "POST",
    body: JSON.stringify({
      candidate_id: candidateId,
    }),
  });
}
```

The helper centralizes the API base and passes response failures to the caller. Voting.tsx uses the returned transaction_hash or tx_hash field to update its confirmation state after the request succeeds. The voting smart contract restricts successful vote submission to the configured relayer, which is responsible for submitting transactions on behalf of authenticated and eligible voters. This access-control mechanism limits direct ballot submission by unauthorized blockchain accounts, subject to the correctness of the contract configuration and relayer authorization. A reliable evaluation should use a documented test protocol with labelled genuine and impostor attempts.

A.2 Camera Sequence Capture

The following excerpt from src/pages/VoterLogin.tsx shows the active capture method. It records 14 frames and displays the blink instruction during capture. The server remains responsible for deciding whether the submitted sequence satisfies its checks.

```
const captureLivenessSequence = async (
  video: HTMLVideoElement,
  canvas: HTMLCanvasElement,
  onProgress: (message: string) => void,
): Promise<string[]> => {
  const context = canvas.getContext("2d");
  if (!context) {
    throw new Error(
      "The browser could not prepare the camera capture. Please try again.",
    );
  }
  const maxFrameDimension = 720;
  const scale = Math.min(
    1,
    maxFrameDimension / Math.max(video.videoWidth, video.videoHeight),
  );
  canvas.width = Math.round(video.videoWidth * scale);
  canvas.height = Math.round(video.videoHeight * scale);
  const frames: string[] = [];
  // Sample long enough to catch a natural blink without making login drag.
  const maxFrames = 14;
  const frameDelayMs = 100;
  for (let index = 0; index < maxFrames; index += 1) {
    if (index === 3) onProgress("Blink once now; keep both eyes visible.");
    if (index === 11) onProgress("Hold still; the capture is finishing…");
    context.drawImage(video, 0, 0, canvas.width, canvas.height);
    frames.push(canvas.toDataURL("image/jpeg", 0.72));
    if (index < maxFrames - 1) {
      await new Promise((resolve) =>
        window.setTimeout(resolve, frameDelayMs),
      );
    }
  }
  return frames;
};
```

The nominal interval is a browser timing setting. Actual capture and verification time depends on the device, event loop, image transfer and backend processing. This excerpt does not establish a measured latency result.

The supplied project archive contains a voter roster and implementation files, but it does not include an independent, labelled benchmark covering the complete camera-based authentication workflow. Therefore, the available evidence is insufficient to calculate genuine acceptance rate, false acceptance rate, false rejection rate or resistance to presentation attacks. The number of registered voters must not be interpreted as the sample size of a recognition benchmark, as registration and controlled authentication testing measure different aspects of the system.

Appendix B Backend Source Code

B.1 Face Descriptor Validation

The following function from server/face-recognition/main.py validates the representation before it is used. It rejects an unexpected shape, nonfinite values and a zero-norm descriptor.

```
def validate_embedding(embedding: np.ndarray) -> np.ndarray:
    vector = np.asarray(embedding, dtype=np.float32)

    if vector.shape != (128,):
        raise ValueError("Invalid face embedding shape.")

    if not np.isfinite(vector).all():
        raise ValueError("Face embedding contains invalid values.")

    norm = float(np.linalg.norm(vector))

    if norm <= 0:
        raise ValueError("Face embedding is empty.")

    return vector
```

B.2 Descriptor Comparison

The comparison function uses face_recognition to calculate a distance and returns both the decision and distance for internal processing. The configured tolerance defaults to 0.55.

```
def compare_faces(
    known_encoding: np.ndarray,
    test_encoding: np.ndarray,
    tolerance: float = MATCH_TOLERANCE,
) -> tuple[bool, float]:
    known = validate_embedding(known_encoding)
    test = validate_embedding(test_encoding)
    distance = float(face_recognition.face_distance([known], test)[0])
    return distance <= tolerance, distance


_LIVENESS_CHALLENGES: dict[str, dict] = {}
```

These functions validate and compare descriptors; they do not establish administrative eligibility. The login and cast routes perform their additional challenge, session and approval checks separately. Internal distance values are not included in the ordinary client-facing authentication response.

A reproducible end-to-end validation run should retain the execution environment, dependency versions, relevant configuration, deployed contract address, blockchain network or chain ID, command output, error logs and transaction receipts. Sensitive configuration values, private keys, session credentials and raw biometric data should not be included in general-purpose test reports.

Appendix C Smart Contract Source Code

C.1 Vote Acceptance and Replay Prevention

The following excerpt is taken from the active server/blockchain/contracts/Voting.sol. It shows the state, candidate and credential guards that run for every relayed vote, followed by the replay-status accessor.

```
function vote(
    uint256 candidateId,
    bytes32 credential
) external onlyRelayer {
    require(state == ElectionState.Active, "Election is not active");
    require(
        candidateId > 0 && candidateId <= countCandidates,
        "Invalid candidate"
    );
    require(credential != bytes32(0), "Credential is required");
    require(
        !credentialUsed[credential],
        "Voting credential has already been used"
    );

    credentialUsed[credential] = true;
    candidates[candidateId].voteCount++;

    emit VoteCast(candidateId, credential, block.timestamp);
}

function checkCredential(
    bytes32 credential
) external view returns (bool) {
    return credentialUsed[credential];
}
```

Only the configured relayer can call vote successfully. Candidate selection and the digest are public transaction data. The contract contains no voter face descriptor and does not independently determine whether the private eligibility review was correct.

The public visibility of candidate selection also limits the system's privacy guarantees. Although the contract does not store facial descriptors, public transaction data and privately retained transaction references may allow an observer with access to the relevant records to associate a voter with a ballot. The current design should therefore not be described as guaranteeing voter anonymity or secret-ballot protection. Stronger privacy would require additional protocol-level safeguards and an evaluation of how public transaction data, relayer activity and private application records could be correlated. This separation creates a trust boundary between the application and the blockchain. The application is responsible for biometric authentication, eligibility decisions and credential lifecycle management, while the contract enforces its on-chain authorization and voting rules.

Appendix D Database Source Code

D.1 Voter Table

The initial voter table stores identity-related fields and the face descriptor. Additional approval, deletion and election-tracking columns are added by the initialization migration logic in main.py.

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS voters (
        voter_id       TEXT PRIMARY KEY NOT NULL,
        role           TEXT NOT NULL DEFAULT 'user',
        face_encoding  TEXT NOT NULL,
        name           TEXT,
        usn            TEXT,
        branch         TEXT,
        validity       TEXT,
        dob            TEXT
    )
""")
```

D.2 Voting Credential Table

This separate table records the credential digest and lifecycle metadata. The transaction reference is private application data that can link the voter record to a public ballot transaction.

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS voting_credentials (
        credential_hash TEXT PRIMARY KEY NOT NULL,
        voter_id TEXT NOT NULL,
        election_id TEXT NOT NULL DEFAULT '',
        issued_at TEXT NOT NULL,
        expires_at TEXT NOT NULL,
        consumed_at TEXT,
        revoked_at TEXT,
        tx_hash TEXT
    )
""")
```

The application uses parameterized SQL for operational queries and updates. SQLite transactions coordinate its local writes, but they cannot make an already confirmed blockchain transaction disappear if a later local operation fails.

The credential digest and the transaction reference have different security and privacy implications. The digest supports credential tracking and replay prevention, while the transaction hash provides a reference for locating the associated blockchain transaction. Although these fields are maintained in the private application database, an operator with access to the database may be able to associate a voter record with a public ballot transaction. Consequently, keeping voter names and other direct identifiers off-chain does not, by itself, guarantee voter anonymity or ballot secrecy. The application uses parameterized SQL statements for operational queries and updates, helping reduce the risk of SQL injection when values are supplied as parameters.

Appendix E Configuration and Reproduction

E.1 Selected Default Settings

The following excerpt from server/face-recognition/config.py shows configurable session, matching, storage and relayer settings. Secret values are empty by default and must be supplied privately for an actual deployment.

```
JWT_EXPIRY_HOURS: int = 2

# ── Face Matching ──────────────────────────────────────
MATCH_TOLERANCE: float = 0.55

# ── Redis ──────────────────────────────────────────────
REDIS_URL: str = "redis://127.0.0.1:6379/0"
FACE_DB_PATH: str = ""
FRONTEND_ORIGINS: str = (
    "http://localhost:8080,http://127.0.0.1:8080,"
    "http://localhost:5173,http://127.0.0.1:5173"
)
COOKIE_SECURE: bool = False
COOKIE_SAMESITE: str = "lax"
BLOCKCHAIN_RPC_URL: str = "http://127.0.0.1:7545"
BLOCKCHAIN_CONTRACT_ADDRESS: str = ""
BLOCKCHAIN_RELAYER_ADDRESS: str = ""
BLOCKCHAIN_RELAYER_PRIVATE_KEY: str = ""
VOTING_CREDENTIAL_TTL_MINUTES: int = 15
```

The configured EAR values remain available to the backend. The current blink function uses a relative bilateral pattern and an open-eye baseline; these settings should be interpreted with that function rather than as a complete independent liveness algorithm.

```
EAR_MIN_CLOSED: float = 0.22
EAR_MIN_OPEN: float = 0.25
```

E.2 Reproduction Commands

The repository documents make demo for the integrated local launcher and make test for its test targets. Individual commands include pnpm run test, pnpm run build, python -m pytest server/face-recognition -q and pnpm run test:blockchain. A complete validation run should retain its environment, contract address, logs and receipts before reporting a result.

These commands address different aspects of validation. A successful frontend build establishes that the build process completed; it does not demonstrate that biometric authentication, database persistence and blockchain submission work correctly together. Similarly, passing isolated unit or contract tests does not establish the reliability of the complete voting workflow. A reproducible end-to-end validation run should retain the execution environment, dependency versions, relevant configuration, deployed contract address, blockchain network or chain ID, command output, error logs and transaction receipts.

Appendix F API Reference

F.1 Principal Browser and Service Operations

The following route reference summarizes selected operations implemented in main.py. All listed paths use the /api/v1 prefix. The table is a navigation aid for the source and does not replace the request models, authorization dependencies or current-state checks within each handler.

Table F.1: Selected API Routes and Their Roles

Method	Path after /api/v1	Principal purpose and input
POST	/register-user	Submit profile fields and enrollment images for public registration
GET	/liveness/challenge	Issue a challenge bound to the voter_id query value
POST	/verify-face	Verify voter_id, challenge_id and images_base64
POST	/admin-login	Authenticate administrator credentials
GET	/auth/me	Read the session represented by the authentication cookie
POST	/auth/refresh	Refresh the current authentication session
POST	/auth/logout	Clear cookies and attempt unused-credential revocation
GET	/voter/eligibility	Read current approval, voting and credential-related state
POST	/voter/cast	Submit candidate_id with the authenticated credential cookies
GET	/election/analytics	Read public election totals and related analytics
GET	/admin/election/audit	Obtain the administrator-only report data

F.2 Contract Operations

The active Voting contract exposes addCandidate, startElection and endElection to its owner. The vote method is restricted to the relayer. Read operations include getCandidate, getCountCandidates, getElectionState and checkCredential. Public state getters additionally expose the declared public fields and mappings.

An API route and a contract function are different interfaces. The voter submits through the service, while the relayer submits the signed contract call. An integration test must identify which interface it invokes and which authority is expected to permit or reject that operation. A reproducible end-to-end validation run should retain the execution environment, dependency versions, relevant configuration, deployed contract address, blockchain network.

Appendix G Terminology and Interpretation

G.1 Project Glossary

The following terms are used consistently throughout the report. Their definitions distinguish user identity, permission to submit and evidence that a transaction was accepted. This distinction is essential when explaining the prototype or reviewing a failed workflow.

Table G.1: Technical Terms Used in the Report

Term	Meaning in Prajatantra
Enrollment	Creation of a stored face descriptor from multiple usable images
Face descriptor	The validated 128-dimensional vector used for comparison
Eye aspect ratio	A landmark-based measure of eye opening used by the blink rule
Challenge	A voter-bound, expiring identifier for one verification attempt
Authentication session	Signed session context identifying a user or administrator
Eligibility	The current approval and account conditions required for voting
Voting credential	A random, short-lived authorization associated with a voter and election
Credential digest	A hash representation used for private lookup or public replay checks
Relayer	The configured server-controlled account that submits vote transactions
Transaction receipt	The blockchain response used to assess an included transaction's status
Reconciliation	Comparison of chain records and private records after normal or interrupted processing
Presentation attack	An attempt to present substituted or artificial biometric input for evaluation

G.2 Boundaries of Common Claims

Authentication does not by itself establish eligibility. A transaction receipt does not prove that the preceding identity review was correct. A public tally does not establish ballot secrecy, and a threshold does not express an accuracy percentage. The report uses these distinctions when describing implemented behavior, planned evaluation and future research.

Source inspection establishes what the implementation expresses. Execution records establish what happened in a particular environment. Controlled evaluation is needed before generalizing either observation into a claim about reliability, biometric performance or suitability for a wider operational setting. Transaction hashes should be checked against available receipts and the configured network rather than treated as proof of success merely because a reference was returned. The documented commands describe the available validation procedure.